

ГКП НА ПХВ ВЫСШИЙ КОЛЛЕДЖ «ASTANA POLYTECHNIC»
АКИМАТА Г. АСТАНА

010000

06130100

К защите допускается
Зам директора по УР
Исмайлова А.Х.
«_____» _____ 2026г.

Веб-сервис планирования деловых встреч
ДИПЛОМНЫЙ ПРОЕКТ

(Пояснительная записка)

ДП.06130100.П-23-616.148.06.26.ПЗ

Дипломник	Номеровченко С.В.
(дата, подпись)	
Руководитель проекта	Попов Д.В.
(дата, подпись)	
Консультант по экономическому разделу	Абдраимова Г.Ф.
(дата, подпись)	
Консультант по охране труда	Огизбаева Г.Е.
(дата, подпись)	
Нормоконтроль	Абенова Г.Н.
(дата, подпись)	
Рецензент	Касаткина К.А.
(дата, подпись)	

Дата защиты _____

Оценка _____

Протокол № _____

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1. АНАЛИТИКО-ПОСТАНОВОЧНАЯ ЧАСТЬ.....	8
1.1 Характеристика предметной области	8
1.2 Анализ существующих решений.....	9
1.3 Постановка задачи	11
1.4 Описание ролей пользователей	12
1.5 Функциональные требования	14
1.6 Нефункциональные требования	15
1.7 Критерии качества	17
РАЗДЕЛ 2. ТЕОРЕТИКО-ПРОЕКТНАЯ ЧАСТЬ.....	19
2.1 Выбор и обоснование технологического стека.....	19
2.2 Архитектура системы	20
2.3 Проектирование базы данных	22
2.4 Описание API	25
2.5 Описание структуры Frontend	30
2.6 Локализация	31
2.7 Описание алгоритмов	33
РАЗДЕЛ 3. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И ОЦЕНКА РЕЗУЛЬТАТОВ	36
3.1 Среда разработки и инструменты	36
3.2 Структура проекта	36
3.3 Реализация ключевых модулей	40
3.4 Реализация базы данных	42
3.5 Развертывание и запуск.....	45
3.6 Реализация локализации	46
3.7 Описание пользовательских сценариев.....	46
3.8 Тестирование.....	47
3.9 Связь требований с реализацией	50
РАЗДЕЛ 4. ЭКОНОМИЧЕСКАЯ ЧАСТЬ	52
4.1 Исходные данные.....	52
4.2 Расчет затрат на разработку	53
4.3 Инфраструктурные расходы	53
4.4 Общие затраты на создание и эксплуатацию.....	54
4.5 Экономический эффект от внедрения	54
4.6 Срок окупаемости и рентабельность	55
РАЗДЕЛ 5. ОХРАНА ТРУДА, ТЕХНИКА БЕЗОПАСНОСТИ И БЕЗОПАСНОСТЬ ЭКСПЛУАТАЦИИ.....	56
5.1 Требования к рабочему месту.....	56
5.2 Режим труда и отдыха	57
5.3 Безопасность эксплуатации системы	57
5.4 Резервное копирование данных.....	59
5.5 Работа с тестовой средой	59
5.6 Защита персональных данных	60
ЗАКЛЮЧЕНИЕ	61
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	63

					ДП.06130100.П-23-616.148.06.26.ПЗ			
Изм	Лист	№ докум.	Подпись	Дата				
Дипломник		Номеровченко С.В.			Веб-сервис планирования деловых встреч	Лит.	Лист	Листов
Руководит.		Попов Д.В.					4	98
н.Контр.		Абендова Г.Н.				ASTANA POLYTECHNIC		
Рецензент		Касаткина К.А.						
Зав. отдел.		Баяндина А.К.						

ПРИЛОЖЕНИЯ.....	65
ПРИЛОЖЕНИЕ А.....	66
ПРИЛОЖЕНИЕ Б	75
ПРИЛОЖЕНИЕ В	78
ПРИЛОЖЕНИЕ Г	88
ПРИЛОЖЕНИЕ Д.....	91
ПРИЛОЖЕНИЕ Е	94

					ДП.06130100.П-23-616.148.06.26.ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата	Веб-сервис планирования деловых встреч	Лит.	Лис	Листов
Дипломник		Номеровченко С.В.						
Руководит.		Попов Д.В.					5	98
н.Контр.		Абенова Г.Н.				ASTANA POLYTECHNIC		
Рецензент		Касаткина К.А.						
Зав. отдел.		Баяндина А.К.						

ВВЕДЕНИЕ

Актуальность темы. Координация деловых встреч остается ручным процессом: время согласуют через мессенджеры, из-за чего теряется информация, возникают накладки и неэффективно используется рабочее время. Для организации из 50 человек (10 организаторов) потери достигают около 260 человеко-часов в год - около 1 170 000 тг годовой экономии при окупаемости 6,4 месяца (Раздел 4).

Тема соответствует стандарту специальности 06130100 «Программное обеспечение»: проект - законченное web-приложение на промышленных фреймворках (ASP.NET Core, React) с СУБД PostgreSQL.

Дополнительное требование - государственная языковая политика РК: интерфейс должен поддерживать казахский язык, которого (как и self-hosted развертывания) не дают универсальные инструменты (Google Calendar, Microsoft Teams, Calendly).

Цель проекта - разработать web-приложение управления встречами с ролями USER, ORGANIZER, ADMIN, проверкой конфликтов расписания, локализацией на казахский язык и документированным REST API.

Задачи проекта:

1. проанализировать предметную область и недостатки текущего процесса, оценить потери рабочего времени;
2. сравнить аналоги (Google Calendar, Microsoft Teams, Calendly) и обосновать невозможность их адаптации;
3. спроектировать архитектуру на основе Vertical Slice Architecture и CQRS (MediatR);
4. спроектировать схему базы данных (сущности, связи, индексы) и миграции EF Core;
5. реализовать backend на ASP.NET Core 9 с JWT-аутентификацией, RBAC и REST API (32 эндпоинта);
6. реализовать frontend на React 18 (Redux Toolkit, i18next, русский и казахский языки);
7. провести функциональное тестирование по 12 сценариям для всех ролей пользователей.

Объект исследования - процесс организации и согласования деловых встреч: планирование, приглашение участников, проверка доступности, изменения и фиксация результатов.

Предмет исследования - методы и средства разработки web-систем управления встречами: паттерны (Vertical Slice, CQRS), фреймворки backend и frontend, СУБД, аутентификация и проверка конфликтов расписания.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

Методы разработки. Vertical Slice Architecture и CQRS (MediatR); REST API связывает frontend и backend; JWT (HMAC-SHA256) - аутентификация, RBAC - авторизация; мягкое удаление сохраняет историю; конечный автомат из семи состояний ведет жизненный цикл встречи.

Методы исследования. Сравнительный анализ аналогов, моделирование предметной области (конечный автомат, ER-диаграмма), проектирование архитектуры, формализация требований (User Stories), тестирование по сценариям и экономическое моделирование - поэтапно по разделам 1-4.

Практическая значимость. Результат - полностью функциональная система (ASP.NET Core 9, React 18 / TypeScript, PostgreSQL 16), покрывающая полный цикл работы со встречами: участники, файлы, уведомления, пользователи. Интерфейс локализован на казахский язык. При затратах 630 000 тг годовая экономия - 1 170 000 тг (окупаемость 6,4 месяца); Vertical Slice позволяет добавлять функции без переработки кода.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

1. АНАЛИТИКО-ПОСТАНОВОЧНАЯ ЧАСТЬ

1.1 Характеристика предметной области

Деловые встречи - один из основных инструментов координации внутри организации. Руководители согласуют решения, технические специалисты синхронизируют ход задач, организаторы договариваются с подрядчиками. Но сам процесс организации нередко хаотичен.

Последствия такого подхода конкретны:

- конфликты расписания не обнаруживаются заранее: организатор не знает, что приглашенный уже занят в указанное время другой встречей;
- история встречи не сохраняется: невозможно восстановить, кто участвовал, когда встреча менялась, кто отказался;
- нет разграничения ответственности: любой участник чата может написать «встреча отменяется», и никакой проверки полномочий нет;
- статус участника не фиксируется: разница между «человек принял приглашение» и «человек просто не ответил» нигде не отражена;
- файлы и материалы к встрече хранятся отдельно, ссылки на них рассылаются вручную и быстро устаревают.

Для небольших команд это создает неудобства. Для организаций с несколькими десятками сотрудников - прямые потери: сорванные встречи, дублирующие бронирования переговорных комнат, потраченное время на ручное согласование расписания.

Разберем процесс по шагам. На этапе планирования организатор предлагает варианты времени перебором в чате; участники отвечают вразнобой, и при расхождении начинается цикл уточнений. Финальное приглашение отправляется без гарантии, что его увидели все. На этапе проведения часть участников не приходит, а после встречи решения остаются в личных заметках - общей базы знаний не формируется.

В организации с несколькими подразделениями проблема усугубляется: встречи не привязаны к структуре отделов, участники не видят занятость друг друга, дублирующие бронирования переговорных обнаруживаются только при накладке. Для администратора отсутствует централизованное управление учетными записями: удаление сотрудника не деактивирует его аккаунт и не перераспределяет созданные им встречи.

Предметная область системы охватывает четыре ключевых процесса:

1. Планирование встречи - создание записи с указанием формата (Offline, Online, Hybrid, Phone), даты, времени начала и окончания, места или ссылки.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						8
Изм.	Лист	№ докум.	Подпись	Дата		

2. Согласование участия - приглашение конкретных пользователей, фиксация их ответа (Pending / Accepted / Declined), отслеживание обязательных участников.
3. Управление жизненным циклом - переход встречи между статусами: Draft → Scheduled → AwaitingConfirmation → Confirmed / Rescheduled / Cancelled → Completed.
4. Администрирование - управление учетными записями, ролями, архивом удаленных пользователей, журналом уведомлений.

Все четыре процесса должны быть автоматизированы в единой web-системе с разграничением прав доступа по ролям.

Количественно оценить масштаб проблемы можно на типовом примере. В организации с 50 сотрудниками, где 10 организаторов проводят в среднем по 3 встречи в неделю, а ручное согласование занимает около 10 минут на встречу, потери рабочего времени составляют около 5 человеко-часов в неделю суммарно по всем организаторам только на координацию. За год это порядка 260 человеко-часов. Автоматизация процесса сокращает эти потери примерно до 1 минуты на встречу (заполнение формы, проверка доступности, отправка приглашения - все операции выполняются в едином интерфейсе).

В таблице 1.1 приведен детальный анализ текущего процесса для каждого участника, выявленные проблемы и ожидаемые изменения после внедрения системы.

Таблица 1.1 Анализ текущего процесса организации встреч

Участник	Что делает сейчас	Проблема	Что должно измениться
Организатор	Пишет в мессенджер, назначает время, отслеживает ответы вручную	Не видит конфликты расписания, теряет историю изменений, тратит время на повторную рассылку	Создаёт встречу через форму, проверяет занятость до приглашения, получает автоматические уведомления
Участник	Отвечает «ок» или «не могу» в общем чате	Ответ не фиксируется, статус участия не хранится, нет единого списка своих встреч	Принимает или отклоняет приглашение через интерфейс, видит все встречи в личном кабинете
Администратор	Управляет доступом вручную, не имеет инструментов контроля	Нет централизованного управления учётными записями, удалённые сотрудники остаются в системе	Управляет пользователями и ролями через UI, мягкое удаление с проверкой, журнал уведомлений

1.2 Анализ существующих решений

Прежде чем формулировать требования, проанализированы три распространенных инструмента: Google Calendar, Microsoft Teams и Calendly. Критерии сравнения выбраны

исходя из потребностей предметной области: ролевая модель, проверка конфликтов, работа с данными, локализация, способ развертывания, а также возможность адаптировать инструмент под корпоративную систему управления встречами без существенных доработок.

Google Calendar - облачный календарь с совместным планированием: создание событий, приглашения по email, подтверждение участия (Accept/Decline/Maybe) и базовая проверка доступности. Однако нет ролевой модели на уровне API, казахскоязычного интерфейса и self-hosted развертывания (данные хранятся в облаке Google); история уведомлений и мягкое удаление событий отсутствуют. API v3 не поддерживает обязательных участников, кастомные статусы встречи и двуязычные уведомления - их пришлось бы реализовывать во внешней надстройке, дублируя данные.

Microsoft Teams - корпоративная платформа с планировщиком, видеоконференциями и проверкой занятости через Outlook. Развертывание сложное: требует Microsoft 365 или собственного сервера Exchange. REST API (Microsoft Graph) требует OAuth2 с правами администратора организации, что усложняет интеграцию. Гибкий RBAC на уровне отдельных эндпоинтов, казахский язык и self-hosted развертывание отсутствуют; мягкое удаление встреч не предусмотрено.

Calendly - инструмент автоматического согласования времени через публичные ссылки; удобен для внешних встреч, но не подходит для внутреннего использования: нет управления пользователями (записаться может любой обладатель ссылки), ролевой модели, мягкого удаления, журнала уведомлений и self-hosted варианта; казахский язык не поддерживается. Calendly решает только выбор слота, не покрывая полный жизненный цикл встречи.

В таблице 1.2 приведено сравнение аналогов с разрабатываемой системой по критериям, непосредственно вытекающим из требований предметной области.

Таблица 1.2 Сравнение существующих решений с разрабатываемой системой

Критерий	Google Calendar	Microsoft Teams	Calendly	Разрабатываемая система
RBAC: роли Admin / Organizer / User	-	Частично	-	+
Проверка конфликтов расписания	+	+	+	+
Soft delete встреч и пользователей	-	-	-	+
Восстановление удаленных записей	-	-	-	+
Казахскоязычный интерфейс	-	-	-	+
Документированный REST API (Swagger)	-	Частично	-	+

Критерий	Google Calendar	Microsoft Teams	Calendly	Разрабатываемая система
Self-hosted / локальное развертывание	-	-	-	+
Журнал email-уведомлений в БД	-	-	-	+
State machine статусов (7 состояний)	-	-	-	+
Управление файлами встречи	-	+	-	+
Обязательные участники (IsRequired)	-	-	-	+

Из анализа следует: ни одно из рассмотренных решений не обеспечивает одновременно гибкую ролевую модель на уровне API, поддержку казахского языка, self-hosted развертывание и полный аудит-журнал уведомлений. Разрабатываемая система закрывает именно этот набор требований.

Критерии охватывают функциональные возможности (ролевая модель, проверка конфликтов, soft delete, конечный автомат, файлы, обязательные участники) и нефункциональные (локализация, документированное API, self-hosted развертывание, аудит-журнал). Разрабатываемая система имеет «+» по всем 11 критериям, так как каждый критерий включен в требования именно из-за его отсутствия в аналогах.

Комбинирование аналогов задачу не решает: Google Calendar можно дополнить внешним RBAC-прокси, Teams - кастомным language pack, но ни то, ни другое не дает журнала email-уведомлений в собственной БД или мягкого удаления с восстановлением. Вывод: для полного покрытия требований необходима разработка специализированной системы.

1.3 Постановка задачи

На основе анализа предметной области и недостатков существующих решений сформулирована задача разработки. Система предназначена для внутреннего использования в организации и не является коммерческим продуктом. Ожидаемое количество пользователей - до 50 человек, из которых до 10 - организаторы встреч. Технической основой выбрана full-stack архитектура на открытых технологиях.

Требуется создать web-систему управления встречами и согласованием участия, которая обеспечивает:

1. Создание и редактирование встреч с поддержкой четырех форматов проведения (Offline, Online, Hybrid, Phone) и валидацией обязательных реквизитов в зависимости от выбранного формата: физическое местоположение для Offline, ссылка на видеоконференцию для Online, оба параметра для Hybrid, контактные данные для Phone.

2. Управление жизненным циклом встречи через конечный автомат с семью состояниями: Draft, Scheduled, AwaitingConfirmation, Confirmed, Rescheduled, Cancelled, Completed. Часть переходов выполняется автоматически: при изменении времени статус Scheduled переходит в Rescheduled; когда все обязательные участники подтверждают участие, статус AwaitingConfirmation переходит в Confirmed.
3. Проверку конфликтов расписания до отправки приглашений - алгоритм пересечения временных интервалов проверяет участие пользователя в других встречах и его роль организатора.
4. Управление участниками: приглашение, фиксация ответа (Pending / Accepted / Declined), пометка обязательных участников (IsRequired), удаление из встречи.
5. Хранение и скачивание файлов встречи с ограничением по допустимым форматам.
6. Email-уведомления при приглашении и изменении встречи с сохранением каждого отправленного письма в журнале (таблица Messages).
7. Разграничение прав доступа через три роли (User, Organizer, Admin) и три политики авторизации (UserPolicy, ManagementPolicy, OnlyAdminPolicy).
8. Управление учетными записями (только для Admin): просмотр пользователей, мягкое удаление с проверкой на активные встречи, восстановление, смена ролей, принудительный сброс пароля.
9. Локализацию интерфейса на русский и казахский языки.
10. Воспроизводимое развёртывание всех компонентов (backend, frontend, база данных) на сервере по единому набору инструкций, без ручной настройки окружения.

Границы проекта. Система не включает: двустороннюю синхронизацию с внешними календарями (Google Calendar, Outlook); push-уведомления в браузере; мобильное приложение; повторяющиеся встречи (recurring meetings); интеграцию с мессенджерами (Slack, Telegram); аудио- и видеоконференции. Эти функции могут быть добавлены в следующих итерациях, но не являются обязательными для достижения цели проекта.

Архитектурные ограничения. Выбор full-stack архитектуры на открытых технологиях продиктован требованиями self-hosted развёртывания и нулевой стоимостью лицензий. ASP.NET Core 9 выбран за встроенную поддержку JWT-аутентификации, ролевого разграничения и EF Core; React 18 - за распространенность и поддержку TypeScript (строгая типизация на обеих сторонах); PostgreSQL 16 - за фильтрованные уникальные индексы и встроенные средства резервного копирования; запуск на одном сервере - как минимально достаточный способ совместной работы трёх компонентов (backend, frontend, база данных).

1.4 Описание ролей пользователей

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						12
Изм.	Лист	№ докум.	Подпись	Дата		

Система поддерживает три роли - три разных сценария использования.

Текущие сценарии работы и ожидаемые изменения по каждой роли приведены в таблице 1.3.

Таблица 1.3 Роли пользователей: текущий процесс, проблема, изменение

Роль	Текущий процесс	Проблема	Что меняет система
User	Получает сообщение в мессенджере, отвечает «ок» или игнорирует	Ответ нигде не зафиксирован, статус участия неизвестен	Фиксирует Accepted / Declined через API; видит список всех своих встреч и статус каждого приглашения
Organizer	Создает встречу в чате, отслеживает ответы участников вручную, пересылает файлы отдельными сообщениями	Не видит конфликты расписания, теряет историю изменений, не знает точного состава подтвердивших	Создает встречу через форму с валидацией, проверяет занятость участников до приглашения, получает автоматическое уведомление при переходе статуса в Confirmed
Admin	Управляет списком сотрудников вручную: добавляет в систему, меняет доступы	Нет централизованного управления - права назначаются неформально, уволенные сотрудники не деактивируются	Мягкое удаление с проверкой активных встреч; восстановление; смена ролей через UI; просмотр журнала всех email-уведомлений

Роль привязана к одной или нескольким политикам авторизации ASP.NET Core: User - только `UserPolicy` (просмотр своих встреч, ответ на приглашения); Organizer - `UserPolicy` + `ManagementPolicy` (создание и редактирование встреч, управление участниками); Admin - все три политики, включая `OnlyAdminPolicy` (управление пользователями, просмотр журнала сообщений). Разграничение проверяется на уровне middleware до выполнения бизнес-логики, что исключает случайный или намеренный доступ к запрещенным эндпоинтам.

Выделение трех ролей обосновано структурой типовой организации: рядовые сотрудники (User) только участвуют во встречах; организаторы (Organizer) - руководители подразделений и проект-менеджеры, создающие и изменяющие встречи; администратор (Admin) отвечает за учетные записи. Такое разделение минимизирует избыточные права.

Ключевые сценарии использования по ролям:

- User (рядовой сотрудник): получает приглашение на email; входит в систему; видит список своих встреч на `DashboardPage`; отвечает Accept или Decline на приглашение; просматривает детали встречи; скачивает прикрепленные файлы; редактирует свои данные в профиле.
- Organizer (организатор): все, что может User; создает встречу с выбором формата (Offline, Online, Hybrid, Phone); перед отправкой приглашений проверяет занятость

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						13
Изм.	Лист	№ докум.	Подпись	Дата		

участников; редактирует встречу; отменяет встречу; прикрепляет и удаляет файлы; видит статус каждого приглашенного участника; получает автоматическое уведомление, когда встреча подтверждена.

- Admin (администратор): все, что может Organizer; просматривает список всех пользователей; меняет роли; выполняет мягкое удаление и восстановление пользователей; просматривает журнал email-уведомлений; управляет ролью других администраторов.

Роли назначаются через таблицу `identity.UserRoles`. Один пользователь может иметь несколько ролей одновременно. При регистрации автоматически назначается роль User. Изменение ролей доступно только Admin через `PATCH /api/v1/users/{id}/roles`.

1.5 Функциональные требования

Функциональные требования сформулированы в формате User Story с критериями приемки, привязанными к конкретным эндпоинтам API. Каждая User Story описывает роль пользователя, его действие и ожидаемый результат. Критерий приемки определяет точное поведение системы, которое можно проверить через API или интерфейс.

Функциональные требования в формате User Story с критериями приемки приведены в таблице 1.4.

Таблица 1.4 Функциональные требования (User Stories и критерии приемки)

ID	User Story	Критерий приемки
US-01	Как Organizer, хочу создать встречу с форматом Online, указав ссылку на конференцию	POST <code>/api/v1/meetings</code> : если <code>Format=Online</code> и <code>MeetingLink</code> не указан - система возвращает HTTP 400. При корректных данных встреча создается со статусом Draft, участники получают email-уведомления
US-02	Как Organizer, хочу проверить занятость участников перед отправкой приглашений	POST <code>/api/v1/meetings/availability</code> : ответ содержит <code>allAvailable (bool)</code> и список <code>conflicts</code> с идентификатором конфликтующей встречи, ее временем и именем участника
US-03	Как User, хочу принять или отклонить приглашение на встречу	PATCH <code>/api/v1/meetings/{id}/participants/{userId}/respond</code> : доступен только самому приглашенному (<code>userId == currentUserId</code>); обновляет <code>InvitationStatus</code> и <code>ResponseAt</code>

ID	User Story	Критерий приемки
US-04	Как Admin, хочу удалить учетную запись пользователя без потери исторических данных	DELETE /api/v1/users/{id}: устанавливает IsDeleted=true, DeletedAt, DeletedBy. Физические данные остаются в БД. Запрос отклоняется, если у пользователя есть активные встречи как организатора
US-05	Как Admin, хочу изменить роль пользователя	PATCH /api/v1/users/{id}/roles: принимает массив ролей; вычисляет разницу с текущими ролями и применяет через UserManager
US-06	Как User, хочу работать с интерфейсом на казахском языке	Компонент LanguageSwitcher переключает язык через <code>i18n.changeLanguage('kk')</code> , все строки интерфейса обновляются немедленно; выбор сохраняется в localStorage
US-07	Как Organizer, хочу прикрепить документ к встрече	POST /api/v1/meetings/{id}/files (multipart/form-data): принимаются файлы форматов .pdf, .doc, .docx, .xls, .xlsx, .ppt, .pptx, .txt, .jpg, .jpeg, .png; файл сохраняется по пути <code>files/meetings/{meetingId}/{guid}_{originalName}</code>
US-08	Как User, хочу скачать прикрепленный к встрече файл	GET /api/v1/meetings/{id}/files/{fileId}/download: поддерживает Range-запросы для частичной загрузки
US-09	Как Organizer, хочу видеть, что встреча подтверждена автоматически	Когда все участники с IsRequired=true ответили Accepted и статус встречи AwaitingConfirmation - система автоматически переводит статус в Confirmed без ручного действия
US-10	Как Admin, хочу видеть журнал всех отправленных email-уведомлений	GET /api/v1/messages: пагинированный список с темой, телом письма, получателем, датой и ссылкой на связанную встречу

Функциональные требования прослеживаются до конкретных эндпоинтов API. Например, US-01 (создание встречи) использует POST /api/v1/meetings: FluentValidation проверяет формат и обязательные поля, затем CreateMeetingHandler проверяет занятость участников, создает запись и отправляет уведомления. US-09 (автоподтверждение) не требует отдельного эндпоинта - логика встроена в RespondToInvitationHandler и срабатывает, когда все обязательные участники ответили Accepted.

1.6 Нефункциональные требования

Помимо функциональных, есть нефункциональные требования - по безопасности, производительности, локализации, эксплуатации. К системе также предъявляются операционные требования: журналирование всех действий с возможностью аудита, автоматическое резервное копирование базы данных, воспроизводимость развертывания.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		15

Безопасность. Аутентификация - JWT с HMAC-SHA256; при каждом входе проверяется флаг IsDeleted: удаленный пользователь получает HTTP 406, что отличает деактивацию от неверного пароля. Авторизация построена на трех политиках (UserPolicy, ManagementPolicy, OnlyAdminPolicy), проверяемых на уровне middleware. Пароли хранятся как хэш через ASP.NET Core Identity; JWT-секрет передается через переменную окружения Auth_Key и не содержится в коде.

Воспроизводимость развертывания. Все три компонента (backend, frontend, база данных) запускаются аналогично и на сервере; миграции применяются автоматически при старте backend с механизмом повторных попыток (10 попыток с экспоненциальной задержкой).

Операционные требования. Серверные операции журналируются через ILogger ASP.NET Core с записью в stdout; уровень логирования задается переменной окружения. Для долгосрочного хранения логов рекомендуется внешний агрегатор (Loki, Elasticsearch), что выходит за границы проекта. Резервное копирование БД выполняется через pg_dump по расписанию cron (см. Приложение Д). Сбор метрик (Prometheus + Grafana) в текущей реализации не предусмотрен.

Критичное операционное требование - журнал email-уведомлений. Каждое отправленное письмо сохраняется в таблице Messages (тема, тело, получатель, дата, ссылка на встречу), что позволяет администратору проверить факт и содержание отправленного приглашения. В аналогах (Google Calendar, Microsoft Teams) такой возможности нет - история хранится только в почтовом ящике получателя.

Производительность. Запросы выполняются через EF Core с составными индексами: IX_Meetings_IsDeleted_Status ускоряет выборку встреч, IX_MeetingParticipants_MeetingId_UserId - проверку дублирующих приглашений. PostgreSQL 16 надежно обрабатывает конкурентные записи при одновременных ответах участников.

Масштабируемость. Backend и frontend работают в отдельных компонентах и масштабируются независимо: backend можно развернуть в нескольких экземплярах за reverse проху, frontend раздает статику и не требует синхронизации состояния (оно хранится в JWT и PostgreSQL). База данных остается единой точкой и при росте нагрузки может быть переведена на кластер с read-replica.

Локализация. Интерфейс поддерживает русский (по умолчанию) и казахский языки; выбор сохраняется в localStorage и определяется при первом посещении по navigator.language, переключение - без перезагрузки. Email-уведомления отправляются на двух языках одновременно.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						16
Изм.	Лист	№ докум.	Подпись	Дата		

Доступность API. Все эндпоинты документированы через OpenAPI (Swagger UI на localhost:7262/swagger), документация генерируется автоматически. Схема дополнена поддержкой Bearer-авторизации для тестирования защищенных эндпоинтов из браузера. Мониторинг - через GET /api/v1/health: эндпоинт проверяет подключение к PostgreSQL и при недоступности БД отвечает HTTP 503, что позволяет балансировщику исключить неисправный экземпляр.

В таблице 1.5 приведены нефункциональные требования в структурированном виде.

Таблица 1.5 Нефункциональные требования

Категория	Требование	Реализация
Безопасность	JWT HMAC-SHA256, проверка IsDeleted при входе, RBAC через политики	ASP.NET Core Identity + JwtBearer 9.0.13
Безопасность	Пароли только в виде хэша	PasswordHash через ASP.NET Identity
Безопасность	JWT-секрет не в коде	Auth_Key через переменную окружения
Развертывание	Запуск несколькими командами, автоматические миграции	локальное развертывание, ApplyMigrationsWithRetryAsync()
Производительность	Индексированные запросы к БД	Составные индексы в EF Core (DbContext.cs)
Локализация	Русский и казахский язык без перезагрузки	i18next 26.3.0 + react-i18next 17.0.8
Документация API	Swagger UI с поддержкой Bearer-токена	Swashbuckle.AspNetCore 9.0.6
Масштабируемость	Независимые	Локальная сеть

1.7 Критерии качества

По результатам анализа требований сформулированы критерии, по которым оценивается готовность системы:

- Полнота реализации требований. Каждое функциональное требование (US-01-US-10) считается выполненным, если его критерий приемки проходит проверку через API. Пропуск хотя бы одного критерия означает неготовность системы к эксплуатации.
- Отсутствие регрессии при добавлении новых фич. Архитектура Vertical Slice гарантирует, что добавление нового эндпоинта не требует изменения существующих классов - достаточно создать файл, имплементирующий IFeatureEndpoint.
- Воспроизводимость развертывания. Система считается воспроизводимой, если она запускается несколькими командами на чистой машине с установленной системой управления версиями (Git).

- Покрытие ролей тестами. Каждый сценарий тестирования (12 сценариев) охватывает как минимум одну из трех ролей (User, Organizer, Admin). Роль Admin проверяется не менее чем тремя сценариями.
- Языковая доступность. Интерфейс должен быть полностью переведен на казахский язык: отсутствие непереуведенных строк на страницах, доступных пользователю, обязательно.
- Валидация входных данных. Серверная валидация должна отклонять некорректные данные на уровне API до выполнения бизнес-логики. Ответ должен содержать код ошибки (BadRequest, ValidationError) и описание, понятное разработчику.
- Консистентность формата ответа. Все эндпоинты должны возвращать единый формат BaseApiResponse<T> с полями data, status, message. Исключения - только стандартные HTTP-ошибки middleware (401, 403, 404, 406).
- Аудируемость операций. Каждое изменение статуса встречи, смена роли пользователя, мягкое удаление и отправка email-уведомления должны быть зафиксированы в БД или логах с указанием времени и инициатора операции. Для email-уведомлений предусмотрена отдельная таблица Messages.
- Устойчивость к сбоям. При недоступности SMTP создание встречи не блокируется: ошибка отправки логируется, встреча сохраняется. Поле SentAt в таблице Messages исключает повторную отпauкку уведомлений при перезапуске.
- Консистентность данных. Ни одна операция не должна создавать встречу без организатора (ON DELETE RESTRICT), участника без встречи или сообщение без получателя (ON DELETE CASCADE). Ограничения целостности реализованы на уровне БД через внешние ключи и продублированы в MediatR-хендлерах.

Эти критерии применялись при проведении функционального тестирования (подраздел 3.8) и подтверждены протоколами.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						18
Изм.	Лист	№ докум.	Подпись	Дата		

РАЗДЕЛ 2. ТЕОРЕТИКО-ПРОЕКТНАЯ ЧАСТЬ

2.1 Выбор и обоснование технологического стека

Выбор технологий определялся тремя критериями: наличием готовых механизмов аутентификации и авторизации, поддержкой строгой типизации на backend и frontend, и возможностью автоматизированного развертывания без внешних зависимостей.

Для реализации пользовательского интерфейса рассматривались три популярных фреймворка: React 18, Angular 17 и Vue 3. Сравнение приведено в таблице 2.1.

Таблица 2.1 Сравнение frontend-фреймворков

Критерий	React 18	Angular 17	Vue 3
Экосистема управления состоянием	Redux Toolkit (официально)	NgRx (сложнее)	Pinia
Подход к типизации	TypeScript опционально, гибко	TypeScript обязателен, жестко	TypeScript опционально
Размер итогового бандла	Малый	Большой	Малый
Интеграция i18next	react-i18next (нативно)	ngx-translate	vue-i18n
Порог входа	Низкий	Высокий	Низкий
Выбор	+	-	-

React 18 выбран по совокупности факторов: Redux Toolkit дает предсказуемое управление состоянием с минимумом шаблонного кода, react-i18next нативно интегрируется с i18next, а TypeScript подключается опционально - без жестких ограничений, навязываемых Angular.

Для backend рассматривались ASP.NET Core 9, Node.js (Express) и Django. Сравнение приведено в таблице 2.2.

Таблица 2.2 Сравнение backend-фреймворков

Критерий	ASP.NET Core 9	Node.js (Express)	Django
Система типов	C# - строгая статическая	JS/TS - опционально	Python - динамическая
Identity + JWT из коробки	Да, через NuGet-пакеты	Нет, требует passport.js	Нет, требует django-rest-framework
ORM	EF Core 9 (нативно)	Prisma / TypeORM	Django ORM
CQRS / MediatR	MediatR 12.4.1	Ручная реализация	Нет
Minimal API	Да (с .NET 6+)	Нет	Нет
Выбор	+	-	-

ASP.NET Core 9 выбран за встроенный ASP.NET Core Identity (управление пользователями, хэширование паролей, роли) и MediatR (CQRS без шаблонного кода диспетчеризации); Minimal API позволяет регистрировать эндпоинты программно через reflection - основа паттерна IFeatureEndpoint.

Для хранения данных сравнивались PostgreSQL 16 и MySQL 8. Сравнение приведено в таблице 2.3.

Таблица 2.3 Сравнение систем управления базами данных

Критерий	PostgreSQL 16	MySQL 8
JSON-поля	+	+
Поддержка PostGIS (геоданные)	+	-
Изоляция таблиц через схемы (identity / public)	+	-
Зрелость Npgsql EF-провайдера	Высокая	Pomelo - менее зрелый
Фильтрованные уникальные индексы	+	Ограниченно
Выбор	+	-

PostgreSQL 16 выбран за поддержку схем: Identity-таблицы изолированы от бизнес-таблиц схемы public, что упрощает аудит и перенос бизнес-логики. Полный технологический стек системы приведен в таблице 2.4.

Таблица 2.4 Итоговый технологический стек

Уровень	Технология	Версия
Backend	ASP.NET Core / .NET	9.0
Backend - ORM	Entity Framework Core + Npgsql	9.0.13 / 9.0.4
Backend - аутентификация	ASP.NET Core Identity + JwtBearer	9.0.13
Backend - CQRS	MediatR	12.4.1
Backend - документация	Swashbuckle.AspNetCore	9.0.6
Backend - дискриминированные типы	OneOf + OneOf.SourceGenerator	3.0.271
Backend - JWT Claims	Duende.IdentityModel	7.0.0
Backend - аннотации кода	JetBrains.Annotations	2025.2.4
Frontend	React + React DOM	18.3.1
Frontend - роутинг	React Router DOM	6.30.1
Frontend - состояние	Redux Toolkit + React Redux	2.8.2 / 9.2.0
Frontend - i18n	i18next + react-i18next	26.3.0 / 17.0.8
Frontend - сборка	Vite	5.4.19
Frontend - стили	SASS	1.89.0
Frontend - типизация	TypeScript	5.8.3
СУБД	PostgreSQL	6
Инфраструктура	VPS (Linux)	2 vCPU / 4 GB
Web-сервер (frontend)	веб-сервер	-

2.2 Архитектура системы

Система построена по двухуровневой модели: независимые компоненты backend и frontend взаимодействуют через HTTP REST API; база данных работает в отдельном компоненте и доступна только backend-сервису.

Запрос от пользователя проходит следующий путь:

Общая схема взаимодействия компонентов системы представлена на рисунке 2.1.

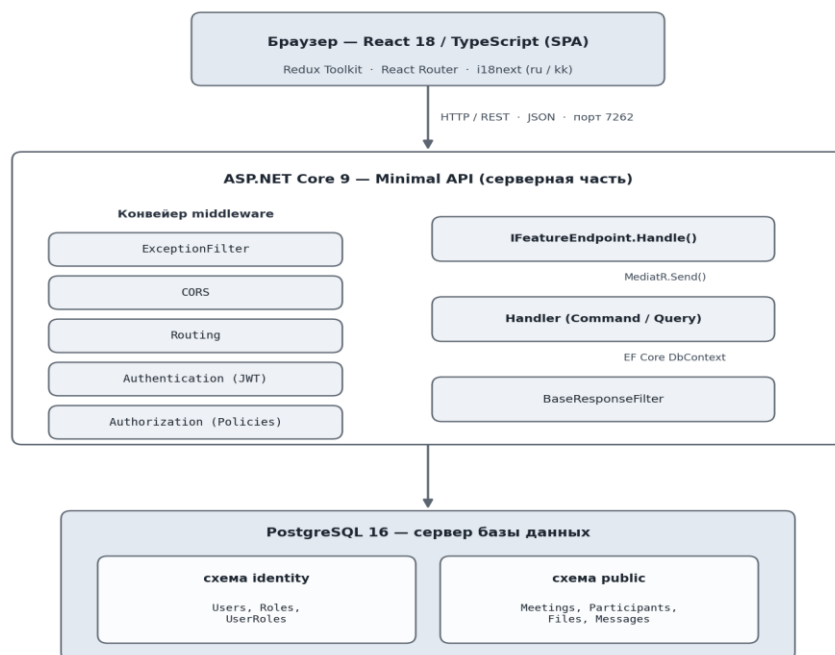


Рисунок 2.1 - Архитектура системы

Ответ всегда оборачивается в единый формат `BaseApiResponse<T>` через `BaseResponseFilter`, зарегистрированный как глобальный `IResultFilter`.

Backend организован по принципу вертикальных срезов. Любая функция системы - создание встречи, проверка конфликтов, ответ на приглашение - представлена одним файлом, который содержит:

- Request - входные данные (DTO или record-тип C#);
- Response - выходные данные;
- Handler : `IRequestHandler<Request, OneOf<Response, Error>>` - бизнес-логика;
- реализацию `IFeatureEndpoint` с методом `Map()` - регистрация маршрута.

При старте `Program.cs` сканирует сборку через `reflection` и автоматически регистрирует все классы, реализующие `IFeatureEndpoint` (листинг А.4); перечислять маршруты вручную не нужно.

CQRS реализован через `MediatR 12.4.1`: команды изменяют состояние, запросы читают данные, диспетчеризация - через `IMediator.Send()`. Pipeline `LoggingBehavior` логирует тип запроса, время выполнения и результат. Возвращаемые типы построены на `OneOf<TSuccess, TError>`, что позволяет обрабатывать ошибки через `pattern matching` без исключений.

`CurrentUserProvider` - `scoped`-сервис, извлекающий `userId` и список ролей из JWT-клеямов через `IHttpContextAccessor`; используется в `handlers` для проверки прав (например, является ли пользователь организатором встречи).

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						21
Изм.	Лист	№ докум.	Подпись	Дата		

EmailNotificationService при создании или изменении встречи формирует двуязычный HTML-шаблон письма (русский и казахский), сохраняет запись в таблицу Messages и отправляет письмо через SmtпClient; ошибки отправки логируются, но не прерывают основной процесс.

Frontend - SPA на React 18 с маршрутизацией через React Router DOM 6. Защита маршрутов - компонент ProtectedRoute, проверяющий наличие валидного JWT в localStorage и соответствие роли пользователя требуемой.

Глобальное состояние разбито на три Redux-слайса:

- authSlice - JWT-токен, данные текущего пользователя, роли, статус аутентификации. При инициализации приложения вызывается hydrateSessionFromStorage(), восстанавливающий сессию из localStorage без повторного запроса к API.
- sessionsSlice - список встреч, выбранная встреча, участники, результат проверки доступности, параметры пагинации.
- uiSlice - очередь toast-уведомлений, флаги глобального загрузчика.

HTTP-клиент (httpClient.ts) автоматически подставляет Bearer-токен в заголовок Authorization для каждого запроса.

2.3 Проектирование базы данных

База данных meetings_db организована в двух схемах PostgreSQL: identity (таблицы ASP.NET Core Identity) и public (бизнес-сущности). Разделение позволяет обновлять Identity-инфраструктуру независимо от бизнес-логики.

Перечень сущностей базы данных и их назначение приведены в таблице 2.5.

Таблица 2.5 Сущности базы данных

Сущность	Схема	Ключевые поля	Связи
Users	identity	Id (PK, uuid), Name (NOT NULL), Age (nullable), RegistrationDate, IsDeleted, DeletedAt, DeletedBy	→ UserRoles, → MeetingFiles, → Messages
Roles	identity	Id (PK, uuid), Name, NormalizedName (UNIQUE)	← UserRoles
UserRoles	identity	UserId (PK, FK→Users), RoleId (PK, FK→Roles)	M:N Users↔Roles
RoleClaims	identity	Id (PK, int), RoleId (FK→Roles), ClaimType, ClaimValue	→ Roles (Cascade)
UserClaims	identity	Id (PK, int), UserId (FK→Users), ClaimType, ClaimValue	→ Users (Cascade)
UserLogins	identity	LoginProvider (PK), ProviderKey (PK), UserId (FK→Users)	→ Users (Cascade)
UserTokens	identity	UserId (PK, FK→Users), LoginProvider (PK), Name (PK)	→ Users (Cascade)

Сущность	Схема	Ключевые поля	Связи
Meetings	public	Id (PK, uuid), Title (varchar 255), Status (int), Format (int), OrganizerId (FK), IsDeleted, CreatedAt, UpdatedAt	← MeetingParticipants, ← MeetingFiles
MeetingParticipants	public	Id (PK, uuid), MeetingId (FK, Cascade), UserId (FK, Cascade), InvitationStatus (int), IsRequired (bool), InvitedAt, ResponseAt, Comment	M:N Meeting↔User
MeetingFiles	public	Id (PK, uuid), MeetingId (FK, Cascade), FileName (varchar 255), FilePath (varchar 500), FileType (varchar 100), UploadedById (FK, Restrict), UploadedAt	N:1 Meeting
Messages	public	Id (PK, uuid), RecipientId (FK, Cascade), Subject (varchar 255), Body (text), SentAt, MeetingId (FK nullable, SetNull)	N:1 User, N:1 Meeting

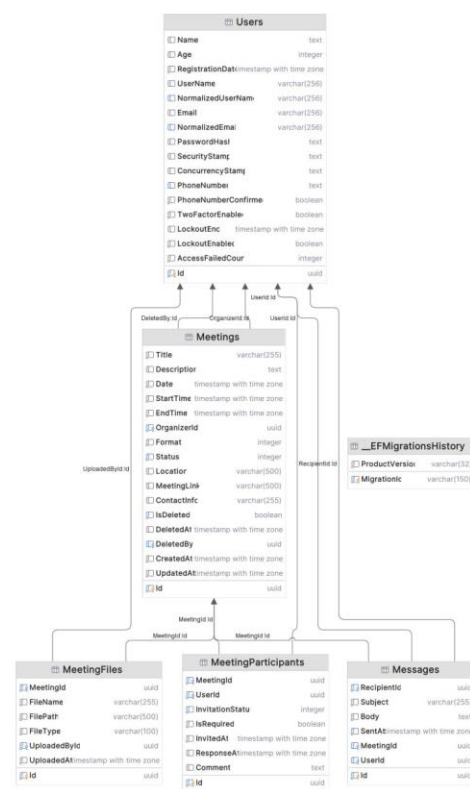
Сущность User расширяет IdentityUser<Guid> полями Name, Age, RegistrationDate и тремя полями мягкого удаления. Сущность Meeting хранит Location, MeetingLink и ContactInfo как опциональные - заполняется только то, которое соответствует выбранному формату.

Между сущностями установлены следующие связи:

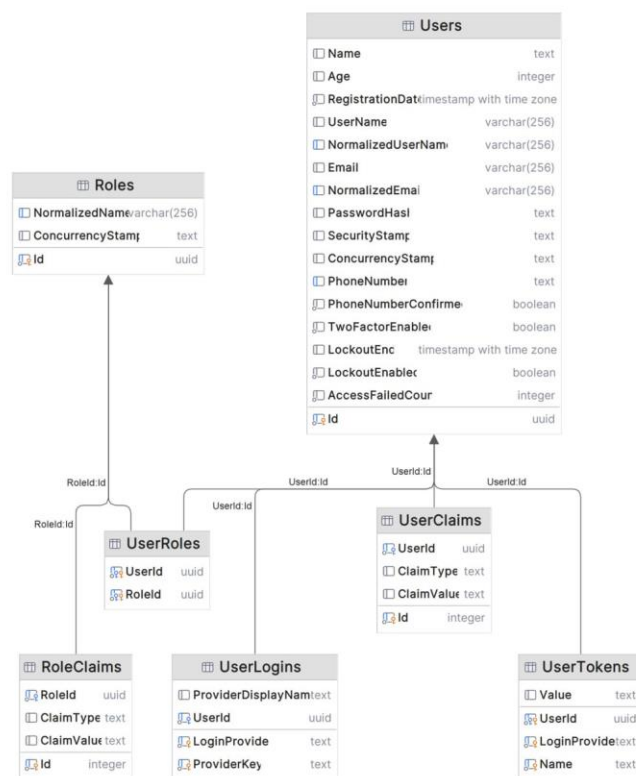
- User → Meeting (1:N через OrganizerId): у каждой встречи ровно один организатор. Удаление организатора запрещено, пока у него есть активные встречи (ON DELETE RESTRICT).
- Meeting → MeetingParticipant (1:N, ON DELETE CASCADE): при удалении встречи все записи об участниках удаляются автоматически.
- User → MeetingParticipant (1:N, ON DELETE CASCADE): при удалении пользователя его записи участника удаляются.
- Meeting → MeetingFile (1:N, ON DELETE CASCADE): файлы встречи удаляются вместе с ней.
- User → MeetingFile (через UploadedById, ON DELETE RESTRICT): нельзя удалить пользователя, пока за ним числятся загруженные файлы.
- User → Message (1:N, ON DELETE CASCADE): при удалении пользователя его журнал уведомлений очищается.
- Meeting → Message (1:N, ON DELETE SET NULL): при удалении встречи ссылка MeetingId в уведомлениях обнуляется, сами записи сохраняются.

Схема базы данных в виде ER-диаграммы представлена на рисунке 2.2.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		23



а) Бизнес-сущности (схема public)



б) Подсистема Identity (схема identity)

Рисунок 2.2 - ER-диаграмма базы данных

Все перечисляемые типы хранятся в БД как целые числа (int). Их расшифровка:

MeetingStatus: Draft = 0, Scheduled = 1, AwaitingConfirmation = 2, Confirmed = 3, Rescheduled = 4, Cancelled = 5, Completed = 6.

MeetingFormat: Offline = 0, Online = 1, Hybrid = 2, Phone = 3.

InvitationStatus: Pending = 0, Accepted = 1, Declined = 2.

Индексы сформированы исходя из типичных паттернов запросов. В таблице 2.6 перечислены все нестандартные индексы.

Таблица 2.6 Индексы базы данных

Таблица	Индекс	Тип	Назначение
identity.Users	IX_Users_IsDeleted	Обычный	Фильтрация активных пользователей
identity.Users	EmailIndex	Уникальный	Уникальность email пользователей

Таблица	Индекс	Тип	Назначение
identity.Users	IX_Users_PhoneNumber	Уникальный фильтрованный (PhoneNumberConfirmed=true)	Уникальность подтвержденных телефонов
Meetings	IX_Meetings_OrganizerId	Обычный	Выборка встреч по организатору
Meetings	IX_Meetings_Status	Обычный	Фильтрация по статусу
Meetings	IX_Meetings_IsDeleted_Status	Составной	Основная выборка: IsDeleted=false + фильтр статуса
MeetingParticipants	IX_MeetingParticipants_MeetingId_UserId	Уникальный составной	Исключение дублирующих приглашений
MeetingParticipants	IX_MeetingParticipants_InvitationStatus	Обычный	Фильтрация по статусу приглашения
MeetingFiles	IX_MeetingFiles_MeetingId	Обычный	Выборка файлов по встрече
Messages	IX_Messages_RecipientId	Обычный	Выборка уведомлений по получателю
Messages	IX_Messages_SentAt	Обычный	Сортировка журнала по дате

Композитный индекс IX_Meetings_IsDeleted_Status ключевой для производительности: почти все запросы к Meetings фильтруют по IsDeleted = false и затем по Status. Уникальный IX_MeetingParticipants_MeetingId_UserId гарантирует отсутствие дублирующих приглашений на уровне БД. Фильтрованные уникальные индексы (EmailIndex, IX_Users_PhoneNumber) индексируют только подходящие строки, уменьшая размер индекса.

2.4 Описание API

Все эндпоинты расположены под базовым путем /api/v1/. Каждый ответ обрабатывается через BaseResponseFilter в формат:

```
{
  "data": { ... },
  "status": "Ok"
}
```

Ошибочные ответы содержат "status": "Error" и описание в поле "data". Авторизация управляется тремя политиками: UserPolicy (любой аутентифицированный), ManagementPolicy (Admin или Organizer), OnlyAdminPolicy (только Admin).

Полный перечень эндпоинтов REST API с политиками доступа приведен в таблице 2.7.

Таблица 2.7 REST API эндпоинты системы

Метод	Маршрут	Политика	Описание
Аутентификация			
POST	/api/v1/auth/sign-up	Публичный	Регистрация; автоматически назначает роль User
POST	/api/v1/auth/login	Публичный	Вход; возвращает JWT. Если IsDeleted=true - HTTP 406
POST	/api/v1/auth/logout	UserPolicy	Stateless-выход (токен не инвалидируется на сервере)
POST	/api/v1/auth/change-password	UserPolicy	Смена пароля текущим пользователем
POST	/api/v1/auth/reset-password	OnlyAdminPolicy	Принудительный сброс пароля администратором
Встречи			
GET	/api/v1/meetings	UserPolicy	Список встреч с пагинацией и фильтром по статусу. Admin видит все; остальные - только свои (организатор или участник)
POST	/api/v1/meetings	UserPolicy	Создание встречи. Валидация обязательных полей по формату. Статус по умолчанию - Draft

Метод	Маршрут	Политика	Описание
GET	/api/v1/meetings/{id}	UserPolicy	Детали встречи: поля + participantCount + fileCount
PATCH	/api/v1/meetings/{id}	UserPolicy	Частичное обновление. Только организатор или Admin. При изменении времени у Scheduled-встречи автоматический переход → Rescheduled
DELETE	/api/v1/meetings/{id}	UserPolicy	Мягкое удаление. Только организатор или Admin
POST	/api/v1/meetings/availability	UserPolicy	Проверка конфликтов расписания для списка участников
GET	/api/v1/meeting-formats	ManagementPolicy	Список значений enum MeetingFormat для форм UI
GET	/api/v1/meeting-statuses	ManagementPolicy	Список значений enum MeetingStatus для форм UI
Файлы встреч			
GET	/api/v1/meetings/{id}/files	UserPolicy	Список прикрепленных файлов

Метод	Маршрут	Политика	Описание
POST	/api/v1/meetings/{id}/files	ManagementPolicy	Загрузка файла (multipart/form-data). Только организатор или Admin. Допустимые форматы: .pdf, .doc, .docx, .ppt, .pptx, .xls, .xlsx, .txt, .jpg, .jpeg, .png
DELETE	/api/v1/meetings/{id}/files/{fileId}	ManagementPolicy	Удаление файла из БД и с диска
GET	/api/v1/meetings/{id}/files/{fileId}/download	UserPolicy	Скачивание файла. Поддерживает Range-запросы
Участники			
GET	/api/v1/meetings/{id}/participants	UserPolicy	Список участников. Query-параметр status фильтрует по InvitationStatus
POST	/api/v1/meetings/{id}/participants	UserPolicy	Приглашение участников (только организатор). Дублирующие приглашения игнорируются. Email-уведомление каждому
DELETE	/api/v1/meetings/{id}/participants/{userId}	UserPolicy	Удаление участника. Только организатор

Метод	Маршрут	Политика	Описание
PATCH	/api/v1/meetings/{id}/participants/{userId}/respond	UserPolicy	Ответ на приглашение (Accepted / Declined). Только сам приглашенный (userId == currentUserId)
Пользователи			
GET	/api/v1/user	UserPolicy	Данные текущего пользователя, декодированные из JWT
GET	/api/v1/users	OnlyAdminPolicy	Список активных пользователей с пагинацией и поиском
GET	/api/v1/users/deleted	OnlyAdminPolicy	Список удаленных пользователей (IsDeleted=true)
DELETE	/api/v1/users/{id}	OnlyAdminPolicy	Мягкое удаление. Блокируется при наличии активных встреч или загруженных файлов. Нельзя удалить себя
PATCH	/api/v1/users/{id}/restore	OnlyAdminPolicy	Восстановление удаленного пользователя
GET	/api/v1/users/{id}/roles	OnlyAdminPolicy	Получение текущих ролей пользователя
PATCH	/api/v1/users/{id}/roles	OnlyAdminPolicy	Замена набора ролей. Тело: { "roles": ["Organizer"] }

Метод	Маршрут	Политика	Описание
GET	/api/v1/roles	OnlyAdminPolicy	Список всех ролей системы
GET	/api/v1/organizer/users	ManagementPolicy	Список активных пользователей для выбора участников при создании встречи
Сообщения			
GET	/api/v1/messages	ManagementPolicy	Журнал email-уведомлений с пагинацией
UI			
GET	/api/v1/ui/select-options	UserPolicy	Enum-опции для выпадающих списков: форматы встреч, статусы, направления сортировки, размеры страниц

2.5 Описание структуры Frontend

Исходный код frontend расположен в директории frontend/src/. Структура организована по функциональным группам, а не по типу файлов. Директории разбиты по слоям приложения.

Структура директорий frontend приведена в таблице 2.8.

Таблица 2.8 Структура директорий frontend/src/

Директория	Файлы	Назначение
pages/auth/	LoginPage.tsx, RegisterPage.tsx	Страницы входа и регистрации. Публичный доступ
pages/dashboard/	DashboardPage.tsx	Главная панель. Содержимое зависит от роли пользователя
pages/sessions/	SessionsPage.tsx, SessionDetailPage.tsx	Список встреч и детальная страница

Директория	Файлы	Назначение
pages/profile/	ProfilePage.tsx	Профиль текущего пользователя, смена пароля
pages/admin/	AdminDashboardPage.tsx, AdminUsersPage.tsx, AdminDeletedUsersPage.tsx, AdminMessagesPage.tsx	Административные страницы. Доступны только Admin
components/layout/	AppShell.tsx, Header.tsx, PageSection.tsx	Каркас приложения: шапка, навигация, обертка контента
components/meetings/	MeetingCard.tsx, MeetingForm.tsx, ParticipantsPanel.tsx, ParticipantsDropdown.tsx	Компоненты работы со встречами
components/ui/	Button, Input, Select, Textarea, Card, Badge, ConfirmDialog, EmptyState, GlobalLoader, Spinner, ToastViewport	Переиспользуемые UI-примитивы
components/i18n/	LanguageSwitcher.tsx	Переключатель языка (ru / kk)
store/slices/	authSlice.ts, sessionsSlice.ts, uiSlice.ts	Redux-состояние: авторизация, встречи, UI
http/	httpClient.ts, authService.ts, sessionsService.ts, usersService.ts, messagesService.ts, uiOptionsService.ts	HTTP-клиенты с JWT interceptor
router/	index.tsx, ProtectedRoute.tsx	Маршруты приложения, защита по роли
hooks/	useAuth.ts, useToast.ts, useUiSelectOptions.ts	Переиспользуемые React-хуки
utils/	jwt.ts, format.ts, meetingLabels.ts, userLabels.ts, profile.ts	Утилиты: работа с JWT, форматирование дат, метки enum
types/	index.ts	TypeScript-интерфейсы: CurrentUser, Meeting, MeetingParticipant и др.
styles/	global.scss, variables.scss, mixins.scss	Глобальные стили, SCSS-переменные, миксины

`ProtectedRoute` проверяет наличие токена в `localStorage`, декодирует его через `jwt.ts` и сверяет роль с требуемой для маршрута; при несоответствии перенаправляет на `/auth/login`. Это единственная точка контроля доступа на стороне frontend.

2.6 Локализация

Интерфейс системы поддерживает два языка: русский (ru) и казахский (kk). Локализация реализована через библиотеку `i18next 26.3.0` совместно с `react-i18next 17.0.8`.

При инициализации язык определяется через `detectLanguage()` (листинг А.6): восстанавливается из `localStorage` или определяется по `navigator.language`. `LanguageSwitcher`

вызывает `il8n.changeLanguage(lang)`, после чего компоненты с хуком `useTranslation()` перерисовываются без перезагрузки. Email-уведомления содержат текст на обоих языках в одном письме.

Примеры ключей локализации интерфейса (ru/kk) приведены в таблице 2.9.

Таблица 2.9 Примеры ключей локализации (ru / kk)

Ключ	Русский	Казахский	Комментарий
<code>common.appName</code>	Система управления встречами	Кездесулерді басқару жүйесі	Название приложения в шапке и заголовке вкладки браузера
<code>nav.meetings</code>	Встречи	Кездесулер	Пункт главного меню навигации
<code>auth.loginButton</code>	Войти	Кіру	Кнопка отправки формы входа
<code>meeting.status.Draft</code>	Черновик	Бастапқы нұсқа	Статус встречи: черновик, приглашения еще не разосланы
<code>meeting.status.Scheduled</code>	Запланирована	Жоспарланған	Статус встречи: время назначено, приглашения отправлены
<code>meeting.status.AwaitingConfirmation</code>	Ожидает подтверждения	Растауды күтуде	Статус встречи: ожидает подтверждения участников
<code>meeting.status.Confirmed</code>	Подтверждена	Расталған	Статус встречи: подтверждена обязательными участниками
<code>meeting.status.Rescheduled</code>	Перенесена	Қайта жоспарланған	Статус встречи: время изменено после планирования
<code>meeting.status.Cancelled</code>	Отменена	Бас тартылған	Статус встречи: отменена организатором
<code>meeting.status.Completed</code>	Завершена	Аяқталған	Статус встречи: завершена (состоялась)
<code>meeting.format.Offline</code>	Офлайн	Офлайн	Формат встречи: очно, по физическому адресу

Ключ	Русский	Казахский	Комментарий
meeting.format.Online	Онлайн	Онлайн	Формат встречи: онлайн, по ссылке на конференцию
meeting.format.Hybrid	Гибрид	Гибрид	Формат встречи: гибридный (очно и онлайн)
meeting.format.Phone	Телефон	Телефон	Формат встречи: телефонный звонок
meeting.invitation.Pending	Ожидает ответа	Жауап күтілуде	Статус приглашения: участник еще не ответил
meeting.invitation.Accepted	Принято	Қабылданды	Статус приглашения: участник принял приглашение
meeting.invitation.Declined	Отклонено	Қабылданбады	Статус приглашения: участник отклонил приглашение
roles.Admin	Администратор	Әкімші	Роль: администратор системы
roles.Organizer	Организатор	Ұйымдастырушы	Роль: организатор встреч
roles.User	Пользователь	Пайдаланушы	Роль: рядовой пользователь
sessions.allAvailable	Все участники свободны	Барлық қатысушылар бос	Сообщение: все участники свободны в выбранное время
sessions.conflictsFound	Найдены конфликты	Қақтығыстар табылды	Сообщение: обнаружены конфликты расписания

2.7 Описание алгоритмов

Проверка доступности участников выполняется эндпоинтом `POST /api/v1/meetings/availability` через обработчик `CheckAvailabilityQuery`. Задача алгоритма - для каждого пользователя из переданного списка найти все активные встречи, временной интервал которых пересекается с запрашиваемым.

Условие пересечения интервалов $[A_start, A_end]$ и $[B_start, B_end]$: $A_start < B_end$ AND $A_end > B_start$. Для каждого `userId` алгоритм находит встречи, где пользователь - участник или организатор, время пересекается с запрашиваемым, статус отличен от `Cancelled` и `Draft`, `IsDeleted` = false.

Полная реализация на EF Core приведена в листинге A.2. Если для хотя бы одного пользователя обнаружен конфликт, ответ содержит `allAvailable: false` и список объектов `ConflictInfo` с идентификатором и временем конфликтующей встречи.

Жизненный цикл встречи управляется конечным автоматом с семью состояниями. Часть переходов инициируется пользователем вручную, часть - автоматически системой.

Диаграмма состояний жизненного цикла встречи представлена на рисунке 2.3.

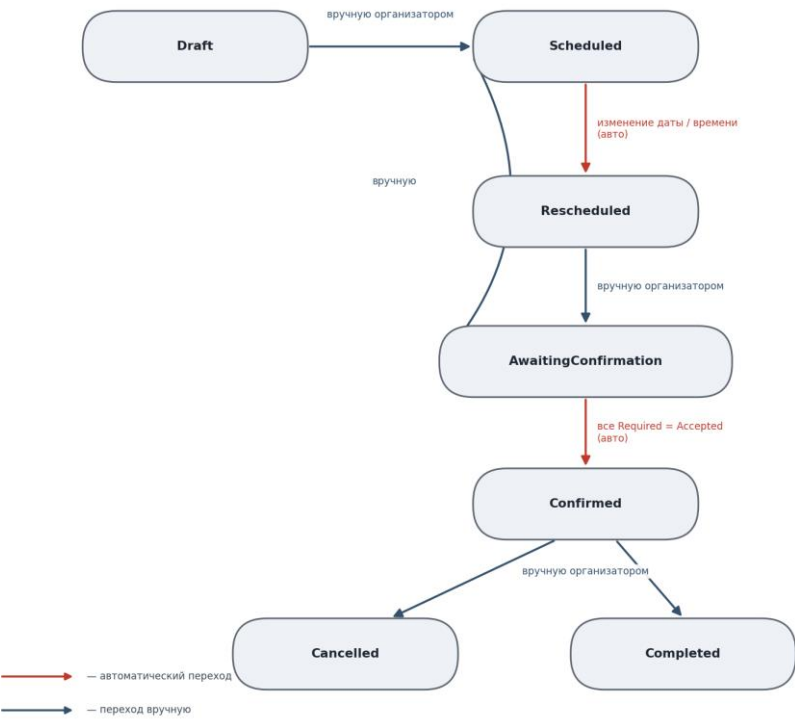


Рисунок 2.3 - Конечный автомат статусов встречи

Автоматические переходы:

- Scheduled → Rescheduled: срабатывает в UpdateMeetingCommand, если изменены поля Date, StartTime или EndTime и текущий статус встречи равен Scheduled.
- AwaitingConfirmation → Confirmed: срабатывает в RespondToInvitationCommand после каждого ответа Accepted. Система проверяет: все ли участники с IsRequired = true ответили Accepted. Если да - статус встречи автоматически обновляется до Confirmed.

Физическое удаление записей в системе не применяется. Вместо этого на сущностях Meeting и User установлены три поля:

- IsDeleted (bool, default false) - флаг удаления;
- DeletedAt (DateTime?, nullable) - момент удаления;
- DeletedBy (Guid?, nullable, FK → User) - идентификатор администратора, выполнившего удаление.

При запросах списков к этим сущностям всегда добавляется фильтр WHERE IsDeleted = false. При мягком удалении пользователя дополнительно проверяется, что у него нет

активных встреч в роли организатора и нет загруженных файлов; нарушение любого условия возвращает HTTP 400.

Восстановление выполняется обратной операцией: `IsDeleted = false, DeletedAt = null, DeletedBy = null`.

Процесс аутентификации охватывает пять шагов.

1. Клиент отправляет `POST /api/v1/auth/login` с телом `{ "email": "...", "password": "..." }`.
2. `LoginCommand.Handler` находит пользователя по email через `UserManager`.
3. Проверяется флаг `IsDeleted`: если `true` - возвращается HTTP 406 (Not Acceptable). Это намеренный выбор кода: 401 означает «неверные учетные данные», 406 - «аккаунт деактивирован».
4. Верифицируется пароль через `SignInManager.CheckPasswordSignInAsync`.
5. Формируется JWT-токен: метод `GenerateJwtToken` (листинг A.1) создает токен с клеймами `sub (userId)`, `iat (время выпуска)` и `role (список ролей)`. Алгоритм подписи - HMAC-SHA256, ключ передается через переменную окружения `Auth__Key`. Срок жизни токена - 5 часов.

После получения токена frontend сохраняет его в `localStorage` и добавляет заголовок `Authorization: Bearer <token>` к каждому запросу. Backend принимает токен также через query-параметр `access_token` (например, для прямых ссылок на скачивание файлов). `JwtBearerMiddleware` верифицирует подпись и извлекает клеймы, доступные через `CurrentUserProvider`.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						35
Изм.	Лист	№ докум.	Подпись	Дата		

РАЗДЕЛ 3. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И ОЦЕНКА РЕЗУЛЬТАТОВ

3.1 Среда разработки и инструменты

Разработка велась на персональном компьютере под управлением ОС Windows 11. Для backend-части использовалась среда разработки Visual Studio 2022; для frontend и работы с конфигурационными файлами - Visual Studio Code. Все внешние сервисы (база данных, backend-сервер при локальном тестировании) запускались как отдельные локальные сервисы. Используемые инструменты разработки и их версии приведены в таблице 3.1.

Таблица 3.1 Инструменты разработки и их версии

Инструмент	Версия	Назначение
Visual Studio 2022	17.x	Разработка backend (C#, ASP.NET Core)
Visual Studio Code	1.9x	Разработка frontend (TypeScript, React), конфигурация
.NET SDK	9.0	Компиляция и запуск backend
Node.js	20+	Управление зависимостями frontend, Vite dev-server
PostgreSQL	16 (отдельный сервис)	СУБД - только через компоненты, не напрямую
Swagger UI	-	Тестирование API (localhost:7262/swagger)

Для тестирования эндпоинтов API применялся Swagger UI, встроенный в backend через Swashbuckle.AspNetCore 9.0.6. Это позволяло проверять все маршруты непосредственно в браузере без сторонних инструментов. BearerSecuritySchemeTransformer добавляет в интерфейс Swagger поле для ввода JWT-токена, после чего все защищенные эндпоинты становятся доступны для тестирования.

3.2 Структура проекта

Проект разделен на две независимые части - backend и frontend - каждая из которых запускается как самостоятельный сервис.

backend/ ├─ TemplateProject/ │ └─ Program.cs миграции │ └─ AuthorizationPolicy.cs policy, OnlyAdminPolicy │ └─ IFeatureEndpoint.cs │ └─ TemplateProject.sln │ └─ TemplateProject.csproj │ └─ Domain/ │ └─ User.cs Deleted, ... │ └─ Role.cs n/User/Organizer │ └─ UserRole.cs │ └─ Meeting.cs │ └─ MeetingParticipant.cs	- точка входа: DI, middleware, seed, - константы UserPolicy, ManagementPo - интерфейс регистрации эндпоинтов - файл решения Visual Studio - зависимости проекта - доменные сущности и перечисления - IdentityUser<Guid> + Name, Age, Is - IdentityRole<Guid>, константы Admi - связь пользователь-роль - сущность встречи со всеми полями - участник встречи, InvitationStatus
--	--

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						36
Изм.	Лист	№ докум.	Подпись	Дата		

		, IsRequired	MeetingFile.cs	- прикрепленный файл
			Message.cs	- запись email-уведомления
eted			MeetingStatus.cs	- enum: Draft, Scheduled, ..., Compl
			MeetingFormat.cs	- enum: Offline, Online, Hybrid, Pho
ne			InvitationStatus.cs	- enum: Pending, Accepted, Declined
t), конфигурация схем			DataAccess/	
			DatabaseContext.cs	- EF Core контекст (IdentityDbContext)
осов			Features/	
			LoggingBehavior.cs	- MediatR pipeline: логирование запр
			Meetings/	
			CreateMeetingCommand.cs	- POST /api/v1/meetings
			UpdateMeetingCommand.cs	- PATCH /api/v1/meetings/{id}
			DeleteMeetingCommand.cs	- DELETE /api/v1/meetings/{id}
			GetMeetingQuery.cs	- GET /api/v1/meetings/{id}
			ListMeetingsQuery.cs	- GET /api/v1/meetings
			CheckAvailabilityQuery.cs	- POST /api/v1/meetings/availability
			UploadMeetingFileCommand.cs	- POST /api/v1/meetings/{id}/files
			DeleteMeetingFileCommand.cs	- DELETE файла
			DownloadMeetingFileQuery.cs	- GET скачивание файла
			ListMeetingFilesQuery.cs	- GET список файлов
			GetMeetingFormatsQuery.cs	- GET форматы
			GetMeetingStatusesQuery.cs	- GET статусы
			Participants/	
			InviteParticipantsCommand.cs	
			RemoveParticipantCommand.cs	
			RespondToInvitationCommand.cs	
			ListParticipantsQuery.cs	
			Messages/	
			ListMessagesQuery.cs	
			User/	
			LoginCommand.cs	
			SignUpCommand.cs	
			LogoutCommand.cs	
			ChangePasswordCommand.cs	
			ResetPasswordCommand.cs	
			Users/	
			ListUsersQuery.cs	
			ListDeletedUsersQuery.cs	
			GetUserQuery.cs	
			GetUserRolesQuery.cs	
			GetUsersForOrganizerQuery.cs	
			DeleteUserCommand.cs	
			RestoreUserCommand.cs	
			UpdateUserRolesCommand.cs	
			GetRolesQuery.cs	
			Ui/	
			GetSelectOptionsQuery.cs	
			Middlewares/	
			ExceptionHandlerMiddleware.cs	- перехват необработанных исключений
			BaseResponseFilter.cs	- обертка ответов в BaseApiResponse<T
>			Services/	
			CurrentUserProvider.cs	- userId и роли из JWT-клеимов
			EmailNotificationService.cs	- отправка email + сохранение в Messa

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						37
Изм.	Лист	№ докум.	Подпись	Дата		

```

ges
├── Settings/
│   └── AuthSettings.cs           - Issuer, Audience, Key, Lifetime
├── Common/
│   ├── ApiErrors.cs             - стандартные ошибки (BadRequest, For
bidden, ...)
│   ├── BaseApiResponse.cs       - обертка ответа { data, status }
│   └── Unit.cs                  - тип-заглушка для команд без возвращ
аемого значения
├── OpenApiDocumentTransformers/
│   ├── BearerSecuritySchemeTransformer.cs
│   └── OneOfSimplifier.cs
└── Migrations/
    ├── 20260310154043_Initial.cs
    ├── 20260508134322_AddedMainEntities.cs
    ├── 20260510200303_AddedNameProperty.cs
    └── 20260530181425_AddedSmtFunc.cs

```

Описание директорий backend. Директория Features/ содержит все вертикальные фичи системы - каждая в отдельной папке с именем, соответствующим доменной области (Auth, Meetings, Users, Messages). Внутри каждой папки - файлы, реализующие IFeatureEndpoint с командой, запросом или обработчиком. Директория DataAccess/ содержит конфигурацию EF Core; директория Migrations/ - миграции. Директория Services/ - вспомогательные сервисы (email, JWT). Директория Models/ - DTO и маппинг через Mapster. Директория Common/ - общие утилиты: ApiErrors, BaseApiResponse, CurrentUserProvider, PagedRequest. Директория OpenApiDocumentTransformers/ - кастомизации Swagger (добавление Bearer-схемы, OneOf simplifier). Такая структура обеспечивает слабую связанность: каждая фича не зависит от других и может быть удалена или заменена без побочных эффектов.

```

frontend/src/
├── main.tsx                     - точка входа: Redux Provider, BrowserRouter, i1
8n init
├── i18n.ts                     - конфигурация i18next (ru/kk, localStorage, aut
odetect)
├── router/
│   ├── index.tsx               - React Router DOM маршруты
│   └── ProtectedRoute.tsx      - защита маршрутов по JWT и роли
├── store/
│   ├── index.ts                - конфигурация Redux store
│   └── hooks.ts                - useDispatch, useSelector (типизированные
)
├── slices/
│   ├── authSlice.ts            - аутентификация, loginThunk, logoutThunk
│   ├── sessionsSlice.ts       - встречи, участники, доступность
│   └── uiSlice.ts              - toast-уведомления, глобальный лоадер
└── pages/
    ├── auth/                   - LoginPage, RegisterPage
    ├── dashboard/              - DashboardPage
    ├── sessions/               - SessionsPage, SessionDetailPage
    └── profile/                 - ProfilePage

```

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						38
Изм.	Лист	№ докум.	Подпись	Дата		

	└─ admin/	- AdminDashboardPage, AdminUsersPage, AdminDeletedUsersPage, AdminMessagesPage
	└─ components/	
	└─ layout/	- AppShell, Header, PageSection
	└─ meetings/	- MeetingCard, MeetingForm, ParticipantsPanel, ParticipantDropdown
	└─ ui/	- Button, Input, Select, Textarea, Card, Badge, ConfirmDialog, EmptyState, GlobalLoader, Spinner, ToastViewport
	└─ i18n/	- LanguageSwitcher
	└─ http/	- httpClient, authService, sessionsService, usersService, messagesService, uiOptionsService
	└─ hooks/	- useAuth, useToast, useUiSelectOptions
	└─ utils/	- jwt.ts, format.ts, meetingLabels.ts, userLabels.ts, profile.ts
	└─ types/	- index.ts (CurrentUser, Meeting, MeetingParticipant, ...)
	└─ styles/	- global.scss, variables.scss, mixins.scss

Описание директорий frontend. Директория pages/ содержит по одной папке на маршрут приложения: каждая страница - самостоятельный модуль с компонентом и, при необходимости, вложенными компонентами. Директория components/ разделена на три уровня: layout/ (обязка приложения - шапка, боковая панель), meetings/ (бизнес-компоненты для работы со встречами), ui/ (переиспользуемый UI-кит: кнопки, поля ввода, модальные окна). Директория store/ содержит Redux store и слайсы для аутентификации, встреч и UI-состояния. Директория http/ - транспортный слой: http-клиент с автоматическим добавлением Bearer-токена и сервисы для каждого домена API. Директория hooks/ - кастомные React-хуки (useAuth, useToast). Директория utils/ - утилиты для работы с JWT, форматированием дат, метками встреч и пользователей. Такая структура соответствует Feature-Sliced Design: у каждого слоя - своя ответственность, и она четко определена.

Проект собирается и запускается из корневой директории. Система состоит из трех компонентов: PostgreSQL 16, backend (ASP.NET Core 9) и frontend (React 18 с TypeScript). При первом запуске backend миграции EF Core применяются автоматически. Backend запускается через dotnet run из директории backend/TemplateProject, frontend - через npm run dev из директории frontend. Подробная инструкция приведена в Приложении Д.

Разработка велась поэтапно. Ход реализации зафиксирован в проектной документации: инициализация проекта, настройка Identity и JWT, создание бизнес-сущностей, разработка frontend-компонентов, добавление локализации, тестирование. Такой подход обеспечивает прослеживаемость изменений и соответствие требованию методических указаний о фиксации этапов работ.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						39
Изм.	Лист	№ докум.	Подпись	Дата		

3.3 Реализация ключевых модулей

Все обработчики реализуют интерфейс `IFeatureEndpoint` с методом `Map(WebApplication app)`. При старте `Program.cs` сканирует сборку через `reflection` и вызывает `Map()` у каждого найденного класса (листинг А.4). Добавление новой фичи не требует правки `Program.cs`. Каждый класс фичи самостоятельно определяет свой маршрут, HTTP-метод и политику авторизации через `fluent`-методы `ASP.NET Core Minimal API`. Это делает каждую фичу полностью самодостаточной: файл содержит и маршрутизацию, и бизнес-логику, и валидацию. При удалении фичи достаточно удалить соответствующий файл - ни строчки кода в других файлах менять не нужно.

После успешной проверки учетных данных `handler` вызывает `GenerateJwtToken()` (листинг А.1), формирующий токен с клеймами `sub (userId)`, `iat` и `role`. Ключ подписи `HMAC-SHA256` хранится только в переменных окружения сервера. Срок жизни токена - 5 часов. До генерации выполняется проверка: `IsDeleted == true` → HTTP 406.

Формат встречи определяет набор обязательных полей. Валидация (листинг А.3) проверяет: для `Offline` - `Location` не пуст; для `Online` - `MeetingLink` не пуст; для `Hybrid` - оба поля; для `Phone` - `ContactInfo`. При нарушении `handler` возвращает `ApiErrors.BadRequest.Instance`, и запрос к БД не выполняется. После валидации создается запись `Meeting` со статусом `Draft` и отправляются email-уведомления приглашенным участникам.

Таким образом, процесс создания встречи прослеживается на всех уровнях системы. Организатор заполняет форму создания встречи (поля `Title`, `Format`, дата, время, условные поля и список участников) - форма приведена на рисунке В.7. Обработка запроса `POST /api/v1/meetings` выполняется в контроллере-обработчике `CreateMeetingCommand.cs` (фрагмент валидации приведен в листинге А.3): после успешной валидации формата создается запись в таблице `Meetings` со сгенерированным идентификатором `Id (uuid, первичный ключ)` и начальным статусом `Status = Draft`; структура таблицы `Meetings` (поля, типы, индексы) приведена в приложении Б. Корректность всей цепочки - идентификатор присвоен, статус `Draft`, запись сохранена в БД - подтверждена сценарием 4 протокола функционального тестирования (таблица 3.3).

Полная реализация LINQ-запроса приведена в листинге А.2. Условие фильтрации `newStart < existingEnd AND newEnd > existingStart` реализует классическую формулу пересечения интервалов. Встречи со статусами `Draft` и `Cancelled` исключены из проверки.

Все ответы оборачиваются через `BaseResponseFilter` (реализованный как глобальный `IResultFilter`) в единую структуру `{ "data": ..., "status": "Ok" }`. При

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						40
Изм.	Лист	№ докум.	Подпись	Дата		

ошибках `ExceptionHandlerMiddleware` формирует аналогичный ответ с `"status": "Error"`. Это позволяет frontend всегда ожидать одинаковый формат ответа независимо от эндпоинта.

Слайс содержит `loginThunk` для входа и функцию `hydrateSessionFromStorage` для восстановления сессии из `localStorage` (листинг A.5). `hydrateSessionFromStorage` вызывается в `main.tsx` при инициализации: если в `localStorage` есть действующий токен - пользователь считается авторизованным без повторного запроса к `/api/v1/user`.

Конфигурация `il8next` (листинг A.6) включает автоопределение языка при инициализации и два словаря - русский и казахский, сгруппированных по доменным областям: `common`, `nav`, `auth`, `meeting`, `sessions`, `roles`. При добавлении нового текстового элемента достаточно добавить пару ключей в оба словаря.

Управление статусами встречи реализовано в `GetMeetingStatusAsync()` без отдельного файла состояния - логика распределена по хендлерам, которые изменяют встречу:

- `Draft` → `Scheduled`: вызывается при создании встречи после валидации. Статус устанавливается через `meeting.Status = MeetingStatus.Scheduled`.
- `Scheduled` → `Rescheduled`: вызывается при изменении времени (`UpdateMeetingCommand.cs`). Проверка: если новое `StartDateTime` отличается от текущего - статус меняется автоматически.
- `Scheduled` → `Cancelled`: вызывается при отмене встречи организатором (`CancelMeetingCommand.cs`). Файлы и участники не удаляются.
- `AwaitingConfirmation` → `Confirmed`: автоматический переход при ответе последнего обязательного участника. Реализован внутри `RespondToInvitationCommand.cs`: после обновления `InvitationStatus` проверяется, все ли участники с `IsRequired == true` ответили `Accepted`. Если да - статус встречи обновляется на `Confirmed`.
- `Scheduled` → `AwaitingConfirmation`: переход по команде организатора (`ConfirmMeetingCommand.cs`). Организатор явно переводит встречу в режим ожидания подтверждений.
- `Rescheduled` → `AwaitingConfirmation`: после переноса организатор может снова запросить подтверждение участников.
- `Completed`: final status - устанавливается при завершении встречи (`CompleteMeetingCommand.cs`).
- `Cancelled`: финальный статус для отмененных встреч.

Автоматический переход `Scheduled` → `Completed` по истечении времени не реализован - он требует фоновой задачи (`Background Service ASP.NET Core`) и отнесен к

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						41
Изм.	Лист	№ докум.	Подпись	Дата		

направлениям развития (раздел Заключение). Архитектура Vertical Slice позволяет добавить его как отдельный хендлер без изменения существующих модулей.

Каждое изменение статуса встречи журналируется на стороне сервера. Все команды проходят через сквозной конвейер MediatR `LoggingBehavior`, который для каждого запроса открывает диагностическую область с именем фичи и телом запроса, фиксирует начало и успешное завершение обработки, а при исключении - запись уровня `Error` с трассировкой. Таким образом, переходы `Scheduled` → `Rescheduled` (хендлер `UpdateMeetingCommand`) и `AwaitingConfirmation` → `Confirmed` (хендлер `RespondToInvitationCommand`) попадают в журнал сервера. Момент последнего изменения дополнительно фиксируется в самой записи встречи - поле `UpdatedAt` обновляется на текущее время при каждой смене статуса. Уровень детализации задается в `appsettings.json` (секция `Logging:LogLevel:Default = Information`). Пример фрагмента журнала сервера при изменении времени встречи (автоматический переход `Scheduled` → `Rescheduled`) приведен ниже.

```
info: TemplateProject.Features.LoggingBehavior[0]
    => Processing feature UpdateMeetingCommand+Request
    MeetingId=3f2a... Status: Scheduled -> Rescheduled
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
    Executed DbCommand: UPDATE "Meetings" SET "Status"=4,
    "UpdatedAt"=@p1 WHERE "Id"=@p0
info: TemplateProject.Features.LoggingBehavior[0]
    Successfully handled. Duration: 18.7ms
```

3.4 Реализация базы данных

`DatabaseContext` наследует `IdentityDbContext<User, Role, Guid, ...>` и управляется через `AddDbContext<DatabaseContext>()` в системе внедрения зависимостей. Строка подключения передается через переменную окружения `ConnectionStrings__DefaultConnection` в формате PostgreSQL connection string. Fluent API конфигурация вынесена в метод `OnModelCreating`, где описаны:

- схема `identity` для всех Identity-сущностей через `builder.HasDefaultSchema("identity");`
- каскадные правила удаления: `Cascade` для участников и файлов при удалении встречи, `Restrict` для связи организатора, `SetNull` для сообщений;
- составные первичные ключи: `UserRoles` - композит (`UserId`, `RoleId`), `MeetingParticipants` - автоинкрементный `Id` с уникальным индексом (`MeetingId`, `UserId`);

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						42
Изм.	Лист	№ докум.	Подпись	Дата		

- фильтр мягкого удаления: `builder.HasQueryFilter(e => !e.IsDeleted)` - все запросы к `Meetings` и `Users` автоматически исключают удаленные записи без явного указания условия.

База данных развертывается через четыре последовательные миграции EF Core. Их хронология и содержание приведены в таблице 3.2.

Таблица 3.2 Миграции EF Core

Название миграции	Дата	Изменения
20260310154043_Initial	2026-03-10	Создает схему <code>identity</code> и все таблицы ASP.NET Core Identity: <code>Users</code> , <code>Roles</code> , <code>UserRoles</code> , <code>UserClaims</code> , <code>UserLogins</code> , <code>UserTokens</code> , <code>RoleClaims</code> . Устанавливает уникальные индексы <code>RoleNameIndex</code> , <code>EmailIndex</code> , <code>UserNameIndex</code> , <code>IX_Users_PhoneNumber</code> (фильтрованный)
20260508134322_AddedMainEntities	2026-05-08	Создает бизнес-таблицы: <code>Meetings</code> (все поля включая <code>IsDeleted</code> , <code>Status</code> , <code>Format</code> , <code>Location</code> , <code>MeetingLink</code> , <code>ContactInfo</code>), <code>MeetingParticipants</code> (уникальный составной индекс <code>MeetingId + UserId</code>), <code>MeetingFiles</code> . Добавляет индексы: <code>IX_Meetings_OrganizerId</code> , <code>IX_Meetings_Status</code> , <code>IX_Meetings_IsDeleted_Status</code> , <code>IX_MeetingParticipants_InvitationStatus</code> , <code>IX_MeetingFiles_MeetingId</code>
20260510200303_AddedNameProperty	2026-05-10	Изменяет колонку <code>Name</code> в <code>identity.Users</code> : тип остается <code>text</code> , убирается <code>nullable</code> , добавляется <code>DEFAULT ''</code> для существующих строк
20260530181425_AddedSmtпFunc	2026-05-30	Добавляет три поля в <code>identity.Users</code> : <code>IsDeleted</code> boolean <code>DEFAULT false</code> , <code>DeletedAt</code> <code>timestampz NULL</code> , <code>DeletedBy</code> <code>uuid NULL</code> . Создает таблицу <code>Messages</code> . Добавляет индексы <code>IX_Users_IsDeleted</code> , <code>IX_Messages_RecipientId</code> , <code>IX_Messages_SentAt</code>

При запуске `backend Program.cs` вызывает `ApplyMigrationsWithRetryAsync()` - метод с 10 попытками и экспоненциальной задержкой (2, 4, 8... секунд). Механизм необходим, так как `backend` и база данных запускаются параллельно, и EF Core может потребовать дополнительного времени на первое подключение после готовности PostgreSQL. После успешного подключения `DatabaseContext.Database.MigrateAsync()` применяет все ожидающие миграции в правильном хронологическом порядке. Если миграция уже была применена к БД, EF Core пропускает ее автоматически, что позволяет безопасно перезапускать `backend` без повторного выполнения одних и тех же изменений схемы.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						43
Изм.	Лист	№ докум.	Подпись	Дата		

Метод `ApplyMigrationsWithRetryAsync` реализован как `extension` для

`WebApplication`:

```
public static async Task ApplyMigrationsWithRetryAsync(this WebApplication app)
{
    using var scope = app.Services.CreateScope();
    var db = scope.ServiceProvider.GetRequiredService<DatabaseContext>();
    var logger = scope.ServiceProvider.GetRequiredService<ILogger<Program>>();
    var retryDelay = 2;
    for (int i = 0; i < 10; i++)
    {
        try
        {
            await db.Database.MigrateAsync();
            logger.LogInformation("Миграции успешно применены");
            return;
        }
        catch (Exception ex) when (i < 9)
        {
            logger.LogWarning("Попытка {i}: {Message}", i + 1, ex.Message);
            await Task.Delay(TimeSpan.FromSeconds(retryDelay));
            retryDelay *= 2;
        }
    }
}
```

`Program.cs` собирает конвейер `middleware` из следующих компонентов в порядке выполнения:

1. `ExceptionHandlerMiddleware` - глобальный обработчик необработанных исключений. Перехватывает любые исключения из нижележащих слоев и возвращает единый JSON-ответ `BaseApiResponse` со статусом `Error` и кодом HTTP 500. Не позволяет деталям исключения (`stack trace`, внутренние сообщения) попасть в HTTP-ответ.
2. `UseAuthentication` - `middleware` JWT-аутентификации. Верифицирует подпись токена, проверяет срок действия, извлекает клеймы и заполняет `HttpContext.User`.
3. `UseAuthorization` - `middleware` авторизации. Проверяет политики (`UserPolicy`, `ManagementPolicy`, `OnlyAdminPolicy`) перед выполнением эндпоинта. При несовпадении политики возвращает HTTP 403 без вызова бизнес-логики.
4. `Map всех IFeatureEndpoint` - регистрация маршрутов через `reflection` (листинг А.4).
5. `Swagger middleware` (`UseSwagger`, `UseSwaggerUI`) - доступен только в `Development`-окружении.

Регистрация сервисов в DI-компонента включает: `DatabaseContext (scoped)`, `MediatR (scoped)` - все хендлеры команд и запросов регистрируются автоматически), `HttpContextAccessor` и `CurrentUserProvider (scoped)` - извлекает `userId` и роли из JWT), `EmailNotificationService (singleton)`, `Mapster (singleton)` - глобальная конфигурация

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						44
Изм.	Лист	№ докум.	Подпись	Дата		

маппинга DTO), FluentValidation (scoped - автоматическая валидация через ValidationPipelineBehavior MediatR).

SQL-эквиваленты создания таблиц, генерируемые EF Core, приведены в Приложении Б (Б.1). Структура включает таблицы Meetings, MeetingParticipants, MeetingFiles и Messages с foreign key-ограничениями на каскадное удаление и индексами для оптимизации запросов.

3.5 Развертывание и запуск

Система состоит из трех компонентов, каждый из которых запускается независимо: сервер базы данных (PostgreSQL 16), серверная часть (ASP.NET Core 9), клиентская часть (React 18 с TypeScript).

Запуск базы данных. PostgreSQL 16 должен быть установлен и запущен на хост-машине. Параметры подключения (пользователь, пароль, имя базы данных) настраиваются через конфигурационный файл appsettings.json или переменные окружения. База данных meetings_db должна быть создана до запуска серверной части.

Запуск backend. Серверная часть запускается из директории backend/TemplateProject командой:

```
dotnet run
```

Либо, после публикации:

```
dotnet publish -c Release -o ./publish
dotnet ./publish/TemplateProject.dll
```

При запуске backend автоматически применяет миграции EF Core через ApplyMigrationsWithRetryAsync() - метод с механизмом повторных попыток (до 10 попыток с экспоненциальной задержкой) на случай, если база данных еще не готова к приему подключений. После успешного подключения DatabaseContext.Database.MigrateAsync() применяет все ожидающие миграции в правильном хронологическом порядке. Если миграция уже была применена к БД, EF Core пропускает ее автоматически, что позволяет безопасно перезапускать серверную часть без повторного выполнения одних и тех же изменений схемы. Сразу после миграций создаются seed-данные (тестовые пользователи и встречи).

Backend доступен по адресу <http://localhost:7262>, Swagger UI - <http://localhost:7262/swagger>.

Запуск frontend. Клиентская часть запускается из директории frontend командами:

```
npm install
npm run dev
```

Либо, для production-окружения, после сборки статических файлов:

```
npm run build
```

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						45
Изм.	Лист	№ докум.	Подпись	Дата		

Собранные статические файлы из `frontend/dist` раздаются любым веб-сервером (например, встроенным сервером разработки Vite на порту 3000). В режиме разработки Vite проксирует API-запросы на backend через настройку `vite.config.ts`, что исключает проблемы с кросс-доменными запросами (CORS).

Переменные окружения backend, необходимые для работы:

- `ConnectionStrings__DefaultConnection` - строка подключения к PostgreSQL;
- `Auth__Key` - секретный ключ для JWT HMAC-SHA256 (минимум 32 символа);
- `Smtp__Host`, `Smtp__Port`, `Smtp__Username`, `Smtp__Password` - настройки SMTP-сервера.

Система запускается на любой машине с установленными .NET 9 SDK, Node.js 20+ и PostgreSQL 16. Для локальной разработки достаточно трех терминалов: в первом запущен PostgreSQL, во втором - backend (`dotnet run`), в третьем - frontend (`npm run dev`). Полная пошаговая инструкция по запуску оформлена в виде файла `README.md` в корне репозитория проекта и продублирована в Приложении Д, что обеспечивает воспроизводимость развертывания и соответствует требованию методических указаний об обязательном наличии файла `README`.

3.6 Реализация локализации

Локализация реализована на двух уровнях: интерфейс приложения (`i18next`) и email-уведомления (`EmailNotificationService`).

Компонент `LanguageSwitcher` вызывает `i18n.changeLanguage(lang)` - `i18next` немедленно переключает словарь, и все компоненты с хуком `useTranslation()` перерисовываются без перезагрузки страницы. Выбранный язык сохраняется в `localStorage` и восстанавливается через `detectLanguage()`.

`EmailNotificationService` формирует HTML-письма, содержащие текст на двух языках одновременно. Письмо разделено на два блока: русский сверху, казахский снизу. Отправлять два отдельных письма не нужно - получатель видит уведомление на понятном ему языке вне зависимости от настроек почтового клиента.

После формирования HTML-содержимого метод сохраняет запись в таблицу `Messages` и отправляет письмо через `SmtpClient`. Если SMTP-сервер недоступен - исключение логируется, но не прерывает основной процесс: встреча создается, запись в `Messages` сохраняется.

3.7 Описание пользовательских сценариев

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						46
Изм.	Лист	№ докум.	Подпись	Дата		

Для демонстрации работы системы описаны ключевые сценарии с использованием seed-данных из Program.cs.

Сценарий 1. Organizer создает встречу и приглашает участников

Пользователь авторизован под organizer@gmail.com. На странице /sessions отображаются три существующие встречи. Организатор нажимает «Создать встречу», заполняет форму: заголовок, формат Online, ссылка на конференцию, дата и время. Перед отправкой приглашений проверяется доступность участников через POST /api/v1/meetings/availability. При отсутствии конфликтов встреча создается со статусом Draft, участники получают email-уведомления. При изменении времени статус автоматически переходит из Scheduled в Rescheduled.

Сценарий 2. User отвечает на приглашение

Пользователь авторизован под user@gmail.com. На DashboardPage отображается ближайшая встреча с InvitationStatus = Pending. На странице деталей встречи в панели участников доступны кнопки «Принять» и «Отклонить». При нажатии «Принять» вызывается PATCH /api/v1/meetings/{id}/participants/{userId}/respond. Статус обновляется до Accepted, заполняется ResponseAt. Если пользователь был последним обязательным участником - статус встречи автоматически переходит в Confirmed.

Сценарий 3. Admin управляет пользователями

Администратор авторизован под admin@gmail.com. На странице /admin/users отображается таблица активных пользователей с возможностью смены ролей. Admin выбирает пользователя с ролью User, нажимает «Редактировать роли», отмечает чекбокс Organizer и сохраняет. После обновления выбранный пользователь может создавать встречи. На странице /admin/users/deleted отображается список деактивированных учетных записей с кнопкой «Восстановить». При восстановлении IsDeleted сбрасывается в false, пользователь может войти в систему. Попытка удалить пользователя с активными встречами блокируется на уровне API.

Сценарий 4. Двуязычное уведомление при изменении встречи

Organizer изменяет время существующей встречи со статусом Scheduled. Backend автоматически переводит статус в Rescheduled, отправляет email-уведомления всем участникам. Письмо содержит два блока: русский («Встреча перенесена») и казахский («Кездесу ауыстырылды») - получатель видит уведомление на обоих языках независимо от настроек почтового клиента.

3.8 Тестирование

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						47
Изм.	Лист	№ докум.	Подпись	Дата		

Тестирование системы проводилось по сценариям, охватывающим все три роли пользователей и ключевые бизнес-процессы. Цель тестирования - подтвердить, что каждое функциональное требование (US-01-US-10) и каждое нефункциональное требование из Раздела 1 реализовано и работает в соответствии с критерием приемки.

Методика тестирования. Каждый сценарий состоит из пяти шагов: описание исходного состояния, выполняемые действия (последовательность HTTP-запросов или действий в интерфейсе), ожидаемый результат, фактический результат и вывод (выполнено/не выполнено). Тестирование проводилось вручную: API-вызовы - через Swagger UI, проверка интерфейса - через браузер. Тестовые данные (7 пользователей и 7 встреч со всеми статусами) создаются автоматически при первом запуске через seed-механизм Program.cs. Для каждого сценария фиксировался фактический результат; все расхождения между ожидаемым и фактическим результатом отсутствуют.

Критерии остановки тестирования. Тестирование считается завершенным, когда все 12 сценариев выполнены. При обнаружении расхождения между ожидаемым и фактическим результатом по любому сценарию тестирование приостанавливается до исправления дефекта. Расхождений не обнаружено, все 12 сценариев пройдены без приостановок.

Окружение тестирования. Все тесты проводились на локальной машине разработчика (Windows 11). Backend работал как сервис на .NET 9, frontend раздавался веб-сервером, PostgreSQL 16 - как отдельный сервис базы данных. Доступ к backend через localhost:7262/swagger. Время выполнения тестирования по всем 12 сценариям - не более 20 минут. Результаты зафиксированы в таблице 3.3.

Таблица 3.3 Протокол функционального тестирования

№	Цель проверки	Исходное состояние	Выполненные действия	Ожидаемый результат	Фактический результат	Вывод
1	JWT-аутентификация	Пользователь не авторизован	POST /api/v1/auth/login: admin@gmail.com / Admin123!	HTTP 200, поле data.token содержит JWT, status: "Ok"	Токен получен, frontend выполнил редирект на /	Требование выполнено
2	Блокировка удаленного аккаунта	Флаг IsDeleted=true у пользователя	POST /api/v1/auth/login с корректными данными удаленного пользователя	HTTP 406 NotAcceptable	Ответ HTTP 406, вход заблокирован	Требование выполнено

Продолжение таблицы 3.3

№	Цель проверки	Исходное состояние	Выполненные действия	Ожидаемый результат	Фактический результат	Вывод
3	Валидация: Online без ссылки	Organizer авторизован	POST /api/v1/meetings: Format=Online, поле MeetingLink не указано	HTTP 400 BadRequest	HTTP 400 возвращен, встреча не создана	Требование выполнено
4	Создание встречи (успех)	Organizer авторизован	POST /api/v1/meetings: Format=Online, MeetingLink заполнен, все поля корректны	HTTP 200, встреча создана со Status=Draft, Id присвоен	Встреча создана, Id в ответе, email-уведомления отправлены	Требование выполнено
5	Обнаружение конфликта расписания	Встреча «Еженедельный стендап» 09:00-09:30 существует	POST /api/v1/meetings/availability: StartTime=09:15, EndTime=09:45, participantIds=[userId]	allAvailable: false, поле conflicts содержит конфликтующую встречу	Конфликт определен, возвращены название и время конфликтующей встречи	Требование выполнено
6	Мягкое удаление встречи	Встреча со статусом Draft существует	DELETE /api/v1/meetings/{id} (организатор)	IsDeleted=true в БД, встреча не отображается в GET /api/v1/meetings	Запись скрыта из выборки, данные сохранены в БД	Требование выполнено
7	Ответ участника на приглашение	InvitationStatus=Pending у участника	PATCH /api/v1/meetings/{id}/participants/{userId}/respond: { "status": "Accepted" }	InvitationStatus=Accepted, поле ResponseAt заполнено	Статус обновлен, ResponseAt содержит текущее время	Требование выполнено

№	Цель проверки	Исходное состояние	Выполненные действия	Ожидаемый результат	Фактический результат	Вывод
8	Автоматическое подтверждение встречи	Последний обязательный участник не ответил, статус встречи AwaitingConfirmation	PATCH .../respond: { "status": "Accepted" } последним обязательным участником	Status встречи автоматически переходит в Confirmed	Статус встречи обновлен до Confirmed без ручного действия	Требование выполнено
9	RBAC: запрет создания встречи для User	Пользователь с ролью User авторизован	POST /api/v1/meetings	HTTP 403 Forbidden	HTTP 403 возвращен	Требование выполнено
10	Переключение языка интерфейса	Интерфейс отображается на русском языке	Нажатие на LanguageSwitcher → выбор «Қазақша»	Все строки интерфейса переключаются на казахский язык, выбор сохраняется в localStorage	Интерфейс переключен немедленно, при перезагрузке страницы язык сохранен	Требование выполнено
11	Управление ролями пользователя	Admin авторизован	PATCH /api/v1/users/{id}/roles: { "roles": ["Organizer"] }	Роль обновлена в БД, старые роли заменены	Роль изменена, при следующем логине пользователь получает токен с ролью Organizer	Требование выполнено
12	Загрузка и скачивание файла	Organizer авторизован, встреча существует	POST /api/v1/meetings/{id}/files (multipart, файл meeting-agenda.pdf)	Файл сохранен по пути files/meetings/{id}/..., запись в MeetingFiles создана	Файл доступен через GET /api/v1/meetings/{id}/files/{fileId}/download	Требование выполнено

Все 12 сценариев завершились с ожидаемым результатом. Расхождений между ожидаемым и фактическим поведением не зафиксировано.

3.9 Связь требований с реализацией

В таблице 3.4 приведено соответствие каждого функционального требования (User Story из раздела 1.5) конкретному реализованному компоненту и номеру подтверждающего теста.

Таблица 3.4 Матрица требования ↔ реализация ↔ тест

ID	User Story	Реализованный компонент	Тест
US-01	Создание встречи с валидацией формата	CreateMeetingCommand.cs + MeetingForm.tsx	№3 (отказ), №4 (успех)
US-02	Проверка занятости участников	CheckAvailabilityQuery.cs	№5
US-03	Ответ участника на приглашение	RespondToInvitationCommand.cs + ParticipantsPanel.tsx	№7
US-04	Мягкое удаление пользователя	DeleteUserCommand.cs	№6
US-05	Смена роли пользователя	UpdateUserRolesCommand.cs + AdminUsersPage.tsx	№11
US-06	Казахскоязычный интерфейс	i18n.ts + LanguageSwitcher.tsx	№10
US-07	Прикрепление файлов к встрече	UploadMeetingFileCommand.cs	№12
US-08	Скачивание файла встречи	DownloadMeetingFileQuery.cs	№12
US-09	Автоматическое подтверждение встречи	RespondToInvitationCommand.cs (логика автоперехода)	№8
US-10	Журнал email-уведомлений	ListMessagesQuery.cs + AdminMessagesPage.tsx	№4 (email при создании)

Все 10 функциональных требований закрыты реализованными компонентами и подтверждены тестовыми сценариями.

Полная доказательная база проекта - backlog (перечень задач), схема архитектуры, диаграмма базы данных, исходный код, история коммитов, пользовательские сценарии, API-описание, тестовые сценарии, скриншоты, сценарий демонстрации, SQL-скрипты, README и инструкции запуска - формировалась непрерывно на протяжении всей разработки, а не подбиралась после написания записки. Состав доказательной базы с указанием места фиксации каждого элемента сведен в Приложении Е.

РАЗДЕЛ 4. ЭКОНОМИЧЕСКАЯ ЧАСТЬ

Разрабатываемая система - внутренний инструмент автоматизации. Экономическая эффективность оценивается как сокращение времени на ручную координацию встреч. До внедрения организаторы тратят время на отправку сообщений, отслеживание ответов в чатах, ручной контроль занятости. Система переводит эти операции в автоматический режим. Расчет построен на сопоставлении затрат на создание и эксплуатацию с денежным эквивалентом высвобождаемого рабочего времени.

Методика расчета включает следующие этапы:

1. Определение перечня и стоимости трудозатрат на разработку (таблица 4.1-4.2).
2. Определение инфраструктурных расходов на первый год эксплуатации (таблица 4.3).
3. Сводный расчет общих затрат (таблица 4.4).
4. Оценка экономии рабочего времени после внедрения (таблица 4.5).
5. Расчет итоговых показателей: срок окупаемости, ROI, чистый дисконтированный доход (таблица 4.6).

Исходные данные для расчета приведены в таблице 4.1.

4.1 Исходные данные

Расчет выполнен для условной организации с 50 сотрудниками (10 организаторов). Стоимость часа - 5 000 тг (средний показатель для технического персонала в РК в 2026 г.). Расчетный период - один год.

Таблица 4.1 Исходные данные для экономического расчета

Параметр	Значение
Общее число пользователей системы	50 человек
Из них организаторов встреч	10 человек
Стоимость 1 часа рабочего времени (С_час)	5 000 тг
Стоимость аренды сервера	15 000 тг/мес
Расчетный период	12 месяцев (1 год)
Среднее число встреч в неделю на одного организатора	3 встречи
Среднее время ручной координации одной встречи (до внедрения)	10 мин
Среднее время координации одной встречи через систему (после)	1 мин
Экономия времени на одну встречу	9 мин
Обучение сотрудников	6 ч × 10 организаторов = 60 ч суммарно, 6 ч на стоимость обучения

4.2 Расчет затрат на разработку

Разработка выполнялась одним разработчиком. Трудозатраты определены по этапам жизненного цикла. Трудозатраты на разработку и их стоимость приведены в таблице 4.2.

Таблица 4.2 Трудозатраты на разработку и их стоимость

Этап разработки	Трудозатраты, ч	Ставка, тг/ч	Стоимость, тг
Анализ требований, проектирование архитектуры и БД	15	5 000	75 000
Разработка backend (ASP.NET Core 9, EF Core, MediatR, JWT)	30	5 000	150 000
Разработка frontend (React 18, Redux Toolkit, i18next, SCSS)	25	5 000	125 000
Тестирование и отладка	10	5 000	50 000
Итого	80	-	400 000

Наибольшая доля трудозатрат приходится на разработку backend - 30 часов. Причина - 32 эндпоинта, система авторизации с тремя политиками, четыре миграции базы данных, механизм email-уведомлений и конечный автомат статусов встречи. Frontend потребовал 25 часов: реализованы 10 страниц, Redux-хранилище из трех слайсов и двуязычная локализация.

4.3 Инфраструктурные расходы

Весь программный стек системы построен на открытом программном обеспечении с лицензиями, разрешающими коммерческое использование без лицензионных отчислений. Единственная статья инфраструктурных расходов - аренда сервера для развертывания компонентов.

Таблица 4.3 Инфраструктурные расходы

Компонент	Лицензия	Стоимость	Примечание
.NET 9 / ASP.NET Core 9	MIT	0 тг	Открытая платформа Microsoft
React 18	MIT	0 тг	Открытая библиотека Meta
PostgreSQL 16	PostgreSQL License	0 тг	Открытая СУБД
nginx (веб-сервер)	BSD 2-Clause	0 тг	Открытый web-сервер для раздачи статики frontend
Аренда сервера (VPS, 2 vCPU, 4 GB RAM)	-	15 000 тг/мес	Достаточно для запуска трех компонентов
Итого за год	-	180 000 тг	15 000 × 12 месяцев

Выбранная конфигурация сервера (2 виртуальных ядра, 4 ГБ оперативной памяти) достаточна для одновременной работы трёх компонентов: базы данных (PostgreSQL 16), серверной части (ASP.NET Core 9) и клиентской части (React 18). При росте нагрузки система масштабируется горизонтально за счет независимой архитектуры компонентов без изменения кода.

4.4 Общие затраты на создание и эксплуатацию

Суммарные затраты рассчитываются по формуле (4.1):

$$C_{\text{общ}} = C_{\text{разр}} + C_{\text{инфр}} + C_{\text{обуч}} + C_{\text{эксп}} \quad (4.1)$$

где $C_{\text{общ}}$ - общие затраты; $C_{\text{разр}}$ - стоимость разработки (80 ч * 5 000 тг/ч); $C_{\text{инфр}}$ - инфраструктурные расходы (15 000 тг/мес * 12 мес); $C_{\text{обуч}}$ - обучение пользователей (6 ч * 5 000 тг/ч); $C_{\text{эксп}}$ - техническое сопровождение (4 ч * 5 000 тг/ч).

Подстановка дает значение:

$$C_{\text{общ}} = 400\,000 + 180\,000 + 30\,000 + 20\,000 = 630\,000 \text{ тг}$$

Таблица 4.4 Сводная таблица затрат

Статья затрат	Сумма, тг	Доля, %
Разработка (80 ч × 5 000 тг)	400 000	63,5
Аренда сервера (12 мес × 15 000 тг)	180 000	28,6
Обучение пользователей (6 ч × 5 000 тг)	30 000	4,8
Техническое сопровождение (4 ч × 5 000 тг)	20 000	3,1
Итого ($C_{\text{общ}}$)	630 000	100

Наибольшая доля затрат (63,5 %) - разработка. Это типично для внутренних систем автоматизации: единовременные вложения в создание продукта значительно превышают стоимость его последующей эксплуатации.

4.5 Экономический эффект от внедрения

Экономический эффект выражается в сокращении времени на ручную координацию (10 мин/встреча до, 1 мин - после). Годовая экономия рассчитывается по формуле (4.2):

$$\mathcal{E} = N_{\text{орг}} \times K_{\text{нед}} \times T_{\text{нед}} \times W_{\text{год}} \times (C_{\text{час}} / 60) \quad (4.2)$$

где $N_{\text{орг}}$ - количество организаторов (10); $K_{\text{нед}}$ - количество встреч на организатора в неделю (3); $T_{\text{нед}}$ - экономия времени на встречу в минутах (9); $W_{\text{год}}$ - количество рабочих недель в году (52); $C_{\text{час}}$ - стоимость часа специалиста (5 000 тг).

Подстановка дает значение:

$$\mathcal{E} = 10 \times 3 \times 9 \times 52 \times (5\,000 / 60) = 1\,170\,000 \text{ тг/год}$$

Для наглядности в таблице 4.5 приведено сравнение временных затрат.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						54
Изм.	Лист	№ докум.	Подпись	Дата		

Таблица 4.5 Сравнение временных затрат до и после внедрения

Показатель	До внедрения	После внедрения	Экономия
Время координации одной встречи, мин	10	1	9
Встреч в неделю (10 организаторов × 3)	30	30	-
Время в неделю, ч	5	0,5	4,5
Время в год, ч	260	26	234
Денежный эквивалент сэкономленного времени, тг	-	-	1 170 000

Экономия составляет 234 часа рабочего времени в год, что эквивалентно примерно 10 % годового фонда рабочего времени одного сотрудника. В денежном выражении - 1 170 000 тенге при ставке 5 000 тг/час.

4.6 Срок окупаемости и рентабельность

Период окупаемости рассчитывается по формуле (4.3):

$$T_{ок} = C_{общ} / Э \quad (4.3)$$

где $T_{ок}$ - период окупаемости (лет); $C_{общ}$ - суммарные затраты на создание и первый год эксплуатации (630 000 тг); $Э$ - годовая экономия (1 170 000 тг).

Подстановка дает значение: $T_{ок} = 630\,000 / 1\,170\,000 = 0,54$ года $\approx 6,4$ месяца.

Рентабельность инвестиций рассчитывается по формуле (4.4):

$$ROI = (Э - C_{общ}) / C_{общ} \times 100 \% \quad (4.4)$$

где ROI - рентабельность инвестиций, %.

Подстановка дает значение: $ROI = (1\,170\,000 - 630\,000) / 630\,000 \times 100 \% = 85,7 \%$

Каждый вложенный тенге приносит 0,86 тенге чистой экономии за первый год.

Таблица 4.6 Итоговые показатели экономической эффективности

Показатель	Значение
Суммарные затраты ($C_{общ}$)	630 000 тг
Годовая экономия ($Э$)	1 170 000 тг
Чистый эффект за первый год ($Э - C_{общ}$)	540 000 тг
Срок окупаемости ($T_{ок}$)	$\approx 6,4$ месяца
Рентабельность инвестиций (ROI)	85,7 %

Со второго года эксплуатации затраты составляют 200 000 тг/год, чистый эффект - 970 000 тг/год. Разовые вложения окупаются в первый год, каждый последующий система приносит чистую экономию, в 4,85 раза превышающую расходы на поддержку.

РАЗДЕЛ 5. ОХРАНА ТРУДА, ТЕХНИКА БЕЗОПАСНОСТИ И БЕЗОПАСНОСТЬ ЭКСПЛУАТАЦИИ

Раздел рассматривает два взаимосвязанных аспекта: условия труда разработчика и пользователей системы на рабочем месте, а также меры по обеспечению информационной безопасности при эксплуатации системы управления встречами. Оба аспекта привязаны к конкретному программному стеку: ASP.NET Core 9, React 18, PostgreSQL 16.

5.1 Требования к рабочему месту

Разработка и эксплуатация системы - длительная работа за ПК. Требования к рабочему месту регламентированы СанПиН РК (2.2.2/2.4.1340-03) и Трудовым кодексом Республики Казахстан.

Рабочее место должно иметь совмещенное освещение: естественное (боковое одностороннее) и искусственное. Нормируемая освещенность рабочей поверхности - не менее 300 лк при общем освещении. Яркость монитора должна соответствовать яркости фона: экран не должен быть направлен к окну, чтобы избежать бликов. При развертывании серверной инфраструктуры в выделенном помещении требования снижаются до аварийного уровня - не менее 50 лк у панелей оборудования.

Оборудование подключается к сети 220 В через трехпроводную схему с защитным заземлением. Удлинитель без заземляющего контакта запрещены. Серверное оборудование должно быть подключено через ИБП с автономией не менее 10 минут - этого достаточно для корректного завершения работы сервисов и базы данных при пропадании питания. Внезапное отключение сервера базы данных при активных транзакциях может повредить данные PostgreSQL. Проверка заземления - не реже одного раза в год.

Работа с ПК относится к категории Ia (легкие физические работы). Оптимальные параметры микроклимата: температура 23-25 °С, влажность 40-60 %, скорость воздуха не более 0,1 м/с. При размещении сервера в серверной необходима принудительная вентиляция с кратностью воздухообмена не менее 10-15 объемов в час для отвода тепла от серверного оборудования. Перегрев процессора выше 80 °С ведет к снижению тактовой частоты и деградации производительности. Требования к условиям труда на рабочем месте приведены в таблице 5.1.

Таблица 5.1 Требования к условиям труда на рабочем месте

Параметр	Норматив	Источник
Освещенность рабочей поверхности	≥ 300 лк	СанПиН РК
Температура воздуха (теплый период)	23-25 °С	СанПиН 2.2.2/2.4.1340
Относительная влажность	40-60 %	СанПиН 2.2.2/2.4.1340
Уровень шума на рабочем месте	≤ 50 дБА	ГОСТ 12.1.003

Параметр	Норматив	Источник
Напряжение питания	220 В, TN-S (заземление)	ПУЭ РК
Время автономии ИБП (сервер)	≥ 10 мин	Рекомендация

5.2 Режим труда и отдыха

Работа разработчика и администратора системы относится к работе с видеодисплейными терминалами (ВДТ). Нормы режима труда при работе за ПК установлены СанПиН и типовыми инструкциями по охране труда.

При работе с ВДТ суммарное время непосредственной работы за экраном в течение рабочего дня не должно превышать 6 часов. Для разработчиков, выполняющих работу категории В (творческая работа в режиме диалога с ЭВМ), регламентированные перерывы устанавливаются:

- через 1,5-2 часа от начала рабочей смены - перерыв 15 минут;
- через 1,5-2 часа после обеденного перерыва - перерыв 15 минут.

При восьмичасовом рабочем дне суммарное время регламентированных перерывов - 50 минут. В течение перерывов рекомендуется выполнять гимнастику для глаз (по 3-5 минут) и физические упражнения для снятия статического напряжения мышц.

Для администратора, в обязанности которого входит мониторинг работоспособности сервисов системы, рекомендуется установить расписание проверок: утром (при начале рабочего дня), в середине дня и перед завершением смены. Постоянный непрерывный мониторинг без перерывов не требуется - служба настроена на автоматический перезапуск при сбоях. Режим работы с видеодисплейным терминалом приведен в таблице 5.2.

Таблица 5.2 Режим работы с ВДТ (категория В)

Показатель	Норма
Максимальное время работы за экраном в смену	6 ч
Регламентированные перерывы	2 раза по 15 мин
Суммарное время перерывов	50 мин/смену
Рекомендуемый ритм мониторинга компонентов	3 раза в день

5.3 Безопасность эксплуатации системы

Система обрабатывает персональные данные пользователей (email-адреса, имена, пароли) и данные деловых встреч организации. Требования к информационной безопасности реализованы на нескольких уровнях.

Доступ к системе - через JWT-аутентификацию (HMAC-SHA256). Токен содержит идентификатор пользователя и его роли, срок действия - пять часов (Auth__Lifetime:

"05:00:00"). После истечения пользователь проходит повторную аутентификацию, что ограничивает окно для использования перехваченного токена.

Авторизация реализована через три политики доступа:

- UserPolicy - любой аутентифицированный пользователь;
- ManagementPolicy - только роли Admin или Organizer;
- OnlyAdminPolicy - только Admin.

Политики проверяются на уровне middleware ASP.NET Core до выполнения бизнес-логики. Попытка обратиться к защищенному ресурсу без соответствующей роли возвращает HTTP 403, не раскрывая информации о содержимом ресурса.

Пароли хранятся как хэш PBKDF2 с HMAC-SHA256 и случайной солью через ASP.NET Core Identity. JWT-секрет Auth__Key не в исходном коде - он передается через переменную окружения сервера (см. файл переменных окружения в Приложении Д). В production требуется замена на случайную строку длиной не менее 32 символов.

Угрозы информационной безопасности и меры их нейтрализации приведены в таблице 5.3.

Таблица 5.3 Угрозы информационной безопасности и меры их нейтрализации

Угроза	Уровень риска	Мера защиты	Реализация в проекте
Перехват JWT-токена	Средний	Ограниченный срок действия токена	Auth__Lifetime: "05:00:00" в переменных окружения
Утечка JWT-секрета	Высокий	Секрет только в переменных окружения	Auth__Key - environment variable, не в коде
Вход деактивированного пользователя	Средний	Проверка IsDeleted при каждом логине	LoginCommand: IsDeleted == true → HTTP 406
Несанкционированный вызов API	Высокий	RBAC через три политики авторизации	UserPolicy, ManagementPolicy, OnlyAdminPolicy
Хранение паролей в открытом виде	Критический	Хэширование PBKDF2 через ASP.NET Identity	Поле PasswordHash в identity.Users
SQL-инъекции	Высокий	Параметризованные запросы через EF Core	Все операции - через LINQ, без raw SQL
XSS-атаки через email-шаблоны	Низкий	Статичные HTML-шаблоны без пользовательских данных в script-тегах	EmailNotificationService - только статический HTML

Угроза	Уровень риска	Мера защиты	Реализация в проекте
Несанкционированный доступ к БД	Высокий	Соединение с БД только изнутри внутренней сети сервера	PostgreSQL доступен только через внутреннюю сеть, порт 5432 не проброшен наружу в prod
Перебор паролей (brute-force)	Средний	Механизм блокировки ASP.NET Identity	LockoutEnabled, AccessFailedCount в identity.Users

Все сервисы системы работают в единой внутренней сети сервера. Из публичной сети доступны только порт 3000 (веб-сервер, статика React) и 7262 (backend API и Swagger UI). Порт 5432 PostgreSQL не публикуется в production - соединение устанавливается только от серверной части (backend) внутри внутренней сети сервера. Дополнительно на уровне backend настроена политика CORS: в среде разработки разрешены запросы с любого источника (AllowAnyOrigin), а при разворачивании в производственной среде доступ необходимо ограничить доменом клиентского приложения.

5.4 Резервное копирование данных

Данные хранятся в PostgreSQL 16 в базе meetings_db. Каталог данных PostgreSQL /var/lib/postgresql/data хранится на файловой системе сервера. Резервное копирование выполняется утилитой pg_dump. Рекомендуемая периодичность: ежедневно в 02:00 по cron (команда в Приложении Д). Копии должны храниться вне сервера не менее 30 дней. Корректность проверяется раз в месяц пробным восстановлением в тестовой среде.

Регламент резервного копирования приведен в таблице 5.4.

Таблица 5.4 Регламент резервного копирования

Параметр	Значение
База данных	meetings_db (PostgreSQL 16)
Сервис БД	PostgreSQL (отдельный сервис)
Каталог данных	/var/lib/postgresql/data
Инструмент	pg_dump
Периодичность создания копии	Ежедневно в 02:00
Место хранения	Отдельный сервер / облачное хранилище
Срок хранения копий	30 дней
Периодичность проверки восстановления	1 раз в месяц

5.5 Работа с тестовой средой

При первом запуске backend автоматически создает тестовых пользователей и встречи через Program.cs. Тестовая среда отделена от рабочей и развёрнута в отдельном окружении на отдельном сервере, что исключает пересечение с production. Перечень тестовых учетных записей приведен в таблице 5.5.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						59
Изм.	Лист	№ докум.	Подпись	Дата		

Таблица 5.5 Тестовые учетные записи (seed-данные)

Роль	Email	Пароль
Admin	admin@gmail.com	Admin123!
Organizer	organizer@gmail.com	Organizer123!
Organizer (резервный)	backup.organizer@gmail.com	BackupOrganizer123+
User	user@gmail.com	User123!
User (QA Engineer)	qa.engineer@gmail.com	QaEngineer123+
User (Business Analyst)	business.analyst@gmail.com	Analyst123+
User (Support Manager)	support.manager@gmail.com	Support123+

Seed-встречи охватывают все семь статусов жизненного цикла: Draft, Scheduled, AwaitingConfirmation, Confirmed, Rescheduled, Cancelled, Completed. Это позволяет тестировать поведение системы для каждого состояния без ручного создания тестовых данных.

В производственной среде тестовые учетные данные заменяются на реальные через переменные окружения файл переменных окружения (полный перечень - в Приложении Д). Использование тестовых паролей в производственной среде недопустимо.

5.6 Защита персональных данных

Система обрабатывает персональные данные: полное имя, email, номер телефона, возраст. В соответствии с Законом РК «О персональных данных и их защите» от 21 мая 2013 года № 94-V обработка данных должна быть законной, ограниченной по цели и минимизированной.

Пароли хранятся как хэш PBKDF2 с 10 000 итераций HMAC-SHA256 в поле PasswordHash таблицы identity.Users - восстановление из хэша вычислительно нецелесообразно. В production весь трафик должен передаваться по HTTPS (веб-сервер перенаправляет HTTP → HTTPS, сертификат - Let's Encrypt).

Доступ к данным ограничен на уровне API: пользователь видит только свои данные, список пользователей - только Admin, организаторы видят ограниченный список для выбора участников (без конфиденциальных полей). Прямой доступ к БД исключен: все операции через EF Core с параметризованными запросами.

При удалении пользователя устанавливается IsDeleted = true; данные физически не удаляются для сохранения целостности истории встреч. Окончательное удаление выполняется администратором БД прямым SQL-запросом после создания резервной копии.

ЗАКЛЮЧЕНИЕ

В дипломном проекте разработана web-система управления встречами и согласованием участия. Работа охватывает полный цикл - от анализа предметной области и требований до проектирования, реализации, тестирования и экономического обоснования. Все семь задач Введения выполнены.

Задачи 1-2 (анализ и сравнение). Описан реальный процесс организации встреч через мессенджеры и его дефекты: ответы не фиксируются, конфликты расписания не видны, история теряется, ответственность не разграничена. Сравнение Google Calendar, Microsoft Teams и Calendly по 11 критериям показало, что ни один инструмент одновременно не дает гибкого RBAC, казахскоязычного интерфейса, self-hosted развертывания и журнала уведомлений. Сформулированы 10 функциональных требований (User Story с критериями приемки) и 8 нефункциональных (безопасность, производительность, локализация, масштабируемость).

Задачи 3-4 (архитектура и база данных). Система двухуровневая: backend из самодостаточных классов (каждый реализует IFeatureEndpoint и регистрирует маршрут через reflection), frontend - SPA на React 18 с тремя Redux-слайсами; взаимодействие через REST API с единым форматом BaseApiResponse<T>, конвейер middleware из пяти компонентов в строгом порядке. База данных - одиннадцать таблиц (семь основных сущностей и четыре служебные таблицы Identity) в двух схемах PostgreSQL, четыре миграции EF Core, одиннадцать индексов; мягкое удаление каскадное, целостность обеспечена внешними ключами (CASCADE для участников и файлов, RESTRICT для организатора, SET NULL для сообщений).

Задачи 5-7 (реализация и тестирование). Backend на ASP.NET Core 9 - 32 REST-эндпоинта в семи доменных областях с JWT-аутентификацией (HMAC-SHA256), тремя политиками RBAC, конечным автоматом статусов, алгоритмом обнаружения конфликтов расписания и email-уведомлениями с журналом Messages. Frontend на React 18 (TypeScript, Vite) - 10 страниц с разграничением по ролям, Redux Toolkit и локализацией на русский и казахский. Тестирование по 12 сценариям пройдено полностью, расхождений не выявлено. Тем самым все 10 функциональных требований, сформулированных во Введении, подтверждены реализацией и тестовыми сценариями; матрица соответствия «требование - компонент - тест» приведена в подразделе 3.9.

Итоговый результат. Система покрывает полный жизненный цикл встречи - от создания (Draft) до завершения (Completed). Backend содержит 32 REST-эндпоинта с единым форматом ответа BaseApiResponse<T>, frontend - 10 страниц с разграничением по трем ролям

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						61
Изм.	Лист	№ докум.	Подпись	Дата		

и локализацией на два языка. Система запускается одной командой, три компонента разворачиваются автономно, миграции применяются при первом запуске. На практике освоены ASP.NET Core, EF Core и PostgreSQL, JWT, Redux Toolkit, i18next и React Router.

Применены проектирование реляционных баз данных, паттерны (CQRS, Vertical Slice, MediatR), средства развертывания и системы контроля версий. Структура записки соответствует методическим указаниям.

Экономический расчет: затраты на разработку и первый год эксплуатации - 630 000 тг, годовая экономия - 1 170 000 тг, срок окупаемости - 6,4 месяца, ROI - 85,7 %. Учтены только прямые потери от ручного согласования (около 260 человеко-часов в год) без косвенных эффектов - сокращения опозданий, числа отмен и нагрузки на менеджеров; внедрение целесообразно в организациях от 20 сотрудников.

Раздел охраны труда выявил два фактора вредности (зрительное напряжение и гиподинамия) и предложил меры профилактики - зрительную гимнастику, структурированные перерывы, эргономичную организацию рабочего места; дополнительных рисков для здоровья система не создает.

Ограничения. Уведомления доставляются только по email, отсутствует синхронизация с внешними календарями, интерфейс доступен только через браузер, повторяющиеся встречи и двухфакторная аутентификация не реализованы. Архитектура Vertical Slice позволяет добавить их как новые фичи без переработки кода.

Направления развития. SignalR для real-time уведомлений, Google Calendar и Microsoft Graph API для интеграции с календарями, React Native для мобильного приложения, iCalendar RRULE для повторяющихся встреч, TOTP для двухфакторной аутентификации; наиболее критично - добавление unit-тестов для backend (MediatR-хендлеры) и компонентных тестов для frontend (React Testing Library).

Разработанная система полностью соответствует заданию дипломного проекта, цели и задачам Введения. Результаты применимы в организациях любого профиля, где требуется автоматизировать управление встречами с ролевой моделью, проверкой конфликтов расписания и локализацией на казахский язык.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						62
Изм.	Лист	№ докум.	Подпись	Дата		

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Государственный общеобязательный стандарт образования Республики Казахстан. Техническое и профессиональное образование. Специальность 06130100 «Программное обеспечение» (по видам) [Текст] / Министерство просвещения РК. - Астана, 2024. - Использован в разделах: Введение, Раздел 1.
2. Закон Республики Казахстан «О персональных данных и их защите» [Текст] : закон Республики Казахстан от 21 мая 2013 года № 94-V / Республика Казахстан. - Астана, 2013. - Режим доступа: <https://adilet.zan.kz/rus/docs/Z1300000094> (дата обращения: 01.06.2026). - Использован в Разделе 5.
3. Закон Республики Казахстан «Об информатизации» [Текст] : закон Республики Казахстан от 24 ноября 2015 года № 418-V / Республика Казахстан. - Астана, 2015. - Режим доступа: <https://adilet.zan.kz/rus/docs/Z1500000418> (дата обращения: 01.06.2026). - Использован в разделах: Введение, Раздел 1.
4. Закон Республики Казахстан «Об образовании» [Текст] : закон Республики Казахстан от 27 июля 2007 года № 319-III / Республика Казахстан. - Астана, 2007. - Режим доступа: https://adilet.zan.kz/rus/docs/Z070000319_ (дата обращения: 01.06.2026). - Использован в разделах: Введение, Раздел 5.
5. ASP.NET Core documentation [Электронный ресурс] / Microsoft Corporation. - 2024. - Режим доступа: <https://docs.microsoft.com/aspnet/core> (дата обращения: 01.06.2026). - Использован в разделах: 2, 3.
6. Entity Framework Core documentation [Электронный ресурс] / Microsoft Corporation. - 2024. - Режим доступа: <https://learn.microsoft.com/ef/core> (дата обращения: 01.06.2026). - Использован в разделах: 2, 3.
7. i18next documentation [Электронный ресурс] / i18next contributors. - 2024. - Режим доступа: <https://www.i18next.com> (дата обращения: 01.06.2026). - Использован в разделах: 2, 3.
8. Jones, M. B. JSON Web Token (JWT) [Текст] : RFC 7519 / M. B. Jones, J. Bradley, N. Sakimura. - IETF, 2015. - Режим доступа: <https://tools.ietf.org/html/rfc7519> (дата обращения: 01.06.2026). - Использован в разделах: 2, 3, 5.
9. MediatR. Simple mediator implementation in .NET [Электронный ресурс] / J. Bogard. - 2024. - Режим доступа: <https://github.com/jbogard/MediatR/wiki> (дата обращения: 01.06.2026). - Использован в разделах: 2, 3.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						63
Изм.	Лист	№ докум.	Подпись	Дата		

10. OpenAPI Specification, Version 3.1.0 [Электронный ресурс] / OpenAPI Initiative. - 2021. - Режим доступа: <https://swagger.io/specification> (дата обращения: 01.06.2026). - Использован в Разделе 2.
11. PostgreSQL 16 Documentation [Электронный ресурс] / The PostgreSQL Global Development Group. - 2024. - Режим доступа: <https://www.postgresql.org/docs/16/> (дата обращения: 01.06.2026). - Использован в разделах: 2, 3, 5.
12. React. The library for web and native user interfaces [Электронный ресурс] / Meta Platforms, Inc. - 2024. - Режим доступа: <https://react.dev> (дата обращения: 01.06.2026). - Использован в разделах: 2, 3.
13. Redux Toolkit. The official, opinionated, batteries-included toolset for efficient Redux development [Электронный ресурс] / Redux contributors. - 2024. - Режим доступа: <https://redux-toolkit.js.org> (дата обращения: 01.06.2026). - Использован в разделах: 2, 3.
14. Vite. Next generation frontend tooling [Электронный ресурс] / VoidZero Inc. - 2024. - Режим доступа: <https://vite.dev> (дата обращения: 01.06.2026). - Использован в Разделе 2.

ПРИЛОЖЕНИЯ

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						65
Изм.	Лист	№ докум.	Подпись	Дата		

Листинги основных модулей

Таблица А.1 Перечень листингов

Обозначение	Файл	Содержание
A.1	LoginCommand.cs	Генерация JWT-токена: claims, HMAC-SHA256, проверка IsDeleted
A.2	CheckAvailabilityQuery.cs	Алгоритм обнаружения конфликтов расписания (LINQ + EF Core)
A.3	CreateMeetingCommand.cs	Валидация форматов встречи и создание записи
A.4	Program.cs	Авторегистрация IFeatureEndpoint через reflection
A.5	authSlice.ts	loginThunk, hydrateSessionFromStorage (Redux Toolkit)
A.6	i18n.ts	Конфигурация локализации: detectLanguage, resources ru/kk
A.7	DatabaseContext.cs	Конфигурация EF Core: схемы, каскады, индексы

Листинг А.1 - LoginCommand.cs (фрагмент: проверка пользователя и генерация JWT)

```
public class LoginCommand : IFeatureEndpoint
{
    public void Map(WebApplication app) =>
        app.MapPost("/api/v1/auth/login", Handle)
            .AllowAnonymous();

    public record Request(string Email, string Password);
    public record Response(string Token);

    public class Handler(
        UserManager<Domain.User> userManager,
        SignInManager<Domain.User> signInManager,
        IOptions<AuthSettings> authOptions,
        TimeProvider timeProvider)
        : IRequestHandler<Request, OneOf<Response, Error>>
    {
        private readonly AuthSettings _authOptions = authOptions.Value;
        private readonly TimeProvider _timeProvider = timeProvider;

        public async Task<OneOf<Response, Error>> Handle(
            Request request, CancellationToken ct)
        {
            var user = await userManager.FindByEmailAsync(request.Email);
            if (user is null)
                return ApiErrors.BadRequest.Instance;

            // Блокировка удаленных аккаунтов - HTTP 406
            if (user.IsDeleted)
                return ApiErrors.NotAcceptable.Instance;

            var result = await signInManager
                .CheckPasswordSignInAsync(user, request.Password, false);
            if (!result.Succeeded)
                return ApiErrors.BadRequest.Instance;
        }
    }
}
```



```

        var roles = await userManager.GetRolesAsync(user);
        var token = GenerateJwtToken(user, roles);
        return new Response(token);
    }

    private string GenerateJwtToken(
        Domain.User user, IEnumerable<string> roles)
    {
        var currentDate = _timeProvider.GetUtcNow();
        var claims = new List<Claim>
        {
            new(JwtClaimTypes.Subject, user.Id.ToString()),
            new(JwtClaimTypes.IssuedAt,
                currentDate.ToUnixTimeSeconds().ToString()),
        };
        claims.AddRange(
            roles.Select(role => new Claim(JwtClaimTypes.Role, role)));

        var jwt = new JwtSecurityToken(
            issuer: _authOptions.Issuer,
            audience: _authOptions.Audience,
            claims: claims,
            expires: currentDate.Add(_authOptions.Lifetime).UtcDateTime,
            signingCredentials: new SigningCredentials(
                _authOptions.GetSymmetricSecurityKey(),
                SecurityAlgorithms.HmacSha256));

        return new JwtSecurityTokenHandler().WriteToken(jwt);
    }
}

```

Листинг А.2 - CheckAvailabilityQuery.cs (фрагмент: LINQ-запросы проверки конфликтов)

```

public class CheckAvailabilityQuery : IFeatureEndpoint
{
    public void Map(WebApplication app) =>
        app.MapPost("/api/v1/meetings/availability", Handle)
            .RequireAuthorization(AuthorizationPolicy.UserPolicy);

    public record QueryModel(
        DateTime StartTime,
        DateTime EndTime,
        List<Guid> ParticipantIds);

    public record Request(QueryModel QueryModel);
    public record ConflictInfo(
        Guid UserId, string UserName,
        Guid ConflictingMeetingId, string ConflictingMeetingTitle,
        DateTime ConflictStartTime, DateTime ConflictEndTime);
    public record Response(bool AllAvailable, List<ConflictInfo> Conflicts);

    public class Handler(DatabaseContext db)
        : IRequestHandler<Request, Response>
    {
        public async Task<Response> Handle(
            Request request, CancellationToken ct)

```

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						67
Изм.	Лист	№ докум.	Подпись	Дата		

```

var conflicts = new List<ConflictInfo>();

foreach (var userId in request.QueryModel.ParticipantIds)
{
    var user = await db.Users.FindAsync(userId, ct);
    if (user is null || user.IsDeleted) continue;

    // Проверка как участник встречи
    var asParticipant = await db.MeetingParticipants
        .Include(mp => mp.Meeting)
        .Where(mp => mp.UserId == userId)
        .Where(mp => mp.Meeting != null && !mp.Meeting.IsDeleted)
        .Where(mp =>
            mp.Meeting!.Status != MeetingStatus.Cancelled &&
            mp.Meeting.Status != MeetingStatus.Draft)
        // Формула пересечения интервалов:
        // newStart < existingEnd AND newEnd > existingStart
        .Where(mp =>
            request.QueryModel.StartTime < mp.Meeting!.EndTime &&
            request.QueryModel.EndTime > mp.Meeting.StartTime)
        .Select(mp => new ConflictInfo(
            userId,
            user.Email ?? string.Empty,
            mp.MeetingId,
            mp.Meeting!.Title,
            mp.Meeting.StartTime,
            mp.Meeting.EndTime))
        .ToListAsync(ct);

    // Проверка как организатор встречи
    var asOrganizer = await db.Meetings
        .Where(m => m.OrganizerId == userId)
        .Where(m => !m.IsDeleted)
        .Where(m =>
            m.Status != MeetingStatus.Cancelled &&
            m.Status != MeetingStatus.Draft)
        .Where(m =>
            request.QueryModel.StartTime < m.EndTime &&
            request.QueryModel.EndTime > m.StartTime)
        .Select(m => new ConflictInfo(
            userId,
            user.Email ?? string.Empty,
            m.Id,
            m.Title,
            m.StartTime,
            m.EndTime))
        .ToListAsync(ct);

    conflicts.AddRange(asParticipant);
    conflicts.AddRange(asOrganizer);
}

return new Response(!conflicts.Any(), conflicts);
}
}
}

```

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						68
Изм.	Лист	№ докум.	Подпись	Дата		

Продолжение приложения А

Листинг А.3 - CreateMeetingCommand.cs (фрагмент: валидация формата и создание встречи)

```
public async Task<OneOf<Response, Error>> Handle(
    Request request, CancellationToken ct)
{
    // Валидация обязательных полей по формату
    if (request.Model.Format == MeetingFormat.Offline
        && string.IsNullOrEmpty(request.Model.Location))
        return ApiErrors.BadRequest.Instance;

    if (request.Model.Format == MeetingFormat.Online
        && string.IsNullOrEmpty(request.Model.MeetingLink))
        return ApiErrors.BadRequest.Instance;

    if (request.Model.Format == MeetingFormat.Hybrid &&
        (string.IsNullOrEmpty(request.Model.Location)
        || string.IsNullOrEmpty(request.Model.MeetingLink)))
        return ApiErrors.BadRequest.Instance;

    if (request.Model.Format == MeetingFormat.Phone
        && string.IsNullOrEmpty(request.Model.ContactInfo))
        return ApiErrors.BadRequest.Instance;

    var meeting = new Meeting
    {
        Id = Guid.NewGuid(),
        Title = request.Model.Title,
        Description = request.Model.Description,
        Date = request.Model.Date,
        StartTime = request.Model.StartTime,
        EndTime = request.Model.EndTime,
        OrganizerId = _currentUser.UserId,
        Format = request.Model.Format,
        Status = MeetingStatus.Draft,
        Location = request.Model.Location,
        MeetingLink = request.Model.MeetingLink,
        ContactInfo = request.Model.ContactInfo,
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow
    };

    await _db.Meetings.AddAsync(meeting, ct);

    // Приглашение участников
    if (request.Model.ParticipantIds?.Any() == true)
    {
        foreach (var participantId in request.Model.ParticipantIds)
        {
            var participant = new MeetingParticipant
            {
                Id = Guid.NewGuid(),
                MeetingId = meeting.Id,
                UserId = participantId,
                InvitationStatus = InvitationStatus.Pending,
                IsRequired = false,
                InvitedAt = DateTime.UtcNow
            };
            await _db.MeetingParticipants.AddAsync(participant, ct);
        }
    }
}
```

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						69
Изм.	Лист	№ докум.	Подпись	Дата		

```

        // Email-уведомление каждому приглашенному
        var user = await _db.Users.FindAsync(participantId, ct);
        if (user is { IsDeleted: false })
            await _emailService.SendInvitationAsync(user, meeting, ct);
    }

    await _db.SaveChangesAsync(ct);
    return new Response(meeting.Id);
}

```

Листинг А.4 - Program.cs (фрагмент: авторегистрация IFeatureEndpoint через reflection)

```

// Автоматическая регистрация всех эндпоинтов из сборки
var featureEndpoints = Assembly.GetExecutingAssembly()
    .GetTypes()
    .Where(t => typeof(IFeatureEndpoint).IsAssignableFrom(t)
        && !t.IsInterface
        && !t.IsAbstract)
    .Select(Activator.CreateInstance)
    .Cast<IFeatureEndpoint>();

foreach (var endpoint in featureEndpoints)
    endpoint.Map(app);

// IFeatureEndpoint.cs
public interface IFeatureEndpoint
{
    void Map(WebApplication app);
}

// Пример реализации в фиче:
// public class ListMeetingsQuery : IFeatureEndpoint
// {
//     public void Map(WebApplication app) =>
//         app.MapGet("/api/v1/meetings", Handle)
//             .RequireAuthorization(AuthorizationPolicy.UserPolicy);
//     ...
// }

```

Листинг А.5 - authSlice.ts (loginThunk и hydrateSessionFromStorage)

```

import { createAsyncThunk, createSlice, PayloadAction } from '@reduxjs/toolkit';
import { authService } from '../http/authService';
import { jwtService } from '../utils/jwt';
import type { AppThunk } from '../store';
import type { CurrentUser, LoginRequest } from '../types';

interface AuthState {
    token: string | null;
    user: CurrentUser | null;
    isAuthenticated: boolean;
    isLoading: boolean;
    error: string | null;
}

const initialState: AuthState = {
    token: null,

```

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						70
Изм.	Лист	№ докум.	Подпись	Дата		

```

    user: null,
    isAuthenticated: false,
    isLoading: false,
    error: null,
  };

  // Асинхронный вход в систему
  export const loginThunk = createAsyncThunk(
    'auth/login',
    async (credentials: LoginRequest, { rejectWithValue }) => {
      try {
        const response = await authService.login(credentials);
        jwtService.setToken(response.data.data.token);
        return response.data.data;
      } catch (error: any) {
        return rejectWithValue(
          error.response?.data?.data ?? 'Ошибка входа'
        );
      }
    }
  );

  // Восстановление сессии из localStorage без запроса к API
  export const hydrateSessionFromStorage =
    (): AppThunk => (dispatch) => {
      const token = jwtService.getToken();
      if (token && !jwtService.isExpired(token)) {
        const session = jwtService.getSession(token);
        dispatch(authSlice.actions.setSession(session));
      }
    };

  const authSlice = createSlice({
    name: 'auth',
    initialState,
    reducers: {
      setSession(state, action: PayloadAction<CurrentUser>) {
        state.user = action.payload;
        state.token = jwtService.getToken();
        state.isAuthenticated = true;
      },
      logout(state) {
        state.token = null;
        state.user = null;
        state.isAuthenticated = false;
        jwtService.removeToken();
      },
    },
    extraReducers: (builder) => {
      builder
        .addCase(loginThunk.pending, (state) => {
          state.isLoading = true;
          state.error = null;
        })
        .addCase(loginThunk.fulfilled, (state, action) => {
          state.isLoading = false;
          state.isAuthenticated = true;
          state.token = jwtService.getToken();
          state.user = action.payload;
        })
    }
  });

```

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						71
Изм.	Лист	№ докум.	Подпись	Дата		

```

        .addCase(loginThunk.rejected, (state, action) => {
            state.isLoading = false;
            state.error      = action.payload as string;
        });
    },
});

export const { logout } = authSlice.actions;
export default authSlice.reducer;

```

Листинг А.6 - i18n.ts (конфигурация локализации)

```

import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import ruTranslations from './locales/ru.json';
import kkTranslations from './locales/kk.json';

const LANGUAGE_KEY = 'app-language';

function detectLanguage(): string {
    const saved = localStorage.getItem(LANGUAGE_KEY);
    if (saved && ['ru', 'kk'].includes(saved)) return saved;
    return navigator.language.startsWith('kk') ? 'kk' : 'ru';
}

i18n
    .use(initReactI18next)
    .init({
        lng: detectLanguage(),
        fallbackLng: 'ru',
        resources: {
            ru: { translation: ruTranslations },
            kk: { translation: kkTranslations },
        },
        interpolation: {
            escapeValue: false, // React сам экранирует XSS
        },
    });

// Сохранение выбранного языка при переключении
i18n.on('languageChanged', (lng) => {
    localStorage.setItem(LANGUAGE_KEY, lng);
});

export default i18n;

```

Листинг А.7 - DatabaseContext.cs (фрагмент: конфигурация схем, каскадов и индексов)

```

public class DatabaseContext
    : IdentityDbContext<User, Role, Guid,
        IdentityUserClaim<Guid>, UserRole,
        IdentityUserLogin<Guid>, IdentityRoleClaim<Guid>,
        IdentityUserToken<Guid>>
{
    public DatabaseContext(DbContextOptions<DatabaseContext> options)
        : base(options) { }

    public DbSet<Meeting> Meetings => Set<Meeting>();
}

```

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						72
Изм.	Лист	№ докум.	Подпись	Дата		

```

public DbSet<MeetingParticipant> MeetingParticipants => Set<MeetingParticipa
nt>();
public DbSet<MeetingFile> MeetingFiles => Set<MeetingFile>();
public DbSet<Message> Messages => Set<Message>();

protected override void OnModelCreating(ModelBuilder builder)
{
    base.OnModelCreating(builder);

    // Все Identity-таблицы - в схему identity
    builder.Entity<User>().ToTable("Users", "identity");
    builder.Entity<Role>().ToTable("Roles", "identity");
    builder.Entity<UserRole>().ToTable("UserRoles", "identity");
    builder.Entity<IdentityUserClaim<Guid>>()
        .ToTable("UserClaims", "identity");
    builder.Entity<IdentityUserLogin<Guid>>()
        .ToTable("UserLogins", "identity");
    builder.Entity<IdentityUserToken<Guid>>()
        .ToTable("UserTokens", "identity");
    builder.Entity<IdentityRoleClaim<Guid>>()
        .ToTable("RoleClaims", "identity");

    // Конфигурация Meeting
    builder.Entity<Meeting>(e =>
    {
        e.HasKey(m => m.Id);
        e.Property(m => m.Title).HasMaxLength(255).IsRequired();
        e.Property(m => m.Location).HasMaxLength(500);
        e.Property(m => m.MeetingLink).HasMaxLength(500);
        e.Property(m => m.ContactInfo).HasMaxLength(255);
        e.Property(m => m.IsDeleted).HasDefaultValue(false);

        e.HasOne(m => m.Organizer)
            .WithMany()
            .HasForeignKey(m => m.OrganizerId)
            .OnDelete(DeleteBehavior.Restrict);

        e.HasOne(m => m.DeletedByUser)
            .WithMany()
            .HasForeignKey(m => m.DeletedBy)
            .OnDelete(DeleteBehavior.SetNull);

        e.HasIndex(m => m.OrganizerId)
            .HasDatabaseName("IX_Meetings_OrganizerId");
        e.HasIndex(m => m.Status)
            .HasDatabaseName("IX_Meetings_Status");
        e.HasIndex(m => new { m.IsDeleted, m.Status })
            .HasDatabaseName("IX_Meetings_IsDeleted_Status");
    });

    // Конфигурация MeetingParticipant
    builder.Entity<MeetingParticipant>(e =>
    {
        e.HasKey(mp => mp.Id);
        e.HasIndex(mp => new { mp.MeetingId, mp.UserId })
            .IsUnique()
            .HasDatabaseName("IX_MeetingParticipants_MeetingId_UserId");
        e.HasIndex(mp => mp.InvitationStatus)
            .HasDatabaseName("IX_MeetingParticipants_InvitationStatus");
    });

```

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						73
Изм.	Лист	№ докум.	Подпись	Дата		

```

        e.HasOne(mp => mp.Meeting)
            .WithMany(m => m.Participants)
            .HasForeignKey(mp => mp.MeetingId)
            .OnDelete(DeleteBehavior.Cascade);

        e.HasOne(mp => mp.User)
            .WithMany()
            .HasForeignKey(mp => mp.UserId)
            .OnDelete(DeleteBehavior.Cascade);
    });

    // Конфигурация User (расширение Identity)
    builder.Entity<User>(e =>
    {
        e.Property(u => u.Name).IsRequired();
        e.Property(u => u.IsDeleted).HasDefaultValue(false);
        e.HasIndex(u => u.IsDeleted)
            .HasDatabaseName("IX_Users_IsDeleted");
    });
}
}

```

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						74
Изм.	Лист	№ докум.	Подпись	Дата		

Структура базы данных

Б.1 SQL-скрипты создания бизнес-таблиц

Таблица Meetings

```
CREATE TABLE public."Meetings" (
    "Id"          uuid          NOT NULL DEFAULT gen_random_uuid(),
    "Title"       varchar(255)  NOT NULL,
    "Description" text,
    "Date"        timestampz    NOT NULL,
    "StartTime"   timestampz    NOT NULL,
    "EndTime"     timestampz    NOT NULL,
    "OrganizerId" uuid          NOT NULL,
    "Format"      integer       NOT NULL DEFAULT 0,
    "Status"      integer       NOT NULL DEFAULT 0,
    "Location"    varchar(500),
    "MeetingLink" varchar(500),
    "ContactInfo" varchar(255),
    "IsDeleted"   boolean       NOT NULL DEFAULT false,
    "DeletedAt"   timestampz,
    "DeletedBy"   uuid,
    "CreatedAt"   timestampz    NOT NULL,
    "UpdatedAt"   timestampz    NOT NULL,
    CONSTRAINT "PK_Meetings"
        PRIMARY KEY ("Id"),
    CONSTRAINT "FK_Meetings_Users_OrganizerId"
        FOREIGN KEY ("OrganizerId")
        REFERENCES identity."Users"("Id") ON DELETE RESTRICT,
    CONSTRAINT "FK_Meetings_Users_DeletedBy"
        FOREIGN KEY ("DeletedBy")
        REFERENCES identity."Users"("Id") ON DELETE SET NULL
);

CREATE INDEX "IX_Meetings_OrganizerId"
    ON public."Meetings"("OrganizerId");
CREATE INDEX "IX_Meetings_Status"
    ON public."Meetings"("Status");
CREATE INDEX "IX_Meetings_IsDeleted_Status"
    ON public."Meetings"("IsDeleted", "Status");
```

Таблица MeetingParticipants

```
CREATE TABLE public."MeetingParticipants" (
    "Id"          uuid          NOT NULL DEFAULT gen_random_uuid(),
    "MeetingId"   uuid          NOT NULL,
    "UserId"      uuid          NOT NULL,
    "InvitationStatus" integer   NOT NULL DEFAULT 0,
    "IsRequired"  boolean       NOT NULL DEFAULT false,
    "InvitedAt"   timestampz    NOT NULL,
    "ResponseAt"  timestampz,
    "Comment"     text,
    CONSTRAINT "PK_MeetingParticipants"
        PRIMARY KEY ("Id"),
    CONSTRAINT "UQ_MeetingParticipants_MeetingId_UserId"
        UNIQUE ("MeetingId", "UserId"),
    CONSTRAINT "FK_MeetingParticipants_Meetings"
        FOREIGN KEY ("MeetingId")
        REFERENCES public."Meetings"("Id") ON DELETE CASCADE,
    CONSTRAINT "FK_MeetingParticipants_Users"
        FOREIGN KEY ("UserId")
        REFERENCES identity."Users"("Id") ON DELETE CASCADE
);
```

);

```
CREATE INDEX "IX_MeetingParticipants_InvitationStatus"
ON public."MeetingParticipants" ("InvitationStatus");
```

Таблица MeetingFiles

```
CREATE TABLE public."MeetingFiles" (
    "Id" uuid NOT NULL DEFAULT gen_random_uuid(),
    "MeetingId" uuid NOT NULL,
    "FileName" varchar(255) NOT NULL,
    "FilePath" varchar(500) NOT NULL,
    "FileType" varchar(100) NOT NULL,
    "UploadedById" uuid NOT NULL,
    "UploadedAt" timestamptz NOT NULL,
    CONSTRAINT "PK_MeetingFiles"
        PRIMARY KEY ("Id"),
    CONSTRAINT "FK_MeetingFiles_Meetings"
        FOREIGN KEY ("MeetingId")
        REFERENCES public."Meetings" ("Id") ON DELETE CASCADE,
    CONSTRAINT "FK_MeetingFiles_Users"
        FOREIGN KEY ("UploadedById")
        REFERENCES identity."Users" ("Id") ON DELETE RESTRICT
);
```

```
CREATE INDEX "IX_MeetingFiles_MeetingId"
ON public."MeetingFiles" ("MeetingId");
```

Таблица Messages

```
CREATE TABLE public."Messages" (
    "Id" uuid NOT NULL DEFAULT gen_random_uuid(),
    "RecipientId" uuid NOT NULL,
    "Subject" varchar(255) NOT NULL,
    "Body" text NOT NULL,
    "SentAt" timestamptz NOT NULL,
    "MeetingId" uuid,
    CONSTRAINT "PK_Messages"
        PRIMARY KEY ("Id"),
    CONSTRAINT "FK_Messages_Users"
        FOREIGN KEY ("RecipientId")
        REFERENCES identity."Users" ("Id") ON DELETE CASCADE,
    CONSTRAINT "FK_Messages_Meetings"
        FOREIGN KEY ("MeetingId")
        REFERENCES public."Meetings" ("Id") ON DELETE SET NULL
);
```

```
CREATE INDEX "IX_Messages_RecipientId"
ON public."Messages" ("RecipientId");
```

```
CREATE INDEX "IX_Messages_SentAt"
ON public."Messages" ("SentAt");
```

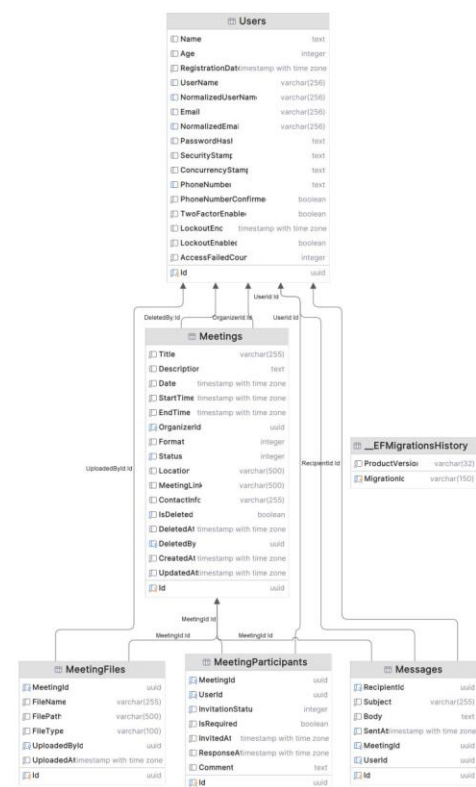
Б.2 Identity-схема

Таблицы ASP.NET Core Identity расположены в схеме identity:

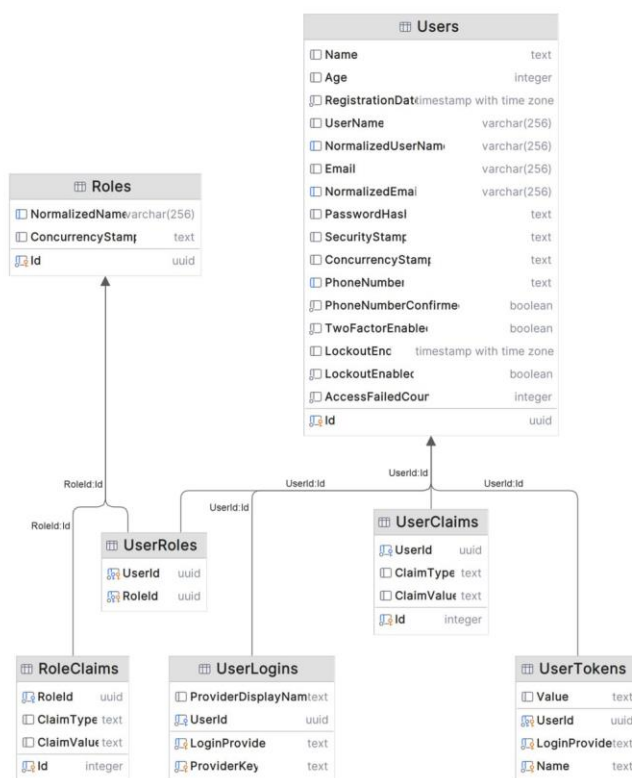
Таблица	Назначение
identity.Users	Учетные записи пользователей (расширенные полями Name, Age, IsDeleted и др.)
identity.Roles	Роли системы: Admin, Organizer, User
identity.UserRoles	Связь пользователь ↔ роль (M:N)
identity.UserClaims	Claims пользователей

identity.UserLogins	Внешние логины (OAuth)
identity.UserTokens	Токены сброса пароля, 2FA
identity.RoleClaims	Claims ролей

Б.3 ER-связи между сущностями



а) Бизнес-сущности (схема public)



б) Подсистема Identity (схема identity)

Скриншоты интерфейса

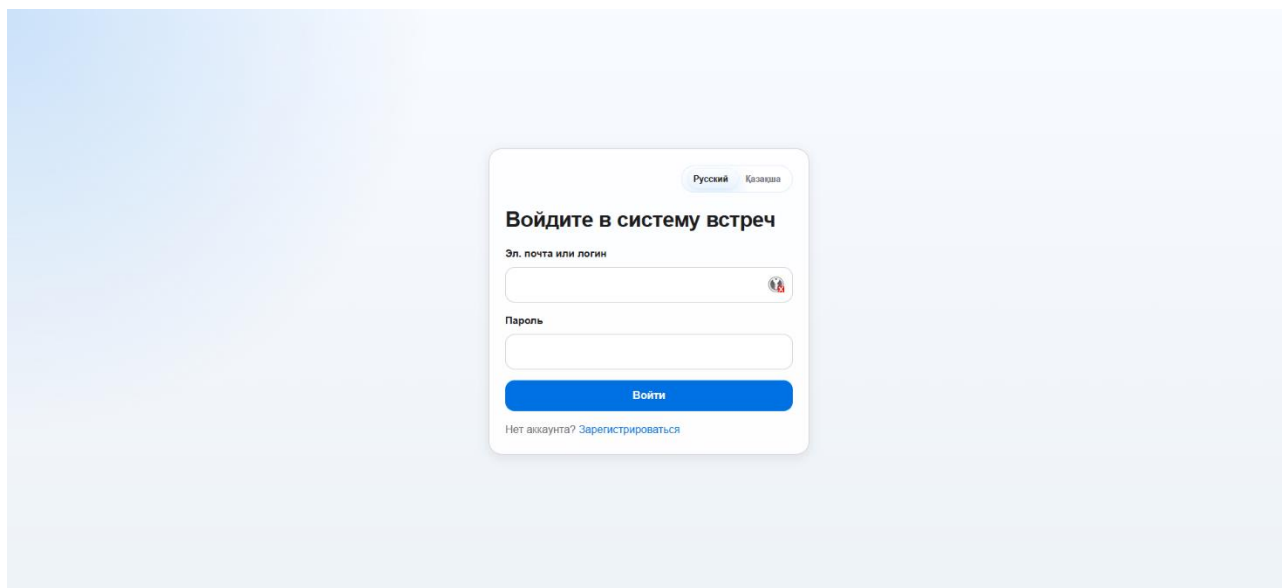


Рисунок В.1 - Страница входа (/auth/login, LoginPage)

Форма входа: поля «Email» и «Пароль», кнопка «Войти», переключатель языка в правом верхнем углу. Экран приводится на русском и казахском языках.

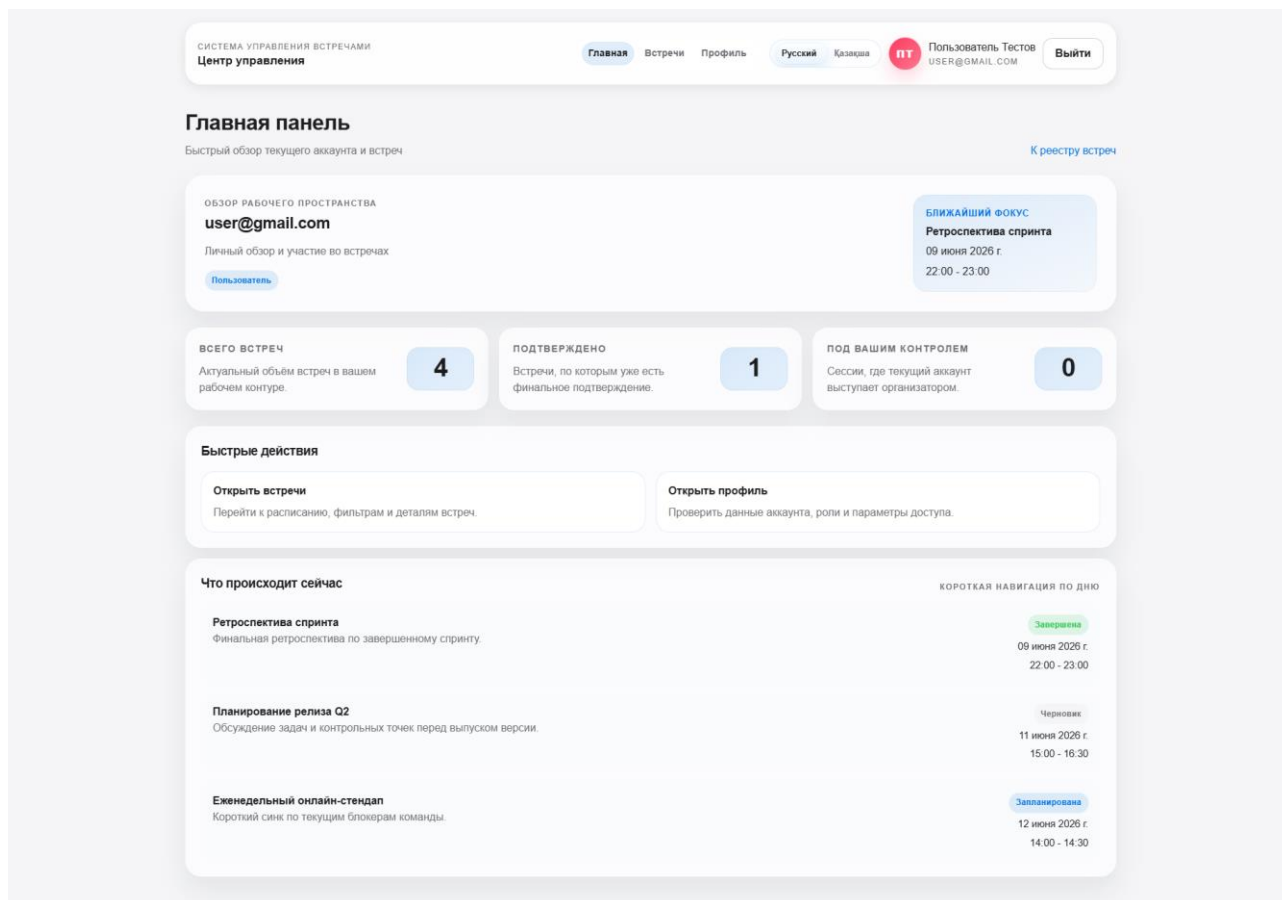


Рисунок В.2 - Главная панель, роль User (/ , DashboardPage)

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						78
Изм .	Лист	№ докум.	Подпись	Дата		

Виджет ближайшей встречи со статусом InvitationStatus = Pending, метрики «Всего встреч» и «Подтверждённых», список ближайших трёх встреч.

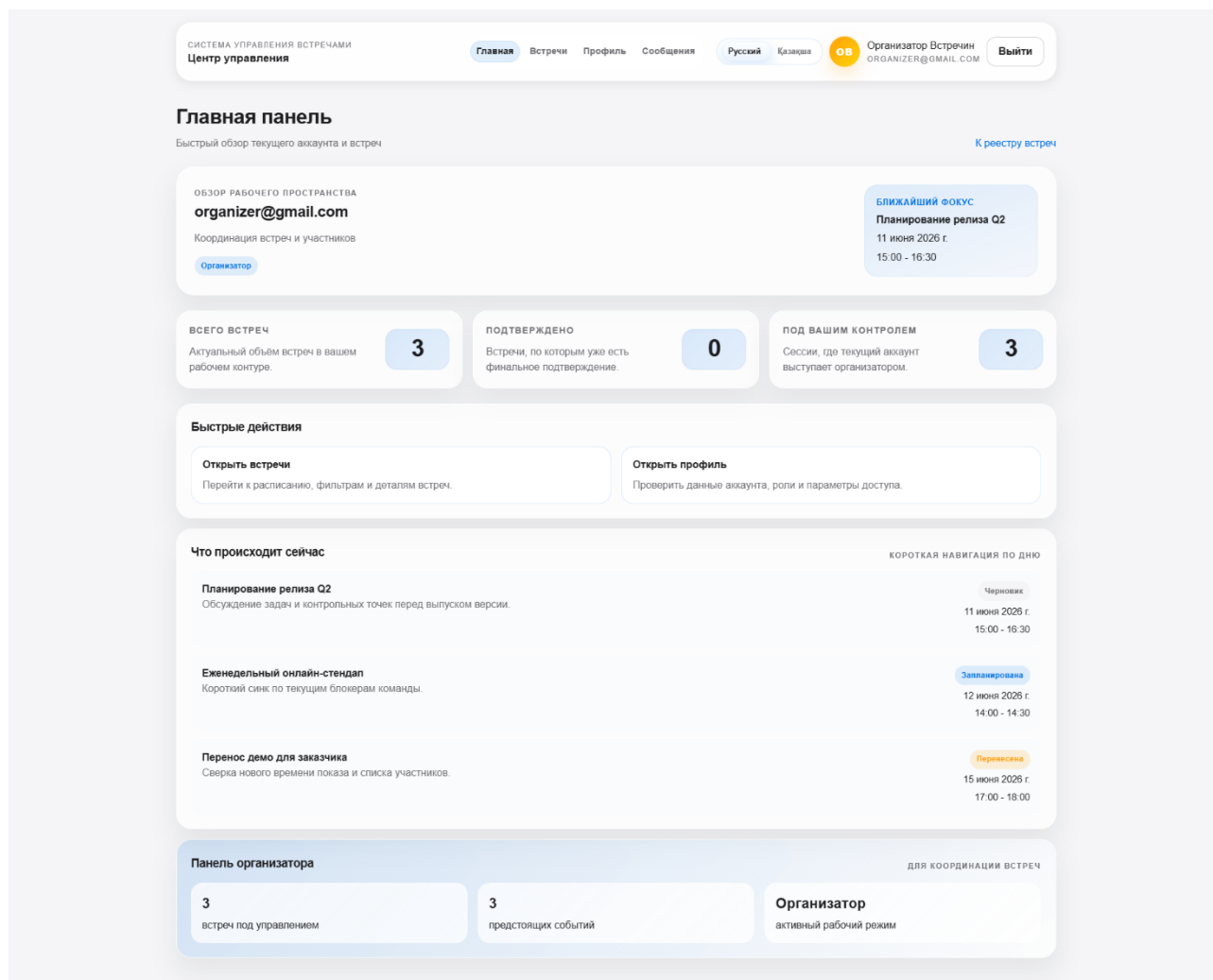


Рисунок В.3 - Главная панель, роль Organizer (/, DashboardPage)

Панель организатора: кнопка «Создать встречу», метрика «Управляемых встреч», список ближайших встреч и быстрые действия.

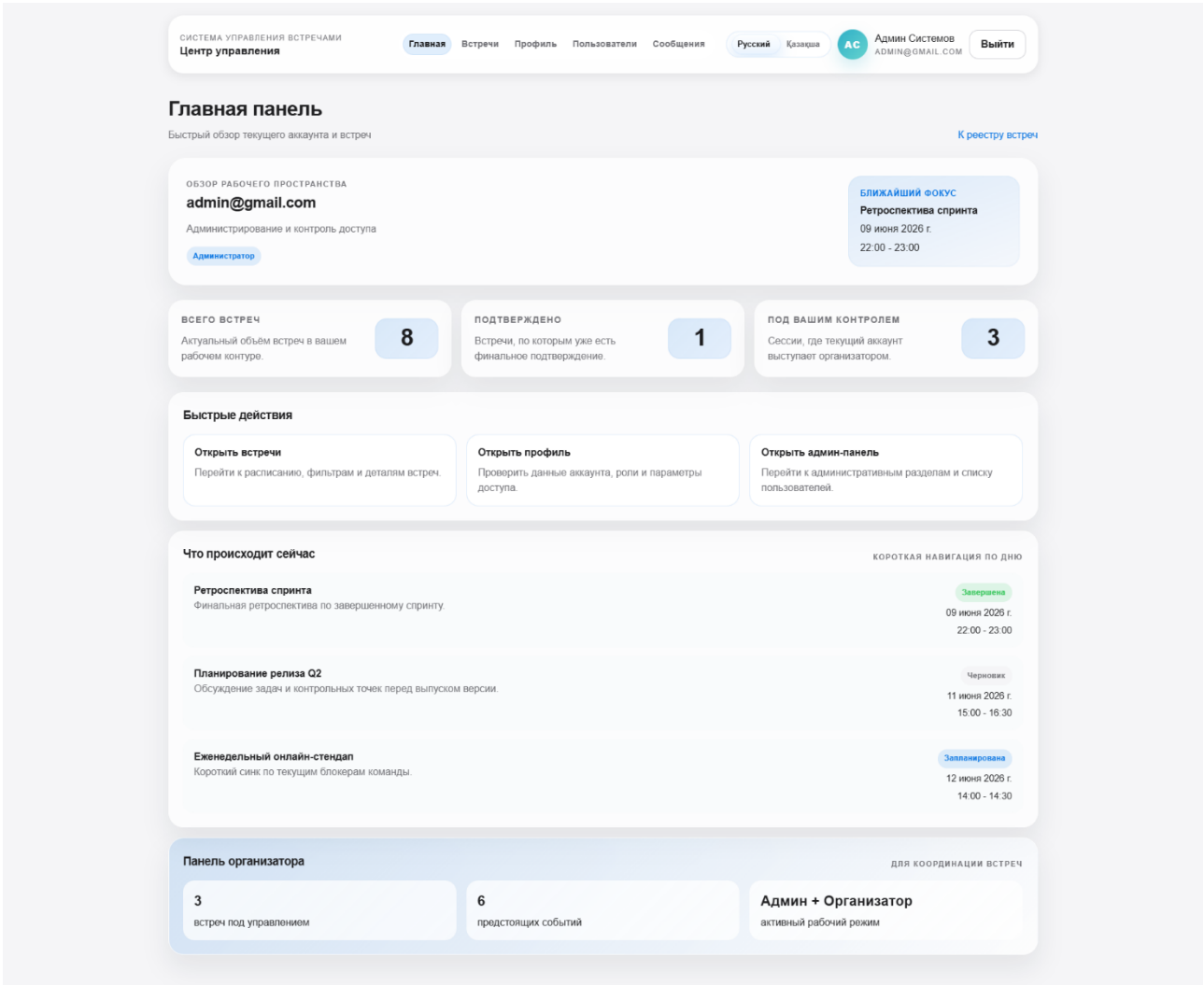


Рисунок В.4 - Панель администратора (/admin, AdminDashboardPage)

Административное меню со ссылками на разделы «Пользователи», «Удалённые пользователи» и «Журнал сообщений».

СИСТЕМА УПРАВЛЕНИЯ ВСТРЕЧАМИ
Центр управления

ГлавнаяВстречиПрофильПользователиСообщенияРусскийКазахшаACАдмин Системов
ADMIN@GMAIL.COMВыйти

Встречи

Список ваших встреч и операций с ними

Название встречи

Дата

Начало

Окончание

Формат

Локация

Ссылка на встречу

Контактная информация

Участники

Описание

Проверить занятость

Создать встречу

Поиск по встречам

Название, описание, статус, формат...

Фильтр по статусу

Фильтр по формату

Все статусы

Все форматы

Все встречи

Отменена

Офлайн

Отмененная встреча по аудиту

Сессия отменена из-за недоступности ключевых участников.

08 июня 2026 г.
16:00 - 17:00

Открыть

Завершена

Гибрид

Ретроспектива спринта

Финальная ретроспектива по завершеному спринту.

09 июня 2026 г.
22:00 - 23:00

Открыть

Ожидает подтверждения

Гибрид

Согласование бюджета проекта

Подтверждение бюджета и обсуждение рисков.

13 июня 2026 г.
19:00 - 20:00

Открыть

Перенесена

Онлайн

Перенос демо для заказчика

Сверка нового времени показа и списка участников.

15 июня 2026 г.
17:00 - 18:00

Открыть

Черновик

Офлайн

Планирование релиза Q2

Обсуждение задач и контрольных точек перед выпуском версии.

11 июня 2026 г.
15:00 - 16:30

Открыть

Запланирована

Онлайн

Еженедельный онлайн-стендап

Короткий синк по текущим блокерам команды.

12 июня 2026 г.
14:00 - 14:30

Открыть

Подтверждена

Телефон

Созвон с подрядчиком

Обсуждение интеграции и сроков поставок.

14 июня 2026 г.
21:00 - 21:45

Открыть

Рисунок В.5 - Список встреч (/sessions, SessionsPage)

Список карточек встреч, поле поиска, выпадающие фильтры по статусу и формату, кнопка «Создать встречу» (для ролей Organizer и Admin).

СИСТЕМА УПРАВЛЕНИЯ ВСТРЕЧАМИ
Центр управления

ГлавнаяВстречиПрофильПользователиСообщенияРусскийКазахшаACАдмин Системов
ADMIN@GMAIL.COMВыйти

Ретроспектива спринта

09 июня 2026 г. • 22:00 - 23:00

Гибрид

21

УЧАСТНИКОВФАЙЛОВ

ОПИСАНИЕ

Финальная ретроспектива по завершенному спринту.

ЛОКАЦИЯ

Комната Agile

ССЫЛКА НА ВСТРЕЧУ

https://meet.example.local/retro

КОНТАКТНАЯ ИНФОРМАЦИЯ

scrum-master@example.local

ДАТА И ВРЕМЯ

09 июня 2026 г. • 22:00 - 23:00

ФАЙЛЫ ВСТРЕЧИ

Материалы и вложения

Загрузить файл

retro-notes.txt

СкачатьУдалить

Название встречи

Ретроспектива спринта

Дата

09.06.2026

Начало

22:00

Окончание

23:00

Формат

Гибрид

Статус

Завершена

Локация

Комната Agile

Ссылка на встречу

https://meet.example.local/retro

Контактная информация

scrum-master@example.local

Описание

Финальная ретроспектива по завершенному спринту.

Удалить встречуСохранить изменения

Участники

2

user@gmail.com

Приглашён 04 июня 2026 г. в 15:39

ПринятоУдалить

business.analyst@gmail.com

Приглашён 04 июня 2026 г. в 15:39

ПринятоУдалить

Добавить участников

Выберите участников

Пригласить выбранных

Рисунок В.6 - Карточка встречи (/sessions/:id, SessionDetailPage)

Детальная страница встречи: заголовок, формат, дата и время, описание, список файлов с кнопками скачивания, панель участников с их статусами InvitationStatus.

СИСТЕМА УПРАВЛЕНИЯ ВСТРЕЧАМИ
Центр управления

Главная Встречи Профиль Пользователи Сообщения Русский Казахча А.С. Админ Системов ADMIN@GMAIL.COM Выйти

Название встречи

Дата

ДД.ММ.ГГГГ

Начало

Окончание

Формат

Офлайн

Локация

Ссылка на встречу

Контактная информация

Участники

Выберите участников

Описание

Проверить занятость Создать встречу

Рисунок В.7 - Форма встречи (MeetingForm)

Форма создания и редактирования встречи: поля Title, Format (выпадающий список), Date, StartTime, EndTime, условные поля Location / MeetingLink / ContactInfo и мультिवыбор участников.

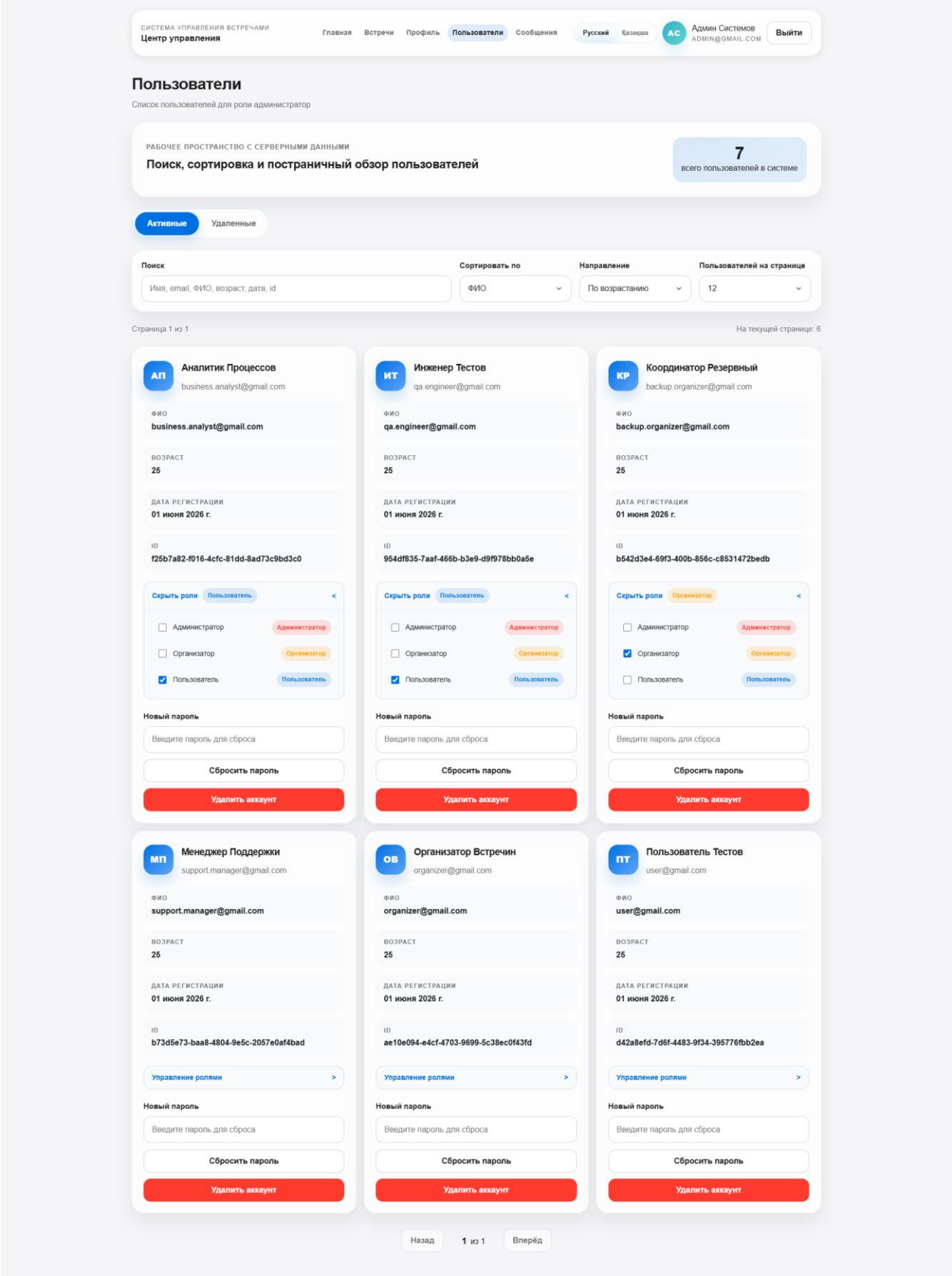


Рисунок В.8 - Управление пользователями (/admin/users, AdminUsersPage)

Продолжение приложения В

Таблица активных пользователей (имя, email, роли), раскрывающийся блок с чекбоксами ролей, кнопки «Сбросить пароль» и «Удалить».

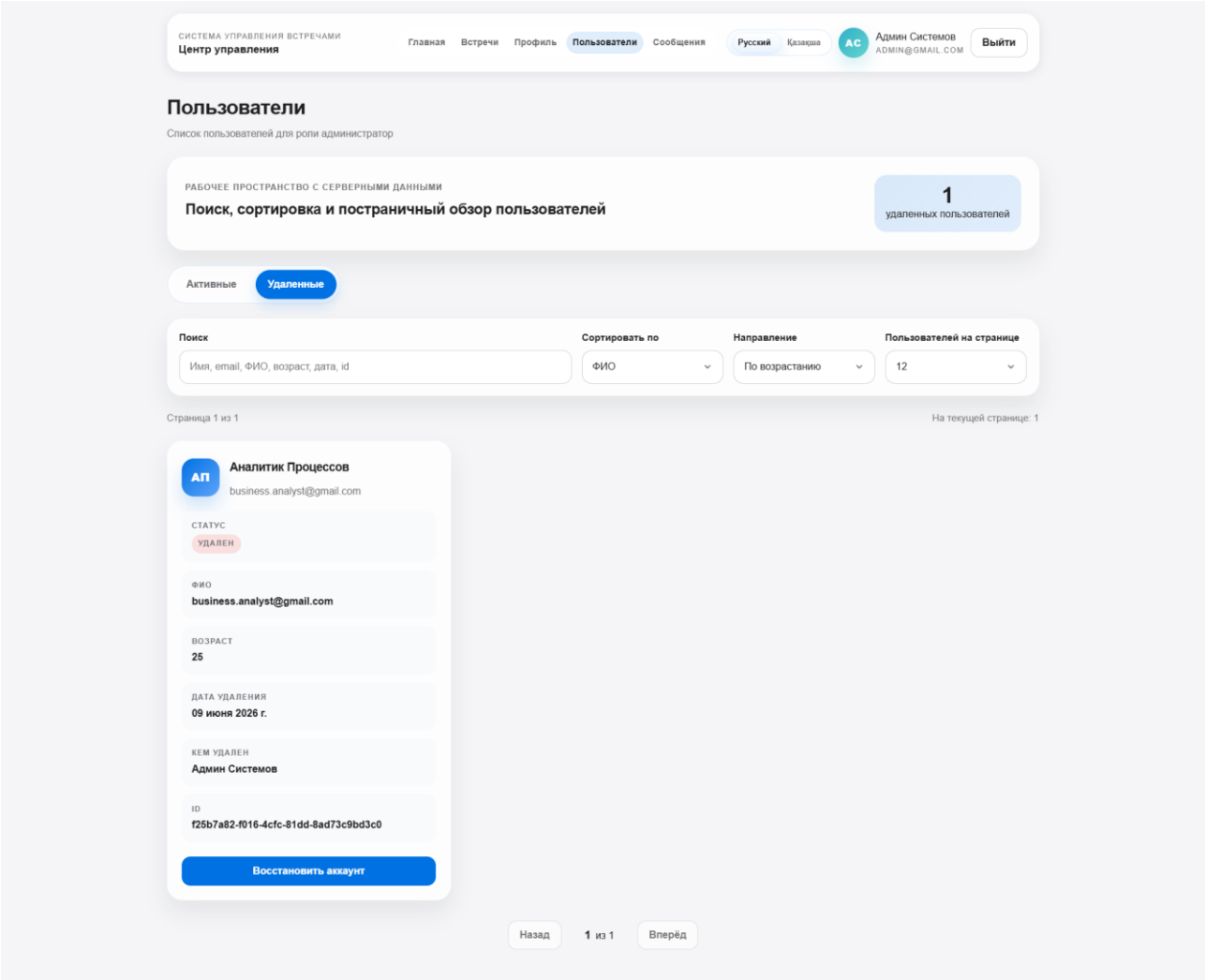


Рисунок В.9 - Удалённые пользователи (/admin/users/deleted, AdminDeletedUsersPage)

Список деактивированных пользователей: имя, email, дата удаления и кем удалён; кнопка «Восстановить».

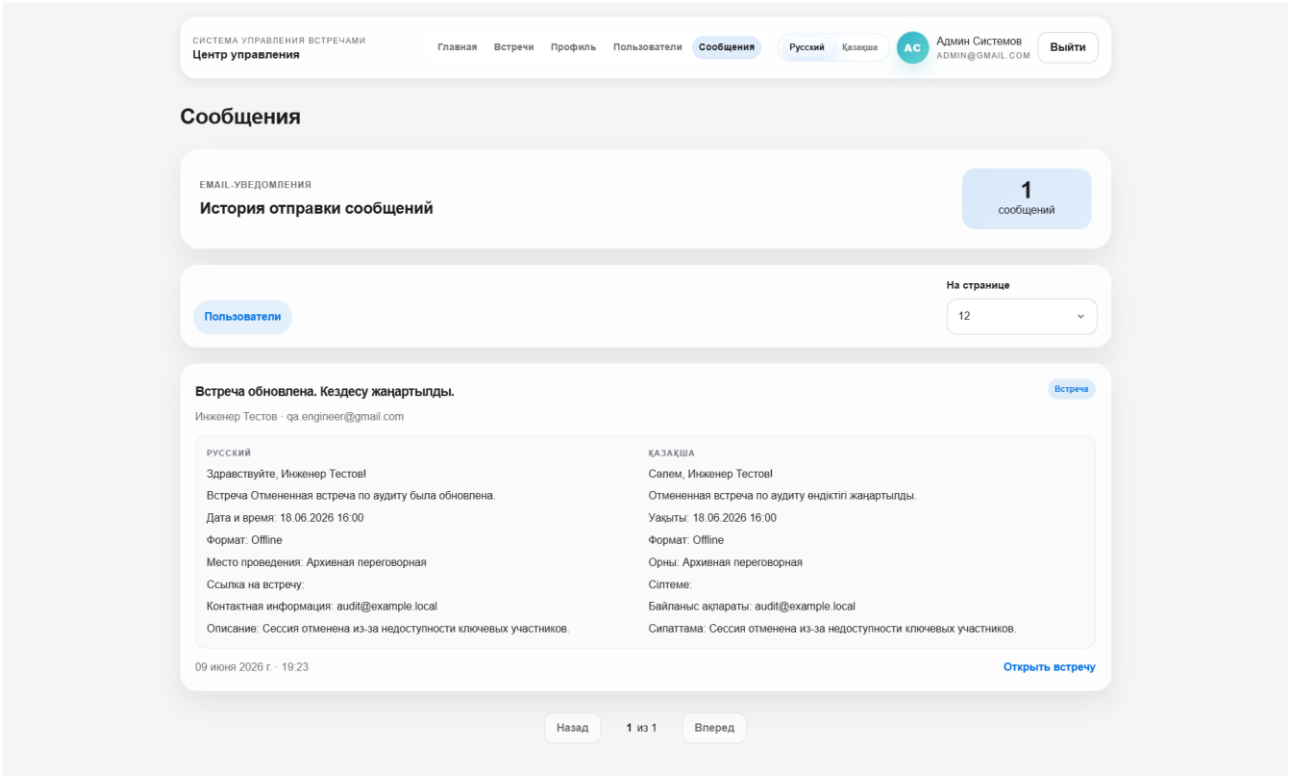


Рисунок В.10 - Журнал уведомлений (/admin/messages, AdminMessagesPage)

Журнал уведомлений: тема письма, получатель, дата, двуязычное тело письма (русский и казахский), ссылка на встречу.

КЕЗДЕСУЛЕРДІ БАСҚАРУ ЖҮЙЕСІ
Басқару орталығы

Басты бетКездесулерПрофильПайдаланушыларХабарламалар

РусскийҚазақша

АС
АДМИН СИСТЕМОВ
ADMIN@GMAIL.COM

Шығу

Кездесулер

Сіздің кездесулеріңіз бен өрекеттер тізімі

Кездесу атауы

Күні

Басталуы

Аяқталуы

Формат

Локация

Кездесу сілтемесі

Байланыс ақпараты

Қатысушылар

Сипаттама

Бос уақытты тексеру

Кездесу құру

Кездесулер бойынша іздеу

Атауы, сипаттамасы, мәртебесі, форматы...

Мәртебе бойынша сүзгі

Формат бойынша сүзгі

Барлық мәртебелер

Барлық форматтар

Барлық кездесулер

Расталған

Телефон

Созвон с подрядчиком

Обсуждение интеграции и сроков поставки.

14 маусым 2026

21:00 - 21:45

Ашу

Растауды күтуде

Гибрид

Согласование бюджета проекта

Подтверждение бюджета и обсуждение рисков.

13 маусым 2026

19:00 - 20:00

Ашу

Басталғы нұсқа

Офлайн

Планирование релиза Q2

Обсуждение задач и контрольных точек перед выпуском версии.

11 маусым 2026

15:00 - 16:30

Ашу

Қайта жоспарланған

Онлайн

Перенос демо для заказчика

Сверка нового времени показа и списка участников.

15 маусым 2026

17:00 - 18:00

Ашу

Жоспарланған

Онлайн

Еженедельный онлайн-стендап

Короткий синк по текущим блокерам команды.

12 маусым 2026

14:00 - 14:30

Ашу

Аяқталған

Гибрид

Ретроспектива спринта

Финальная ретроспектива по завершённом спринту

09 маусым 2026

22:00 - 23:00

Ашу

Жоспарланған

Офлайн

Отмененная встреча по аудиту

Сессия отменена из-за недоступности ключевых участников.

18 маусым 2026

16:00 - 17:00

Ашу

Рисунок В.11 - Список встреч на казахском языке (/sessions, язык kk)

Тот же экран списка встреч после переключения языка: все метки, кнопки и статусы отображаются на казахском языке.

ПРИЛОЖЕНИЕ Г

Расширенные протоколы тестирования

Таблица Г.1 Расширенный протокол функционального тестирования (17 сценариев)

№	Цель проверки	Исходное состояние	Действия	Ожидаемый результат	Фактический результат	Вывод
1	JWT-аутентификация	Не авторизован	POST /api/v1/auth/login: <u>admin@gmail.com</u> / Admin123!	HTTP 200, поле data.token содержит JWT, status: "Ok"	Токен получен, редирект на /	Выполнено
2	Блокировка удаленного аккаунта	IsDeleted=true у пользователя	POST /api/v1/auth/login с корректными данными	HTTP 406 Not Acceptable	HTTP 406 возвращен	Выполнено
3	Неверный пароль	Пользователь существует	POST /api/v1/auth/login с неверным паролем	HTTP 400 Bad Request	HTTP 400 возвращен	Выполнено
4	Валидация Online без ссылки	Organizer авторизован	POST /api/v1/meetings: Format=Online, MeetingLink не указан	HTTP 400, встреча не создана	HTTP 400 возвращен	Выполнено
5	Создание встречи формата Offline	Organizer авторизован	POST /api/v1/meetings: Format=Offline, Location="Переговорная №1"	HTTP 200, Status=Draft, Id присвоен	Встреча создана, email-уведомления отправлены	Выполнено
6	Обнаружение конфликта расписания	Встреча «Стендап» 09:00-09:30 существует	POST /api/v1/meetings/availability: Start=09:15, End=09:45, один участник	allAvailable: false, ConflictingMeetingTitle = «Стендап»	Конфликт определен корректно	Выполнено
7	Нет конфликта - все свободны	Встречи у участника в другое время	POST /api/v1/meetings/availability: Start=14:00, End=15:00	allAvailable: true, conflicts: []	Все свободны, пустой список конфликтов	Выполнено

№	Цель проверки	Исходное состояние	Действия	Ожидаемый результат	Фактический результат	Вывод
8	Мягкое удаление встречи	Встреча Status=Draft существует	DELETE /api/v1/meetings/{id} (организатор)	IsDeleted=true в БД, встреча не возвращается в GET /api/v1/meetings	Запись скрыта, данные сохранены	Выполнено
9	Ответ участника : принять	InvitationStatus=Pending	PATCH /api/v1/meetings/{id}/participants/{userId}/respond: status=Accepted	InvitationStatus=Accepted, ResponseAt заполнен	Статус обновлен	Выполнено
10	Автоподтверждение встречи	Последний IsRequired участник не ответил, статус=AwaitingConfirmation	PATCH .../respond: Accepted (последний обязательный)	Статус встречи автоматически → Confirmed	Переход выполнен без ручного действия	Выполнено
11	RBAC: User не может создать встречу	User авторизован	POST /api/v1/meetings	HTTP 403 Forbidden	HTTP 403 возвращен	Выполнено
12	Смена роли пользователя	Admin авторизован	PATCH /api/v1/users/{id}/roles: { roles: ["Organizer"] }	Роли в БД обновлены	Роль изменена, новый токен после логина содержит Organizer	Выполнено
13	Загрузка файла к встрече	Organizer авторизован, встреча существует	POST /api/v1/meetings/{id}/files: multipart, файл report.pdf	HTTP 200, запись в MeetingFiles, файл по пути files/meetings/{id}/...	Файл сохранен, доступен для скачивания	Выполнено

№	Цель проверки	Исходное состояние	Действия	Ожидаемый результат	Фактический результат	Вывод
14	Скачивание файла с Range-запросом	Файл загружен	GET /api/v1/meetings/{id}/files/{fileId}/download, заголовок Range: bytes=0-1023	HTTP 206 Partial Content, первые 1024 байта файла	Частичный ответ получен	Выполнено
15	Переключение языка и сохранение	Интерфейс на русском	LanguageSwitcher → kk; перезагрузка страницы	Все строки на казахском; после перезагрузки язык восстановлен из localStorage	Язык сохранен, интерфейс на казахском	Выполнено
16	Пагинация списка пользователей	Admin авторизован, 7 пользователей	GET /api/v1/users?page=1&limit=5	5 пользователей в data, поле totalCount=7	Корректная пагинация	Выполнено
17	Журнал email-уведомлений	Admin авторизован, встреча создана	GET /api/v1/messages?page=1&limit=10	Список сообщений с Subject, Body, RecipientId, SentAt	Уведомления отображены	Выполнено

Инструкция по запуску системы

Д.1 Предварительные требования

Перед запуском системы убедитесь, что на компьютере установлено:

Компонент	Минимальная версия	Проверка
.NET SDK	9.0	dotnet --version
Node.js	20.0+	node --version
npm	10.0+	npm --version
PostgreSQL	16	psql --version
Оперативная память	4 ГБ свободно	-

Следующие порты должны быть свободны:

Порт	Сервис
3000	Frontend (Vite dev server)
7262	Backend API / Swagger UI
5432	PostgreSQL 16

Д.2 Запуск системы

Шаг 1. Установить зависимости frontend:

```
cd frontend
npm install
cd ..
```

Шаг 2. Создать базу данных (если еще не создана):

```
createdb -U postgres meetings_db
```

Шаг 3. Запустить backend:

```
cd backend/TemplateProject
dotnet run
```

Консольный вывод показывает процесс применения миграций и создания seed-данных. После успешного запуска будет выведено сообщение о готовности сервера и URL: <http://localhost:7262>.

Шаг 4. Запустить frontend (в отдельном терминале):

```
cd frontend
npm run dev
```

После сборки frontend доступен по адресу <http://localhost:3000>.

Д.3 Доступ к системе

Ресурс	URL
Web-интерфейс (Frontend)	http://localhost:3000
REST API / Swagger UI	http://localhost:7262/swagger
База данных (прямое подключение)	localhost:5432, БД: meetings_db, user: postgres

Д.4 Автоматические миграции

При первом запуске backend выполняет все четыре миграции EF Core:

Порядок	Миграция	Что создает
1	20260310154043_Initial	Схема identity, все таблицы ASP.NET Identity

Порядок	Миграция	Что создает
2	20260508134322_AddedMainEntities	Meetings, MeetingParticipants, MeetingFiles
3	20260510200303_AddedNameProperty	Поле Name NOT NULL в Users
4	20260530181425_AddedSmtFunc	IsDeleted в Users, таблица Messages

Если миграции не применились с первой попытки (база еще не готова), механизм ApplyMigrationsWithRetryAsync повторяет попытку до 10 раз с экспоненциальной задержкой.

Д.5 Тестовые учетные данные

После первого запуска база автоматически наполняется seed-данными: 7 пользователей и 7 встреч со всеми статусами жизненного цикла.

Таблица Д.1 Тестовые учетные записи

Роль	Email	Пароль
Admin	admin@gmail.com	Admin123!
Organizer	organizer@gmail.com	Organizer123!
Organizer (резервный)	backup.organizer@gmail.com	BackupOrganizer123+
User	user@gmail.com	User123!
User (QA Engineer)	qa.engineer@gmail.com	QaEngineer123+
User (Business Analyst)	business.analyst@gmail.com	Analyst123+
User (Support Manager)	support.manager@gmail.com	Support123+

Таблица Д.2 Seed-встречи

Название	Организатор	Формат	Статус
Планирование релиза Q2	organizer@gmail.com	Offline	Draft
Еженедельный онлайн-стендап	organizer@gmail.com	Online	Scheduled
Согласование бюджета проекта	admin@gmail.com	Hybrid	AwaitingConfirmation
Созвон с подрядчиком	backup.organizer@gmail.com	Phone	Confirmed
Перенос демо для заказчика	organizer@gmail.com	Online	Rescheduled
Отмененная встреча по аудиту	admin@gmail.com	Offline	Cancelled
Ретроспектива спринта	backup.organizer@gmail.com	Hybrid	Completed

Д.6 Переменные окружения

Все настройки системы передаются через конфигурационный файл appsettings.json и переменные окружения. В производственной среде следующие значения обязательно заменить:

Переменная	Текущее значение (dev)	Требование для production
Auth__Key	standardsecret_keyfordevelopment!v0_101	Случайная строка ≥ 32 символа
Auth__Issuer	https://localhost:7262	Реальный домен backend
Auth__Audience	https://localhost:3000	Реальный домен frontend
Auth__Lifetime	05:00:00	По политике безопасности организации
AdminCredentials__Password	Admin123!	Безопасный пароль
SmtpSettings__Host	(пусто)	Реальный SMTP-сервер
SmtpSettings__Username	(пусто)	Email отправителя
SmtpSettings__Password	(пусто)	Пароль SMTP

Д.7 Остановка

Для остановки backend и frontend достаточно прервать их процессы (Ctrl+C в терминалах). Для остановки PostgreSQL - команда `pg_ctl stop` (или через службы операционной системы).

Для полной очистки данных удалите и пересоздайте базу данных:

```
dropdb -U postgres meetings_db
createdb -U postgres meetings_db
```

После этого при следующем запуске backend заново применит все миграции и создаст seed-данные.

ПРИЛОЖЕНИЕ Е

Доказательная база проекта

Доказательная база проекта формировалась непрерывно на протяжении всей разработки (с 4 мая по 10 июня 2026 года), а не подбиралась после написания пояснительной записки. Каждый артефакт создавался на своем этапе работы и зафиксирован в репозитории системы контроля версий Git либо в соответствующем разделе записки. В таблице Е.1 сведен полный состав доказательной базы с указанием места фиксации каждого элемента.

Таблица Е.1 Состав доказательной базы проекта

Элемент доказательной базы	Где зафиксирован в проекте
Backlog (перечень задач)	Введение (7 задач проекта), §1.5 (10 User Stories US-01-US-10), Таблица Е.2
Схема архитектуры	§2.2, Рисунок 2.1 (слоистая диаграмма backend / frontend / БД)
Диаграмма базы данных (ER)	§2.3, Рисунок 2.2; Приложение Б.3
Исходный код	§3.3 (листинги ключевых модулей), Приложение А; репозиторий Git
История коммитов	Таблица Е.3 (70 коммитов, 04.05-10.06.2026)
Пользовательские сценарии	§3.7 (4 сценария по ролям ORGANIZER, USER, ADMIN)
API-описание	§2.4 (32 эндпоинта); Swagger / OpenAPI: localhost:7262/swagger
Тестовые сценарии	§3.8, Таблица 3.3 (12 сценариев); Приложение Г
Скриншоты интерфейса	Приложение В
Демонстрация работы системы	§Е.3 (живая демонстрация на защите; сценарий защиты зафиксирован в репозитории)
SQL-скрипты	§3.4.4, Приложение Б (Б.1-Б.2)
README и инструкция запуска	§3.5, Приложение Д (Д.1-Д.7)

Е.1 Перечень задач (backlog)

Работа велась по списку задач, сформированному из функциональных требований раздела 1.5. Каждая задача доведена до состояния «Выполнено» и подтверждена тестовым сценарием раздела 3.8 (Таблица 3.3) и соответствующими коммитами.

Таблица Е.2 Перечень задач проекта (backlog)

ID	Задача (User Story)	Статус	Подтверждение
US-01	Создание встречи с валидацией формата	Выполнено	Тесты №3, №4
US-02	Проверка занятости участников (конфликты)	Выполнено	Тест №5
US-03	Ответ участника на приглашение	Выполнено	Тест №7
US-04	Мягкое удаление пользователя	Выполнено	Тесты №2, №6
US-05	Смена роли пользователя	Выполнено	Тест №11
US-06	Казахскоязычный интерфейс	Выполнено	Тест №10
US-07	Прикрепление файлов к встрече	Выполнено	Тест №12
US-08	Скачивание файла встречи	Выполнено	Тест №12

ID	Задача (User Story)	Статус	Подтверждение
US-09	Автоматическое подтверждение встречи	Выполнено	Тест №8
US-10	Журнал email-уведомлений	Выполнено	Тест №4

Е.2 Контроль версий и история коммитов

Исходный код размещен в публичном репозитории Git на платформе GitHub (ветка main), доступном по адресу: <https://github.com/clayforhome/diplom>. За период разработки выполнено 70 коммитов с 4 мая по 10 июня 2026 года. История коммитов отражает последовательность создания компонентов: от каркаса решения и базовых эндпоинтов до локализации, email-уведомлений и мягкого удаления, а также подготовку доказательной базы и инструкций запуска. В таблице Е.3 приведены ключевые вехи разработки по истории коммитов. Снимки экрана репозитория и истории коммитов приведены на рисунках Е.1 и Е.2.

Таблица Е.3 Ключевые вехи по истории коммитов

Дата	Веха разработки	Коммитов
04.05.2026	Инициализация решения, каркас backend (ASP.NET Core)	1
09-10.05.2026	Базовые эндпоинты пользователей и встреч, политики авторизации (RBAC)	15
11.05.2026	Модель User (поле Name), миграции EF Core, страницы встреч на frontend	17
12-14.05.2026	Загрузка и хранение файлов встреч	3
16-17.05.2026	Переход на UTC, роли, всплывающие окна, валидация, CORS, сценарий защиты	17
28.05.2026	Переключатель языка (локализация ru/kk)	2
31.05.2026	Email-уведомления, мягкое удаление пользователей	2
01.06.2026	Страница сообщений, восстановление пользователей, казахские переводы, проверки статуса удаленных	10
10.06.2026	Подготовка доказательной базы: снимки экрана интерфейса, файл README с инструкцией запуска	3
Итого	04.05-10.06.2026	70

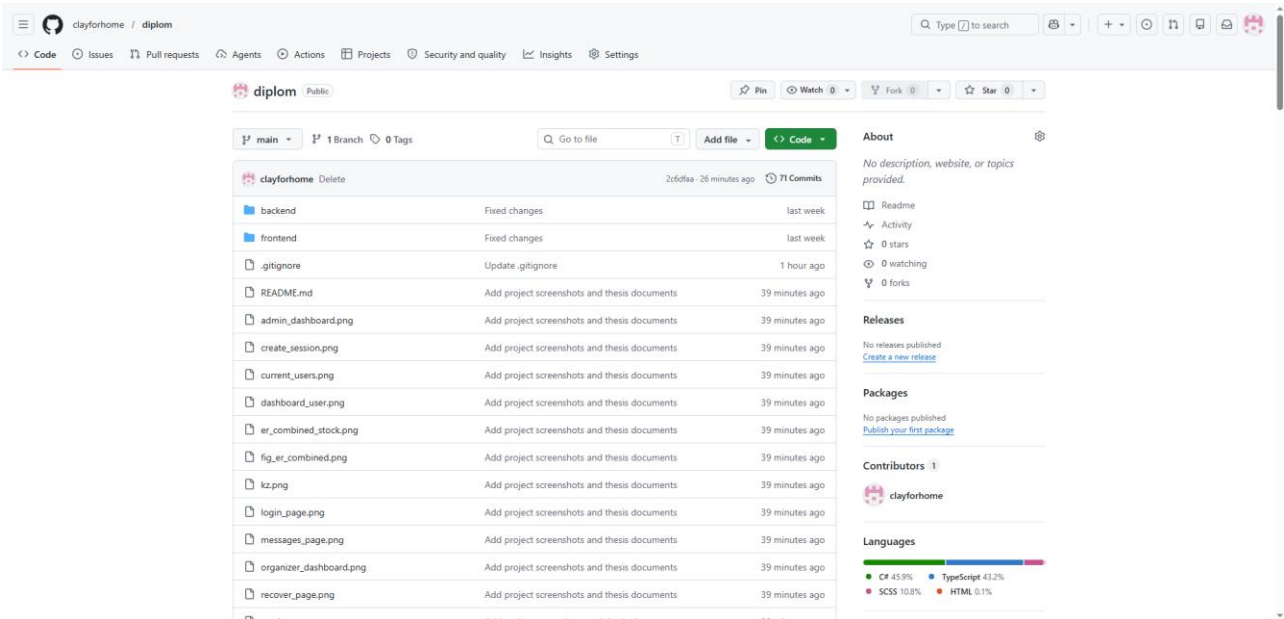


Рисунок Е.1 - Главная страница репозитория проекта на GitHub (clayforhome/diplom, ветка main)

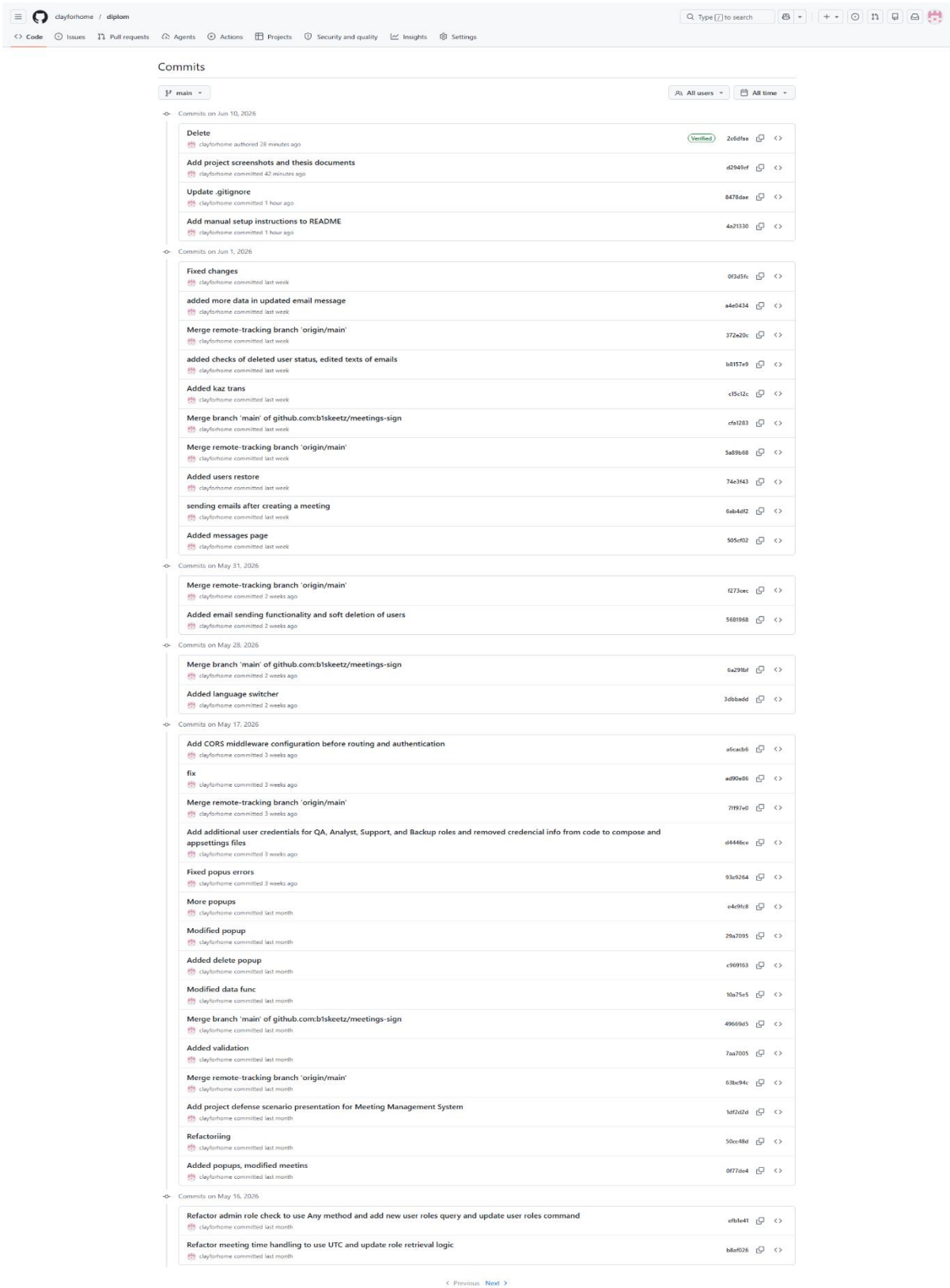


Рисунок Е.2 - История коммитов проекта (70 коммитов, 4 мая - 10 июня 2026 года)

Е.3 Демонстрация работы системы

Работа системы продемонстрирована в действии: запуск трех компонентов (база данных, backend, frontend), вход под ролью ORGANIZER, создание встречи, рассылка приглашений, ответ участника, автоматический переход статуса встречи и переключение языка интерфейса. Сценарий демонстрации соответствует пользовательским сценариям раздела 3.7. Сценарий защиты проекта зафиксирован в репозитории отдельным коммитом (Add project defense scenario presentation, 17.05.2026). Демонстрация работоспособности по этому сценарию выполняется вживую во время защиты на развернутом локальном стенде (база данных, backend и frontend), что подтверждает выполнение ключевых пользовательских сценариев.

					ДП.06130100.П-23-616.148.06.26.ПЗ	Лист
						98
Изм.	Лист	№ докум.	Подпись	Дата		